

RISC-V Mini Shell과 바이너리 실행: 프로세서 설계에서 운영체제까지

확장판 · RV32I ISA와 데이터패스 · SHELL_MODE · UART · syscall · MiniFS · 앱 실행

프로젝트	RISC-V/ (rtl, firmware, scripts, tb, constraints, docs)
대상 보드	AMD ZCU104 (xczu7ev-ffvc1156-2-e), PL RISC-V 12.5 MHz
Toolchain	RISC-V GNU Toolchain (-march=rv32i -mabi=ilp32), Vivado, Icarus Verilog
생성일	2026-09-30
원본	docs/SHELL_BINARY_GUIDE.md (scripts/build_manual_pdf.py로 생성)

ZCU104의 교육용 단일 사이클 RISC-V 코어에서 직접 만든 프로그램을 파일로 저장하고 실행하는 과정을 다루는 교재입니다.

기준일: 2026-09-30. 저장소의 RTL, C/어셈블리, linker script, 빌드 스크립트와 실제 생성된 ELF를 근거로 설명합니다.

대상: 디지털 논리와 C의 기초를 배운 컴퓨터구조·운영체제 수강생. 각 장은 원리, 현재 구현, 관찰 방법을 연결합니다.

확장판: 기존 각 절의 설명을 보강하고, 현재 코어의 ISA·데이터패스·CSR·사이클별 동작을 추가했습니다. 주소와 명령어 예제는 현재 RTL의 구현 범위 안에서 해석합니다.

목차

1. 학습 목표와 전체 실행 경로	6
1.1 이 실습에서 실제로 일어나는 일	6
1.2 학습 후 설명할 수 있어야 하는 것	6
1.3 여기서 말하는 앱과 프로세스	7
2. 최상위 회로와 SHELL_MODE의 설정	8
2.1 zcu104_top은 누가 호출하는가	8
2.2 동일한 이름을 사용하는 두 설정	8
2.3 make vivado-zcu104-shell의 전개	9
2.4 하드웨어 설정에 따른 메모리 선택	10
2.5 top에서 신호를 연결하는 실습	10
3. 바이너리 실행을 가능하게 하는 CPU와 메모리	12
3.1 단일 사이클 코어의 실행 경로	12
3.2 instruction port와 data port가 같은 RAM을 보는 이유	12
3.3 byte 주소와 word 인덱스	13
3.4 MMIO는 주소 디코딩이다	13
3.5 instruction cache가 생기면 추가되는 책임	14
3.6 현재 core의 architectural state와 내부 신호	14
3.7 RV32I와 46개 명령의 정확한 범위	14
3.8 register file과 ABI 이름	15
3.9 32비트 명령어의 공통 필드	16
3.10 R·I·S·B·U·J 형식	16
3.11 immediate 재구성과 부호 확장	16
3.12 ALU의 연산과 signed 해석	17
3.13 산술·논리 opcode의 해독	17
3.14 LOAD: 주소 계산에서 write-back까지	18
3.15 STORE: byte strobe와 데이터 복제	18
3.16 BRANCH: 조건과 target의 분리	19
3.17 JAL과 JALR: 함수 호출을 만드는 두 효과	19
3.18 LUI와 AUIPC: 큰 주소를 만드는 방법	20
3.19 CSR 명령 6종과 pseudo-instruction	20
3.20 구현된 CSR와 빠진 기능	21
3.21 decoder의 기본값과 write-back 선택	21
3.22 trap 우선순위와 정확한 부작용 억제	22
3.23 세 명령을 사이클별로 따라가기	22
3.24 timing, 합성 resource, 소프트웨어 비용	22
3.25 BNE 기계어를 비트 단위로 검사하기	23
3.26 load와 trap을 함께 보는 타이밍 표	23
4. 주소 공간과 세 가지 스택 개념	25
4.1 셸 이미지의 메모리 지도	25
4.2 부팅 스택과 셸 task 스택은 다르다	25
4.3 앱이 사용하는 스택	26
4.4 실행 이미지와 저장 파일의 수명 비교	26
5. 리셋에서 셸 프롬프트까지	28
5.1 구성 시 초기화와 CPU reset의 차이	28

5.2 boot.S의 초기화 순서	28
5.3 가짜 초기 trap frame으로 셸 시작하기	29
5.4 timer를 켜 뒤 나타나는 두 메시지	29
5.5 최초 task 시작의 PC·SP 변화표	30
6. UART에서 명령행까지	31
6.1 serial bit, byte, command line의 세 단계	31
6.2 uart_rx와 uart_tx는 어디에 있는가	31
6.3 FIFO와 polling	32
6.4 한 줄을 만드는 코드	32
6.5 built-in command와 외부 앱	33
6.6 문자 C의 serial frame을 손으로 그리기	33
7. ECALL과 운영체제 서비스의 경계	34
7.1 시스템 콜 번호는 OS가 정한다	34
7.2 한 문자를 출력하는 앱 코드	34
7.3 하드웨어 trap 진입	35
7.4 왜 f->mepc += 4인가	35
7.5 파일 읽기 syscall의 인수·결과 추적	36
8. Trap frame과 셸 모드의 스케줄링	37
8.1 frame의 정확한 메모리 형식	37
8.2 저장·C 호출·복원	37
8.3 일반 OS와 셸 모드의 scheduler 차이	38
8.4 frame에 모든 CSR이 들어 있지는 않다	38
9. MiniFS: 실행 파일을 보관하는 구조	40
9.1 RAM disk의 배치	40
9.2 file table entry	40
9.3 할당·읽기·삭제	41
9.4 60바이트 앱이 ls에서 72바이트인 이유	41
9.5 두 파일을 저장하고 삭제하는 전체 예	41
10. C 프로그램에서 실행 파일까지	43
10.1 세 종류의 이미지 구분	43
10.2 예제 C 프로그램	43
10.3 make hello-app의 빌드 단계	44
10.4 compiler 옵션의 의미	44
10.5 ELF에서 raw binary로 바뀌면서 사라지는 것	45
11. 링커와 앱 ABI의 계약	46
11.1 ENTRY와 실제 첫 바이트는 별개다	46
11.2 왜 다른 주소에 복사하면 안 되는가	46
11.3 앱 시작 때 이미 준비된 것	47
11.4 .data와 .bss	47
11.5 앱 간 연결과 외부 symbol	48
11.6 하위 함수를 호출하는 앱의 stack 예제	48
12. UART 업로드 프로토콜과 APP1 형식	50
12.1 host와 board의 대화	50
12.2 왜 raw byte 대신 ASCII hex를 보내는가	50
12.3 두 가지 hex 표현은 바이트 순서가 다르다	50
12.4 셸의 업로드 임시 버퍼	51

12.5 APP1 header의 바이트 단위 해석	51
12.6 업로드 실패와 현재 프로토콜의 범위	52
13. run 명령의 검증·복사·호출	53
13.1 실제 동작 순서	53
13.2 파일 offset과 목적지 주소	53
13.3 왜 마지막 byte를 한 번 더 읽는가	54
13.4 함수 포인터 호출의 의미	54
13.5 반환 경로가 살아 있어야 하는 이유	54
13.6 잘못된 앱 파일에 대한 검증 경로	55
14. 60바이트 앱의 실제 명령어 추적	56
14.1 명령어와 문자열의 분리	56
14.2 첫 문자 H의 전체 이동	56
14.3 마지막 문자와 복귀	57
14.4 역어셈블에서 보이는 가짜 명령어	57
14.5 한 명령어를 손으로 해석하기	57
15. 실행 중 인터럽트와 세 종류의 복귀	59
15.1 앱 실행 중 timer가 발생하면	59
15.2 ECALL, 함수 호출, context switch 비교	59
15.3 timer 재설정과 interrupt masking	60
15.4 장시간 동작의 timer 제한	60
15.5 64비트 비교와 32비트 재설정의 불일치 예	60
16. ZCU104 실습 절차	62
16.1 호스트 명령과 rv> 명령 구분	62
16.2 처음부터 준비하는 순서	62
16.3 UART에 재접속하여 실행	63
16.4 이미 셸이 실행 중일 때 앱만 바꾸기	63
16.5 screen의 종료·분리·중지	64
16.6 증상에서 계층을 찾아가기	64
17. 검증 방법과 관측 가능한 증거	66
17.1 host artifact 검사	66
17.2 RTL 시뮬레이션이 확인하는 것	66
17.3 실제 보드 검증	67
17.4 현재 자동 검사의 범위	67
17.5 파형을 추가해 관찰할 신호	68
17.6 invariant를 이용한 검증 설계	68
18. 현재 구현의 한계와 확장 설계	70
18.1 M-mode 앱의 권한	70
18.2 앱 종료와 오류 처리	70
18.3 메모리 시스템의 제약	70
18.4 운영체제 기능별 다음 단계	71
18.5 파일 시스템과 프로토콜의 후속 과제	71
19. 수업 구성과 연습문제	73
19.1 4회 실습 수업 구성 예	73
19.2 기본 이해 문제	73
19.3 계산·코드 해석 문제	74
19.4 실험 문제	74

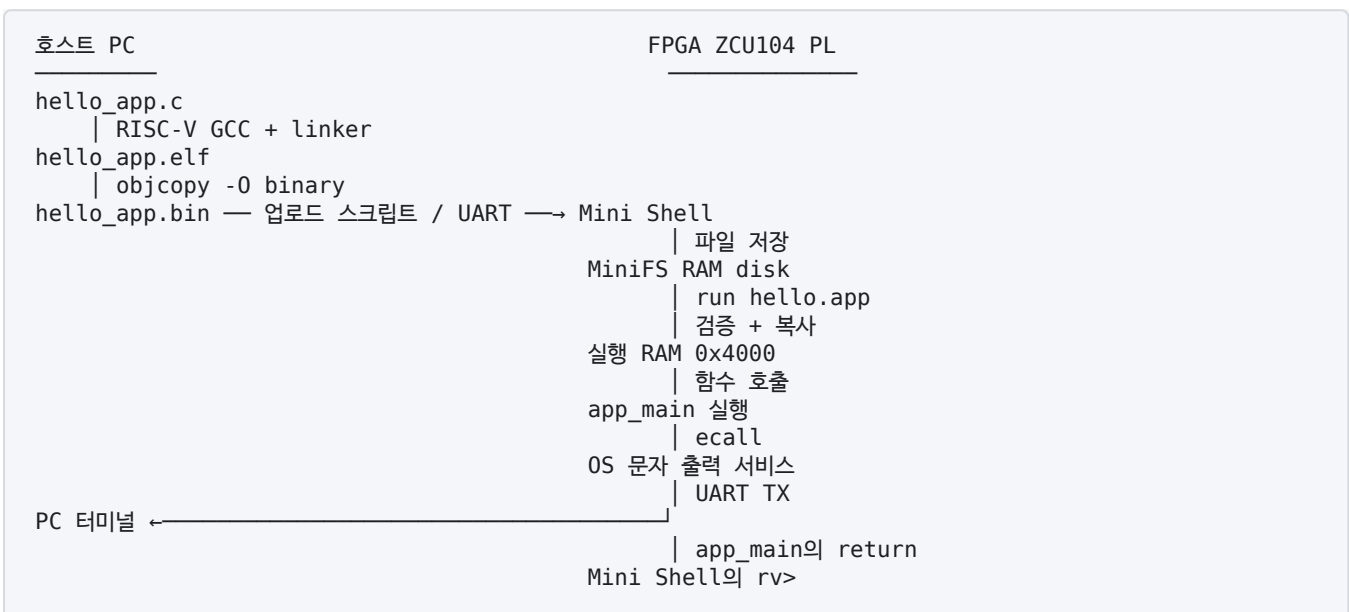
19.5 설계 문제	74
19.6 ISA와 데이터패스 심화 문제	75
20. 해설·용어·코드 찾아보기	76
20.1 기본 문제 해설	76
20.2 계산 문제 해설	76
20.3 설계 문제의 검토 기준	77
20.4 용어 정리	77
20.5 코드 찾아보기	78
20.6 참고 자료와 문서의 범위	79
20.7 ISA 심화 문제 해설	79

1. 학습 목표와 전체 실행 경로

1.1 이 실습에서 실제로 일어나는 일

PC에서 `hello_app.c` 를 RISC-V 기계어로 컴파일하고, USB UART를 통해 FPGA에 전송합니다. FPGA에서 실행 중인 Mini Shell은 받은 바이트를 MiniFS 파일로 저장합니다. 사용자가 `run hello.app` 을 입력하면 셸은 파일을 읽고 검사한 뒤, 명령어를 가져올 수 있는 RAM으로 복사하고 그 시작 주소를 호출합니다. 프로그램은 운영체제의 문자 출력 서비스를 사용하고 셸로 돌아옵니다.

이 과정에서 프로세서가 수행하는 기본 동작은 여전히 명령어 인출, 디코딩, 레지스터 읽기, 연산, 메모리 접근, 다음 PC 선택입니다. 파일 이름이나 `run` 이라는 명령의 의미는 C 프로그램이 해석합니다. CPU는 `hello.app` 이라는 이름을 알지 못합니다. CPU가 직접 보는 것은 PC에 해당하는 32비트 명령어와 레지스터·메모리 값입니다.



위 실행 경로를 시간 순서로 나누면 서로 다른 두 준비 과정이 보입니다. 먼저 FPGA에 CPU 회로와 셸 펌웨어를 구성합니다. 그 후 CPU가 계속 실행되는 동안 앱 바이트를 전송합니다. 두 번째 작업은 기존 회로의 RAM 내용을 바꾸는 것이므로 새 앱을 시험할 때마다 논리 합성·배선을 반복할 필요가 없습니다. 하드웨어 플랫폼을 고정한 상태에서 소프트웨어를 바꾸는 경험이 이 실습의 출발점입니다.

예를 들어 문자열을 `Hello` 에서 `Welcome` 으로 바꾸는 일은 앱 변경입니다. 앱 창을 `0x4000` 에서 다른 주소로 옮기는 일은 loader·linker·메모리 배치의 공동 변경입니다. 곱셈 명령 `MUL` 을 직접 실행하고 싶다면 core의 ISA 확장 또는 소프트웨어 지원이 필요합니다. 변화의 종류를 먼저 분류하면 어떤 산출물을 다시 만들어야 하는지 판단할 수 있습니다.

1.2 학습 후 설명할 수 있어야 하는 것

학습자는 `SHELL_MODE` 가 언제 확정되는지, 프로그램의 파일 위치와 실행 주소가 왜 다른지, `ecall` 의 복귀 주소와 함수의 복귀 주소가 어떻게 구분되는지를 설명할 수 있어야 합니다. 또한 빈 UART 화면, 업로드 후 파일이 사라진 현상, 실행 파일 검증 실패를 계층별로 진단할 수 있어야 합니다.

전체 과정을 이해할 때 다음 네 가지 계약을 함께 보아야 합니다.

계약	약속하는 내용	현재 프로젝트의 예
ISA	명령어 인코딩과 실행 의미	LBU, JALR, ECALL, MRET
하드웨어 플랫폼	주소별 RAM·장치 동작	UART RX 데이터는 0x10000008
소프트웨어 ABI	레지스터, 스택, 서비스 호출 규칙	a7=1, a0=문자, ecall
실행 파일 형식	저장 바이트의 해석과 로딩 규칙	APP1 헤더, 고정 진입 주소 0x4000

ISA가 같다고 아무 운영체제용 실행 파일이나 동작하는 것은 아닙니다. 동일한 RISC-V 명령어로 만들어졌더라도 링크 주소, syscall 번호, 메모리 크기, 초기화 조건이 이 환경과 맞아야 합니다.

실습 기록에는 네 계약 중 어느 계약을 확인했는지 표시하면 좋습니다. objdump에서 JALR를 찾는 것은 ISA와 제어 흐름을 보는 것이고, ls의 72바이트는 파일 형식을 보는 것입니다. PC가 0x4000에 도달한 파형은 주소 배치와 loader를 함께 확인합니다. 각 관측을 연결해야 단순히 문자열이 우연히 출력된 경우와 의도한 경로로 실행된 경우를 구분할 수 있습니다.

권장 학습 순서는 먼저 주소 지도에 메모리와 장치를 표시하고, 명령 하나가 그 지도에 접근하는 과정을 익힌 뒤, 여러 명령으로 구성된 syscall·파일 복사·앱 실행을 추적하는 것입니다. 하드웨어를 처음 공부하는 독자는 3장의 ISA와 데이터패스를 충분히 읽고, C에 익숙한 독자는 같은 연산을 생성된 assembly와 대조하십시오.

1.3 여기서 말하는 앱과 프로세스

현재 앱은 Mini Shell이 호출하는 독립적으로 컴파일된 함수입니다. 앱에는 별도 PID, 주소 공간, 페이지 테이블, 사용자 모드, 전용 task slot이 없습니다. 셸과 앱은 같은 M-mode에서 같은 셸 task의 스택을 사용합니다.

따라서 이 프로젝트는 파일에서 코드를 읽어 실행하는 로더의 핵심을 학습하기에 적합합니다. 동시에 일반적인 OS의 프로세스 생성과 격리에 무엇이 더 필요한지도 비교할 수 있습니다. 본문에서 “앱 실행”은 이 구체적인 실행 모델을 뜻합니다.

함수 호출로 실행하더라도 앱은 별도 파일에서 가져왔으므로 작성·컴파일·배포 단위는 독립적입니다. 반면 실행 중 CPU 문맥의 소유자는 셸 task입니다. 이 둘을 각각 “프로그램 이미지”와 “실행 문맥”으로 나누어 생각하면, 파일을 여러 개 저장하는 기능과 여러 프로그램을 동시에 스케줄링하는 기능이 왜 다른지 이해할 수 있습니다.

현재 hello.app과 다른 앱 파일을 차례로 실행할 수는 있지만 두 앱을 같은 0x4000 실행 창에서 동시에 유지할 수는 없습니다. 다음 앱을 로딩하면 이전 이미지가 덮어써집니다. 정상 반환 후 이전 앱의 실행 상태를 다시 이어가는 suspend/resume 모델도 없습니다. 앱의 수명은 run의 호출부터 함수 반환까지입니다.

2. 최상위 회로와 SHELL_MODE의 설정

2.1 zcu104_top은 누가 호출하는가

Verilog의 module은 회로의 구조와 동작을 기술합니다. 다른 module 안에 인스턴스화하면 그 회로의 일부가 됩니다. 이 프로젝트의 최상위 module은 zcu104_top 이며, 합성 도구에 다음처럼 지정합니다.

```
synth_design -top zcu104_top -part xczu7ev-ffvc1156-2-e ...
```

Vivado는 이 모듈을 루트로 내부 연결을 해석하고 논리소자와 배선으로 구현합니다. 보드에서는 소프트웨어가 zcu104_top() 을 호출하는 것이 아닙니다. 구성된 회로가 클럭과 입력 신호에 따라 계속 동작합니다. 시뮬레이션에서는 testbench가 설계 module을 인스턴스화하고 클럭·리셋·UART 입력을 만들어 줍니다.

현재 계층은 다음과 같습니다. 실제 합성 후에는 최적화로 계층 일부가 바뀔 수 있습니다.

```
zcu104_top
├─ 입력 클럭 버퍼 / 24분주 / CPU 클럭 버퍼
├─ reset 동기화 회로
├─ rv32_soc
│   ├── rv32_core
│   │   ├── rv32_regfile
│   │   ├── rv32_alu
│   │   └── rv32_csr
│   ├── unified instruction/data RAM
│   ├── MiniFS RAM disk
│   ├── UART MMIO와 RX FIFO
│   └── simple_timer
├─ uart_rx : serial_rx
├─ uart_tx : serial
└─ LED 관측 회로
```

보드의 300 MHz 입력을 24분주하여 CPU와 UART를 12.5 MHz로 구동합니다. UART RX/TX 모듈과 SoC가 같은 클럭을 사용하므로 이들 사이의 바이트 전달에는 별도 클럭 도메인 교차가 없습니다. 외부 UART 입력 자체는 비동기이므로 uart_rx 안에서 2단 레지스터로 동기화합니다. 현재 보드 구성은 PL 내부에서 동작하며 ARM PS 소프트웨어나 DDR 초기화를 사용하지 않습니다.

top의 입력·출력 포트 이름은 XDC의 get_ports 와 연결됩니다. 예를 들어 UART_RX 라는 논리 입력이 A20 pin에 배치되고, 내부 serial_rx 의 rx 로 이어집니다. 소스에 wire를 선언하는 것과 실제 package pin을 선택하는 것은 서로 다른 단계입니다. module 연결이 맞더라도 pin이나 I/O 전압 제약이 틀리면 보드에서는 정상 통신을 기대할 수 없습니다.

분주 회로는 0부터 11까지 세고 출력 클럭을 반전합니다. 반 주기에 입력 12클럭, 한 주기에 24클럭이 필요합니다. reset은 버튼으로 비동기 assert되고 4단 shift register를 통해 동기적으로 해제됩니다. 이 장의 회로 구조를 볼 때 clock buffer와 reset 동기화도 CPU가 안정적으로 명령어를 시작하기 위한 플랫폼의 일부로 보아야 합니다.

2.2 동일한 이름을 사용하는 두 설정

SHELL_MODE 는 C와 Verilog 양쪽에 등장하지만 설정 경로가 다릅니다.

위치	지정 방법	효력 시점	결과
C 전처리	GCC의 -DSHELL_MODE	펌웨어 컴파일	셀 함수, 앱 로더, 셀용 스케줄러 선택

위치	지정 방법	효력 시점	결과
Verilog parameter	Vivado의 <code>-generic SHELL_MODE=1</code>	elaboration·합성	32 KiB RAM과 <code>mini_shell.hex</code> 선택
빌드 환경 변수	<code>SHELL_BUILD=1</code>	Tcl 스크립트 실행	셀용 합성 옵션·출력 파일 선택

환경 변수 `SHELL_BUILD` 가 C 매크로나 Verilog parameter로 저절로 변환되는 것은 아닙니다. Makefile과 Tcl 코드가 두 빌드 경로를 연결합니다.

세 이름이 같거나 비슷하다고 하나의 전역 설정이라고 생각하면 빌드 오류를 놓치기 쉽습니다. C 매크로를 바꾸면 compiler가 포함하는 함수와 배열 크기가 달라집니다. Verilog parameter를 바꾸면 합성되는 메모리 용량과 초기화 이미지가 달라집니다. Tcl 환경 변수는 어느 명령행을 실행할지 고르는 host 측 정보이며, FPGA 안에 `SHELL_BUILD` 라는 실행 중 변수가 남는 것은 아닙니다.

학생은 Makefile에서 `-DSHELL_MODE`, Tcl에서 `-generic SHELL_MODE=1`, top에서 `MEM_WORDS` 를 각각 찾아 밑줄을 그어 보십시오. 세 지점이 하나의 build target으로 묶여 있다는 사실이 재현 가능한 시스템 빌드의 핵심입니다. 하나만 수정으로 바꾸고 나머지를 그대로 두면 firmware와 하드웨어의 계약이 어긋날 수 있습니다.

2.3 make vivado-zcu104-shell의 전개

관련 코드는 [Makefile](#)과 [build_zcu104.tcl](#)에 있습니다.

```
make vivado-zcu104-shell
├── make -B mini-shell
│   ├── GCC -DSHELL_MODE ... boot.S trap.S kernel.c minifs.c
│   ├── mini_shell.elf → mini_shell.bin
│   └── bin2hex.py → firmware/mini_shell.hex
└── SHELL_BUILD=1 vivado ... build_zcu104.tcl
    ├── read_verilog / read_xdc
    ├── synth_design ... -generic SHELL_MODE=1
    ├── 배치·배선·타이밍·DRC 검사
    └── build/zcu104_shell/zcu104_mini_shell.bit
```

`make -B`는 펌웨어를 강제로 재생성합니다. `.bit`에는 논리 구조뿐 아니라 합성에 사용한 메모리 초기값도 반영됩니다. 이후 C 파일을 수정해 `.hex`만 바꾸어도 이미 생성된 `.bit`의 내용은 바뀌지 않습니다.

Tcl의 분기 조건은 다음과 같습니다.

```
set shell [info exists ::env(SHELL_BUILD)]
```

이는 환경 변수의 존재 여부를 검사합니다. `SHELL_BUILD=0`으로 실행해도 변수가 존재하므로 셸 분기를 선택합니다. 현재 구현에서 일반 OS를 만들려면 이 변수가 없는 환경에서 일반 target을 사용해야 합니다. C 코드의 `#ifdef SHELL_MODE`도 정의 여부를 검사하므로 `-DSHELL_MODE=0`은 셸 코드를 포함합니다. Verilog의 조건식은 parameter의 숫자 값을 사용합니다.

합성은 RTL의 논리 기능을 FPGA resource로 바꾸고, 배치·배선은 그 resource의 위치와 연결을 정합니다. timing 분석은 register 사이의 경로가 12.5 MHz 클록의 제약을 만족하는지 평가합니다. DRC는 설정·배선 규칙을 검사합니다. 이들 검사를 통과한 것은 회로를 구성할 기반을 확인한 것이며, run의 파일 형식 검사나 C 코드의 의미까지 증명한 것은 아닙니다.

산출물의 갱신 관계를 추적하려면 source 수정 시각뿐 아니라 사용한 build target과 toolchain을 기록해야 합니다. 이 프로젝트의 .bit 에는 메모리 초기값이 들어 있으므로, 펌웨어 수정 후 기존 .bit 를 그대로 다운로드하는 것은 이전 프로그램을 다시 실행하는 결과가 됩니다. 반대로 앱 payload 변경은 이미 실행 중인 loader로 전송할 수 있습니다.

2.4 하드웨어 설정에 따른 메모리 선택

zcu104_top.v 에는 다음 선언과 연결이 있습니다.

```
parameter SHELL_MODE = 0,
parameter MEM_HEX = SHELL_MODE ? "firmware/mini_shell.hex"
                        : "firmware/mini_os.hex"

rv32_soc #(
    .MEM_WORDS(SHELL_MODE ? 8192 : 2048),
    .MEM_HEX(MEM_HEX)
) soc (...);
```

MEM_WORDS 의 단위는 32비트 word입니다. 따라서 셀 구성은 $8192 \times 4 = 32768$ 바이트이고 일반 OS 구성은 $2048 \times 4 = 8192$ 바이트입니다. 셀을 지원하기 위해 CPU에 새로운 명령어 디코더를 추가하는 것이 아니라, 기존 CPU를 더 큰 메모리와 다른 펌웨어로 구성합니다.

make program-zcu104-shell 의 SHELL_BUILD=1 은 이미 만들어진 셀용 .bit 를 선택하는 데 사용됩니다. 다운로드 시 parameter를 새로 변경하는 기능은 없습니다.

Verilog에서 parameter로 결정한 배열 크기는 실행 중 늘어나지 않습니다. 32 KiB 구성에서 앱이 더 많은 주소를 사용한다고 FPGA가 RAM을 추가로 할당하지 않습니다. CPU가 접근 가능한 주소를 decoder가 선택할 뿐입니다. 이후 4장의 linker 경계와 13장의 loader 크기 검사가 필요한 이유가 여기에 있습니다.

일반 OS 이미지와 셀 이미지의 차이는 메모리만이 아닙니다. C 쪽에서는 task 수, scheduler 분기, CRLF 출력, shell parser와 loader의 포함 여부도 달라집니다. 두 이미지는 같은 core를 사용하는 서로 다른 시스템 구성으로 이해해야 합니다. ELF의 symbol 목록을 비교하면 어느 C 함수가 실제로 포함됐는지도 확인할 수 있습니다.

2.5 top에서 신호를 연결하는 실습

module instance의 .포트이름(연결신호) 는 이름이 비슷한 변수를 선언하는 문법이 아니라 실제 회로 연결을 지정합니다. .uart_rx_data(rx_data) 에서 왼쪽은 rv32_soc가 정의한 입력 포트이고 오른쪽은 top 안의 wire입니다. 반대로 .data(rx_data) 는 uart_rx의 출력 포트를 같은 wire에 연결합니다. 두 인스턴스가 같은 이름의 wire를 통해 통신하는 것입니다.

top의 wire	보내는 쪽	받는 쪽	의미
cpu_clk	divider와 BUFG	SoC, RX, TX	공통 12.5 MHz clock
rst	reset synchronizer	SoC, RX, TX	내부 active-high reset
rx_data[7:0]	uart_rx.data	SoC.uart_rx_data	완성된 수신 byte
rx_valid	uart_rx.valid	SoC.uart_rx_valid	수신 byte 유효 pulse
data[7:0]	SoC.uart_tx_data	uart_tx.data	송신할 byte
valid	SoC.uart_tx_valid	uart_tx.valid	송신 요청 pulse
ready	uart_tx.ready	SoC.uart_tx_ready	송신기가 새 byte를 받을 수 있음

top의 wire	보내는 쪽	받는 쪽	의미
pc[31:0]	SoC.debug_pc	LED logic	내부 실행 위치의 일부 관측

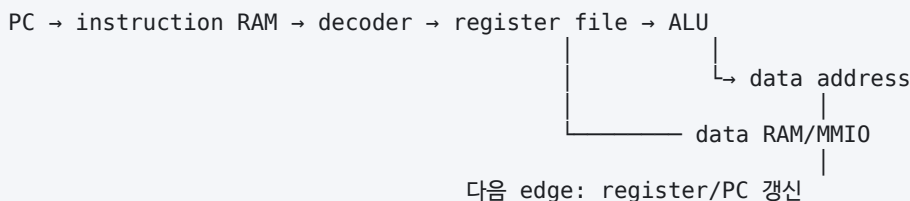
RX의 valid와 TX의 valid는 역할이 다릅니다. RX valid는 외부에서 도착한 데이터 완료를 알리고, TX valid는 내부 software가 준비한 데이터를 보내 달라는 요청입니다. ready는 TX 쪽에만 있으며 현재 RX에는 외부 송신기를 멈추는 backpressure 선이 없습니다. 동일한 handshake 용어라도 방향과 수신 능력을 확인해야 합니다.

이 표를 보드 pin까지 확장하면 UART_RX → rx → rx_data → FIFO → MMIO와 MMIO → data/valid → tx → UART_TX의 두 경로를 그릴 수 있습니다. core가 pin의 serial bit를 직접 읽는 것이 아니라 SoC와 UART 회로가 단계적으로 표현을 바꾼다는 사실을 확인하는 실습입니다.

3. 바이너리 실행을 가능하게 하는 CPU와 메모리

3.1 단일 사이클 코어의 실행 경로

현재 `rv32_core` 는 PC로 명령어를 읽고, 같은 사이클의 조합 논리에서 디코딩·레지스터 읽기·ALU 계산·데이터 읽기를 진행합니다. 다음 rising edge에서 PC와 필요한 레지스터·메모리 상태를 갱신합니다. 메모리가 같은 사이클 안에 값을 반환한다는 계약이 있기 때문에 가능한 구조입니다.



UART 전송이나 파일 복사가 한 사이클에 끝난다는 뜻은 아닙니다. 각각 많은 명령어와 장치 대기로 이루어진 소프트웨어 작업입니다. 한 문자 출력도 ready 레지스터를 여러 번 읽는 반복문과 store 명령을 포함합니다.

한 사이클을 “계산 전 값이 register에 보관된 시점 → 조합 경로가 안정되는 구간 → 다음 edge에서 결과를 저장하는 시점”으로 나누면 이해하기 쉽습니다. 조합 논리에서 `next_pc` 가 여러 번 변할 수 있어도 최종적으로 edge에서 채택되는 값이 실행 결과입니다. 시뮬레이터의 delta cycle 동안 잠시 보이는 중간값과 실제 architectural commit을 구분해야 합니다.

현재 CPU에는 IF/ID, ID/EX 같은 pipeline register가 없습니다. 명령어 A의 계산 결과가 edge에서 저장된 뒤 명령어 B의 조합 계산이 시작됩니다. 그래서 forwarding이나 load-use stall 회로가 필요하지 않지만, 한 사이클의 길이는 가장 느린 명령의 조합 경로에 맞춰야 합니다. LW의 경우 instruction read부터 data read까지 같은 사이클에 들어갑니다.

3.2 instruction port와 data port가 같은 RAM을 보는 이유

`rv32_soc.v` 의 프로그램 메모리는 다음 배열입니다.

```
reg [31:0] mem [0:MEM_WORDS-1];
```

명령어 인출은 `mem[iaddr[15:2]]`, 데이터 접근은 `mem[daddr[15:2]]` 를 읽습니다. CPU 인터페이스에는 instruction/data 포트가 나뉘어 있지만 저장소는 같습니다. 이 구조를 Harvard 형태의 인터페이스를 가진 unified memory라고 설명할 수 있습니다.

앱 로더가 store 명령으로 `mem` 에 기계어 바이트를 쓴 뒤 PC를 해당 주소로 이동시키면, instruction port가 그 바이트들을 명령어로 가져옵니다. 파일이 실행으로 바뀌는 하드웨어상의 핵심이 이 연결입니다.

RAM disk 배열은 별도입니다. `0x80100000` 영역은 데이터 포트에서만 읽고 쓸 수 있으며, 현재 instruction fetch 경로는 이 배열을 선택하지 않습니다. 따라서 파일이 RAM disk에 있다고 그 주소로 바로 분기할 수는 없습니다. 실행 가능한 unified RAM으로 복사해야 합니다.

instruction 포트와 data 포트를 분리한 이유는 load/store를 실행하면서도 현재 명령어를 동시에 공급해야 하기 때문입니다. 하나의 storage를 두 조합 read 경로가 보는 구조이므로, 논리적으로 같은 메모리라도 read port 구현

비용이 발생합니다. FPGA에서 이런 read 형태는 동기식 block RAM의 기본 동작과 다르므로 단순히 배열이라고 모두 BRAM이 되는 것은 아닙니다.

프로그램 RAM에는 코드와 C 데이터가 함께 있습니다. 같은 32비트 값이라도 PC로 읽으면 decoder 입력이고 LW로 읽으면 일반 데이터입니다. CPU가 RAM 각 word에 “이것은 코드”라는 별도 태그를 붙이는 구조가 아닙니다. 현재는 실행 권한 bit도 없으므로 잘못된 분기로 데이터 영역을 실행하려 할 수 있습니다.

3.3 byte 주소와 word 인덱스

RISC-V의 주소는 byte 단위입니다. mem의 한 원소는 4바이트이므로 배열 인덱스는 주소를 4로 나눈 값입니다.

```
앱 주소 0x4000 / 4 = 0x1000 = 4096
따라서 앱 첫 명령어의 저장 위치는 mem[4096]
```

하위 두 비트는 word 안에서의 byte 위치입니다. 예를 들어 주소 0x4001의 byte는 mem[4096]의 비트 [15:8]에 있습니다. LBU는 해당 word를 읽고 offset만큼 오른쪽으로 이동한 뒤 8비트를 zero-extension합니다. SB는 byte write strobe로 필요한 lane만 갱신합니다.

```
// SB의 byte lane 선택
dwstrb_r = 4'b0001 << daddr_r[1:0];
```

이런 byte 연산 덕분에 파일 복사 코드가 임의의 바이트 수를 다룰 수 있습니다. 현재 구현은 자연 정렬을 전제로 하며 잘못 정렬된 word/halfword 접근을 일반적인 정렬 예외로 처리하지 않습니다.

가령 한 word가 0x44332211이면 낮은 주소부터 11 22 33 44가 저장됩니다. 주소 하위 비트가 10인 LBU는 word를 16비트 shift하여 0x33을 얻습니다. 같은 위치에 0xAA를 SB로 쓰면 strobe 0100이 선택되어 새 word는 0x44AA2211입니다. 인덱스 계산과 byte lane 선택을 따로 수행해야 인접 byte가 보존됩니다.

daddr[15:2]라는 slice 자체가 범위 검사 역할을 하는 것은 아닙니다. 우선 daddr < MEM_BYTES 인지 decoder가 검사하고, 선택된 경우에만 slice로 word를 고릅니다. 이 순서를 놓치면 주소의 상위 비트를 잘라 서로 다른 주소가 같은 RAM으로 alias되는 오류를 설명하지 못하게 됩니다.

3.4 MMIO는 주소 디코딩이다

데이터 포트 주소가 프로그램 RAM 범위이면 mem, RAM disk 범위이면 ramdisk, UART 주소이면 UART 상태·데이터 레지스터를 선택합니다. C의 포인터 역참조가 load/store 명령으로 바뀌고, 그 명령의 주소가 회로를 선택합니다.

이 프로젝트의 주소 선택은 memory-mapped I/O입니다. Unix의 mmap() 시스템 콜과는 구분해야 합니다. 현재 OS에는 가상메모리 매핑을 만드는 mmap() 서비스가 없습니다.

volatile 포인터는 compiler가 장치 접근을 일반 변수처럼 없애거나 재사용하지 않도록 하는 C 측 표현입니다. 실제 어떤 장치를 선택하는지는 RTL decoder가 결정합니다. volatile이 UART 회로를 생성하거나 메모리 보호를 설정하는 것은 아닙니다. 장치 register는 읽기 자체가 FIFO pop 같은 부작용을 가질 수 있어 일반 RAM 접근과 의미도 다릅니다.

현재 timer는 0x10001xxx page 전체를 선택하고 내부에서는 주소 bit 2로 두 register를 구분합니다. 따라서 문서상의 정식 주소 이외에도 alias가 존재합니다. firmware는 정식 word-aligned 주소만 사용해야 합니다. 엄격한 peripheral map이 필요하면 decoder의 허용 주소·접근 크기·쓰기 권한을 더 좁혀야 합니다.

3.5 instruction cache가 생기면 추가되는 책임

현재는 cache와 prefetch pipeline이 없고 store와 fetch가 동일 RAM을 사용하므로, 로더의 복사가 끝난 뒤 새 코드를 가져올 수 있습니다. 다른 RISC-V 시스템까지 이 동작을 일반화하면 안 됩니다. 명령어 fetch와 데이터 store의 동기화에는 FENCE.I 를 사용하는 규약이 있습니다. 관련 의미는 [RISC-V Zifencei 명세](#)에 설명되어 있습니다.

현재 core는 FENCE 를 NOP로 처리하지만 FENCE.I 는 구현하지 않습니다. FENCE 와 FENCE.I 를 같은 명령어로 간주하면 안 됩니다. 나중에 instruction cache나 pipeline을 추가할 때는 로더와 명령어 동기화 구현을 함께 설계해야 합니다.

예를 들어 앱 A를 실행한 뒤 같은 RAM 위치에 앱 B를 복사했다고 가정하십시오. instruction cache가 A의 cache line을 보관한다면 data RAM은 B여도 CPU가 A의 명령을 계속 fetch할 수 있습니다. 이때 파일 checksum은 성공할 수 있으므로 파일 검증만으로 원인을 찾기 어렵습니다. data store의 완료와 instruction 관측의 일관성을 별도의 계약으로 다뤄야 합니다.

현재는 앞선 store가 edge에서 RAM에 반영되고 이후 fetch가 동일 배열을 읽습니다. 따라서 이 구현에 필요한 동기화는 구조적으로 단순합니다. pipeline·cache를 추가하는 시점에는 CPU 쪽 flush/invalidate 동작과 OS 쪽 실행 전 동기화 호출을 함께 추가해야 하며, 기존 소프트웨어가 그대로 안전하다고 가정하면 안 됩니다.

3.6 현재 core의 architectural state와 내부 신호

architectural state는 명령어 실행 결과로 프로그램이 관찰할 수 있는 상태입니다. 이 core에서는 PC, x0 - x31, 구현된 CSR, RAM과 MMIO 상태가 중심입니다. alu_a, load_shifted, next_pc 같은 값은 한 명령을 계산하기 위한 내부 조합 신호이며 별도의 프로그래머용 register가 아닙니다.

종류	실제 RTL 요소	갱신 시점 또는 성질
현재 명령 위치	reg [31:0] pc	rising edge
정수 register	regs[1:31], x0의 상수 read	rising edge write, 조합 read
trap 상태	mstatus_r, mepc_r 등	rising edge
연산 operand	alu_a, alu_b	decode에 따른 조합 값
write-back 요청	rd_we, rd_data, rd	edge에서 쓸 값의 조합 계산
다음 명령 후보	next_pc	기본 PC+4, branch/jump에서 변경
memory write 요청	dwstrb_r, dwdata_r	trap gating 전의 조합 요청

Verilog의 reg 라는 자료형 이름만 보고 모두 flip-flop이라고 판단하면 안 됩니다. always @* 에서 모든 경로에 값이 주어지는 reg 변수는 조합 회로가 됩니다. 실제 저장소인지 판단하려면 선언보다 어떤 always block에서 어떤 조건으로 갱신되는지를 보아야 합니다.

3.7 RV32I와 46개 명령의 정확한 범위

이 프로젝트는 RV32I 정수 연산을 기반으로 CSR와 MRET를 추가한 교육용 core입니다. 현재 decoder의 mnemonic 종류를 다음처럼 세면 46개입니다. pseudo-instruction과 syscall 서비스 번호는 이 수에 포함하지 않습니다.

계열	현재 명령	수
register 산술·논리	ADD SUB SLL SLT SLTU XOR SRL SRA OR AND	10

계열	현재 명령	수
immediate 산술·논리	ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI	9
load	LB LH LW LBU LHU	5
store	SB SH SW	3
branch	BEQ BNE BLT BGE BLTU BGEU	6
upper immediate	LUI AUIPC	2
jump	JAL JALR	2
memory order	FENCE, 이 구현에서는 NOP	1
환경 호출	ECALL	1
CSR	CSRRW CSRRS CSRRC CSRRWI CSRRSI CSRRCI	6
trap 복귀	MRET	1
합계		46

표준 RV32I의 40개 명령 중 EBREAK의 breakpoint 동작은 이 core에 없고 illegal instruction으로 처리됩니다. 정렬·접근 fault와 CSR 권한 검사에도 생략이 있습니다. 따라서 “46개가 있으므로 완전한 표준 호환성 검증을 마쳤다”는 결론은 성립하지 않습니다. 기본 ISA의 구분은 [RISC-V RV32I 명세](#), 위 표의 지원 여부는 현재 decoder가 근거입니다.

RV32의 32는 정수 register 폭을 뜻합니다. I는 기본 정수 ISA이며, M 곱셈·나눗셈, A 원자 연산, F/D 부동소수점, C 압축 명령, V vector는 구현하지 않았습니다. 프로그램의 문자열·파일·scheduler는 기본 연산을 반복해서 구현할 수 있으므로 전용 “파일 명령어”가 필요하지 않습니다.

3.8 register file과 ABI 이름

register file은 source 두 개를 동시에 읽고 destination 하나를 edge에서 쓰는 2R1W 구조입니다. R-type의 add x5, x6, x7 은 x6와 x7을 조합으로 읽고 합을 x5에 저장합니다. destination 번호는 insn[11:7] 에서 이미 꺼내 rd_addr 로 연결하므로 opcode case 안에서 별도로 register 이름을 지정하지 않습니다.

register	ABI 이름	소프트웨어의 일반적 역할
x0	zero	읽으면 0, 쓰기는 버림
x1	ra	함수 복귀 주소
x2	sp	stack pointer
x3 / x4	gp / tp	global/thread pointer; 현재 runtime 사용 제한
x5 - x7	t0 - t2	임시 값
x8 - x9	s0 - s1	호출받은 함수가 보존; s0는 선택적 frame pointer
x10 - x17	a0 - a7	함수 인수·결과; Mini OS는 a7을 syscall 번호로 사용
x18 - x27	s2 - s11	호출받은 함수가 보존
x28 - x31	t3 - t6	임시 값

하드웨어 x1에 “return address만 저장해야 한다”는 제약은 없습니다. ra 와 sp 는 ABI의 약속입니다. addi sp, sp, -16 도 core 입장에서는 x2를 대상으로 하는 일반 덧셈입니다. 반면 x0는 RTL에서 실제로 특별 취급하여 읽기는

0, 쓰기는 무시합니다.

reset loop가 x1 - x31까지 0으로 만드는 것은 이 구현의 선택입니다. C runtime은 그 사실에만 기대지 않고 SP와 필요한 상태를 명시적으로 준비합니다. pipeline이 없으므로 이전 명령의 write edge가 지난 다음에는 새 값이 read port에 보이고, 별도의 forwarding 경로가 필요하지 않습니다.

3.9 32비트 명령어의 공통 필드

모든 명령은 4바이트 word로 fetch됩니다. decoder는 기본적으로 opcode를 보고 큰 계열을 선택하고, funct3·funct7로 같은 계열 안의 세부 연산을 선택합니다. 아래 표의 bit 번호는 memory byte 번호가 아니라 fetch된 32비트 word 안의 위치입니다.

필드	bit 위치	의미
opcode	[6:0]	LOAD, STORE, OP 등 큰 연산 계열
rd	[11:7]	destination register가 있는 형식에서 사용
funct3	[14:12]	load 크기·branch 조건·ALU 세부 기능 등
rs1	[19:15]	첫 source register 또는 CSR immediate
rs2	[24:20]	두 번째 source register가 있는 형식에서 사용
funct7	[31:25]	ADD/SUB, SRL/SRA 등의 구분

명령어에 해당 의미의 operand가 없는 경우에도 slice wire 자체는 존재합니다. 예를 들어 store의 [11:7]은 rd가 아니라 immediate의 일부입니다. decoder가 rd_we=0을 유지하므로 우연히 잘라낸 register 번호에 쓰지 않습니다. 필드 추출과 필드 사용 조건을 함께 이해해야 합니다.

3.10 R·I·S·B·U·J 형식

```
R: [ funct7 ][ rs2 ][ rs1 ][f3][ rd ][ opcode ]
I: [      imm[11:0]      ][ rs1 ][f3][ rd ][opcode]
S: [imm[11:5]][rs2][rs1][f3][imm[4:0]][opcode]
B: [imm[12|10:5]][rs2][rs1][f3][imm[4:1|11]][opcode]
U: [      imm[31:12]      ][rd][opcode]
J: [      imm[20|10:1|11|19:12]      ][rd][opcode]
```

도식의 칸 폭은 개념 표현이며 정확한 연결은 아래 식으로 확인합니다. R형은 두 register로 계산하는 연산, I형은 짧은 immediate·load·JALR, S형은 store, B형은 조건 branch, U형은 큰 상수의 상위 부분, J형은 PC-relative jump에 사용합니다.

S/B형에는 destination register가 없으므로 그 위치를 immediate에 사용합니다. B/J offset의 bit 0은 명령어 안에 저장하지 않고 0으로 붙입니다. offset이 2바이트 단위로 인코딩되더라도 현재 코어의 정상적인 instruction address는 4바이트 정렬이어야 합니다. 현재 정렬 fault를 구현하지 않은 사실과 ISA가 정한 정렬 조건은 별개입니다.

3.11 immediate 재구성과 부호 확장

현재 core의 helper function은 다음과 같은 의미입니다. sext는 최상위 부호 bit를 32비트 폭까지 반복한다는 표기입니다.

```

I = sext(insn[31:20])
S = sext({insn[31:25], insn[11:7]})
B = sext({insn[31], insn[7], insn[30:25], insn[11:8], 0})
U = {insn[31:12], 12'b0}
J = sext({insn[31], insn[19:12], insn[20], insn[30:21], 0})

```

I/S immediate는 -2048부터 2047까지 표현합니다. 12비트 0xFFC는 +4092가 아니라 -4이며, 32비트 부호 확장 결과는 0xFFFFFFF0입니다. `addi sp,sp,-16`과 `lw ra,12(sp)`가 같은 I형 immediate 기구를 사용하는 사례입니다.

B offset의 표현 범위는 -4096부터 +4094, J offset은 -1048576부터 +1048574이며 모두 2의 배수입니다. 실제 현재 프로그램의 분기 목적지는 4바이트 정렬 조건도 지켜야 합니다. U형은 하위 12비트를 0으로 만드는 것이고 CSR immediate는 5비트 값을 zero-extension한다는 점이 다릅니다.

3.12 ALU의 연산과 signed 해석

rv32_alu.v의 내부 operation 번호는 ISA opcode와 다릅니다. decoder가 ADD 계열인지 알아낸 뒤 ALU_ADD=0 같은 작은 제어값으로 ALU를 선택합니다. 입력은 두 개의 32비트 값, 출력은 32비트 결과이며 clock 없이 조합적으로 계산합니다.

입력/연산	결과	관찰점
0xFFFFFFFF + 1	0x00000000	32비트 밖 carry는 결과에서 버림
SLT(0xFFFFFFFF, 1)	1	signed에서는 -1 < 1
SLTU(0xFFFFFFFF, 1)	0	unsigned에서는 4294967295 > 1
SRL(0x80000000, 1)	0x40000000	위쪽을 0으로 채움
SRA(0x80000000, 1)	0xC0000000	부호 bit 1을 채움
SLL(1, 32)	1	register shift는 하위 5비트, 즉 0만 사용

덧셈 결과에 signed overflow가 있어도 별도의 arithmetic trap을 만들지 않습니다. 비교는 bit pattern의 해석을 명시적으로 선택합니다. RTL의 \$signed(a)가 필요한 이유는 동일한 wire의 값도 연산에 따라 signed와 unsigned 의미가 달라지기 때문입니다. 별도의 조건 flag register는 없고 branch나 SLT 명령이 직접 비교합니다.

3.13 산술·논리 opcode의 해독

register 연산의 opcode는 0110011, immediate 연산은 0010011입니다. 아래는 현재 case 문을 읽기 위한 간단한 표입니다.

funct3	OP의 funct7=0000000	OP의 funct7=0100000	OP-IMM의 대응
000	ADD	SUB	ADDI
001	SLL	미지원	SLLI, 상위 bit 검증
010	SLT	미지원	SLTI
011	SLTU	미지원	SLTIU
100	XOR	미지원	XORI

funct3	OP의 funct7=0000000	OP의 funct7=0100000	OP-IMM의 대응
101	SRL	SRA	SRLI/SRAI, 상위 bit로 구분
110	OR	미지원	ORI
111	AND	미지원	ANDI

예를 들어 `add a0,a1,a2` 에서 `rd=x10`, `rs1=x11`, `rs2=x12`입니다. opcode와 funct 조합으로 `ALU_ADD`를 선택하면 `alu_y=x11+x12`, `rd_data=alu_y`, `rd_we=1` 이 됩니다. 다음 edge에서 `x10`만 바뀌고 `PC`는 4 증가합니다.

`ANDI` 의 immediate도 부호 확장됩니다. `andi a0,a0,255` 는 하위 8비트 mask지만 `andi a0,a0,-1` 은 모든 bit를 유지합니다. `SLTIU` 는 immediate를 부호 확장한 뒤 unsigned 비교하므로 “U가 붙으면 immediate도 zero-extension”이라고 생각하면 잘못됩니다.

3.14 LOAD: 주소 계산에서 write-back까지

`LOAD` opcode는 `0000011` 입니다. 유효 주소는 `rs1_data + imm_i` 이고, SoC가 해당 aligned word를 조합으로 반환하면 core가 크기·offset에 맞는 값을 골라 `rd`에 씁니다.

funct3	명령	확장 방식
000	LB	8비트 부호 확장
001	LH	16비트 부호 확장
010	LW	32비트 word 그대로
100	LBU	8비트 zero-extension
101	LHU	16비트 zero-extension

`dmem_rdata=0x807F02FF`, 주소 `offset=00`이면 `LB` 결과는 `0xFFFFFFFF`, `LBU` 는 `0x000000FF` 입니다. `offset=30`이면 선택 byte는 `0x80` 이므로 `LB`는 `0xFFFFFFFF80`, `LBU`는 `0x00000080` 입니다. 파일 payload는 unsigned byte들의 열이므로 loader의 byte sum에는 `LBU` 의미가 적합합니다.

`LW`는 word 정렬을 전제로 별도 shift를 하지 않습니다. `LH/LHU`는 `offset 0` 또는 `2`에서 사용해야 한 word 안의 두 byte가 올바르게 선택됩니다. 자연 정렬을 위반하면 현재 하드웨어가 자동으로 두 word를 조합해 주지 않습니다. compiler가 정렬된 C object를 다루도록 주소·자료형을 설계해야 합니다.

3.15 STORE: byte strobe와 데이터 복제

`STORE` opcode는 `0100011` 이고 주소는 `rs1+imm_s` 입니다. `rs2` 는 저장할 데이터이며 destination register는 없습니다. `RAM write`는 조합 계산 즉시가 아니라 clock edge에 발생합니다.

명령	주소 offset	dwstrb	저장 데이터 구성
SB	0 / 1 / 2 / 3	0001 / 0010 / 0100 / 1000	rs2 하위 byte를 4번 복제
SH	0 / 2	0011 / 1100	rs2 하위 halfword를 2번 복제
SW	0	1111	rs2의 32비트 전체

주소 offset=2에서 `sb a0,2(t0)`를 실행하고 a0의 하위 byte가 0x5A이면 write data는 0x5A5A5A5A, strobe는 0100입니다. 실제 RAM은 lane 2만 기록하므로 다른 세 byte는 그대로입니다. 데이터를 네 위치에 복제해 둔 덕분에 별도의 32비트 가변 left shift 없이 lane을 고를 수 있습니다.

SH 역시 하위 16비트를 양쪽 halfword에 복제합니다. 잘못 정렬된 offset=1의 SH도 현재 decoder에서 자동으로 trap되지 않으므로 firmware가 정렬 조건을 지켜야 합니다. “어떤 bit 조합에 RTL 값이 나오는가”와 “ISA와 프로그램이 사용해도 되는 접근인가”를 구분하십시오.

3.16 BRANCH: 조건과 target의 분리

BRANCH opcode는 1100011입니다. target은 현재 PC에 B immediate를 더한 값이고, 조건이 거짓이면 기본값 PC+4를 사용합니다. branch 자체는 rd를 쓰지 않습니다.

funct3	명령	비교
000 / 001	BEQ / BNE	같음 / 다름
100 / 101	BLT / BGE	signed 작음 / 크거나 같음
110 / 111	BLTU / BGEU	unsigned 작음 / 크거나 같음

앱의 `bne a0,zero,0x400C`는 PC 0x401C에서 -16을 더합니다. 다음 순차 주소 0x4020을 기준으로 계산하면 -20이라는 틀린 offset이 나옵니다. branch immediate의 기준은 해당 branch의 PC입니다. target 계산과 조건 비교를 각각 확인하면 off-by-four 오류를 쉽게 찾을 수 있습니다.

이 core에는 MIPS의 branch delay slot이 없습니다. 조건이 성립하면 다음 fetch부터 target의 명령을 가져오며 PC+4의 명령을 의무적으로 한 번 실행하지 않습니다. pipeline이 없으므로 branch flush할 중간 pipeline register도 없습니다.

3.17 JAL과 JALR: 함수 호출을 만드는 두 효과

JAL은 `rd=PC+4`와 `PC=PC+J immediate`를 함께 수행합니다. JALR은 `rd=PC+4`와 `PC=(rs1+I immediate)&0xFFFFFEE`를 수행합니다. 하나의 명령이 복귀 주소 기록과 제어 이동을 동시에 수행하는 것입니다.

```
호출 직전: PC=0x0A18, a5=0x4000
명령어   : jalr ra,0(a5)
edge 이후: ra=0x0A1C, PC=0x4000
```

rd 필드는 모든 JAL/JALR 명령에 인코딩되어 있으므로 RTL opcode branch에서는 `rd_we=1`, `rd_data=pc+4`만 지정해도 됩니다. 실제 destination은 공통 wire `rd=insn[11:7]`입니다. `jal zero,label`은 return address를 버리는 jump이며, `jalr zero,0(ra)`는 return입니다.

JALR는 bit 0만 0으로 만듭니다. 따라서 target이 자동으로 4바이트 정렬된다고 단정할 수 없습니다. 현재 core는 instruction-address-misaligned 예외를 구현하지 않았으므로 잘못된 bit 1은 software가 피해야 합니다. 앱 entry와 trap vector를 4바이트 정렬하는 linker 규칙이 이 전제를 지켜 줍니다.

3.18 LUI와 AUIPC: 큰 주소를 만드는 방법

LUI는 20비트 immediate 뒤에 12개의 0을 붙여 rd에 씁니다. `lui a5,0x4`의 결과가 `0x4000`인 이유입니다. AUIPC는 같은 형태의 값을 현재 PC에 더합니다. 하나는 상수 생성, 다른 하나는 PC-relative 주소 생성에 적합합니다.

12비트 ADDI immediate는 signed이므로 큰 주소를 상·하위로 나눌 때 보정이 필요할 수 있습니다. 주소 `0x12345ABC`를 만들려면 다음과 같이 표현할 수 있습니다.

```
lui t0,0x12346
addi t0,t0,-1348      # -1348 = -0x544
```

`0x12346000 - 0x544 = 0x12345ABC`입니다. 하위 12비트 ABC의 sign bit가 1이므로 상위 부분에 carry 보정이 필요합니다. `%hi`, `%lo`, `la`와 linker relocation은 이런 주소 분할을 처리합니다. 주소를 무조건 상위 20bit와 하위 12bit의 양수로 나누면 오류가 납니다.

현재 `call` pseudo-instruction은 relaxation을 끈 빌드에서 보통 AUIPC+JALR 조합으로 나타납니다. linker가 목적지까지의 거리를 반영하며, 앱의 고정 함수 포인터 호출은 앞에서 본 LUI+JALR 형태로 나타납니다. 같은 “호출”도 주소를 얻는 방식에 따라 앞부분 명령이 달라집니다.

3.19 CSR 명령 6종과 pseudo-instruction

CSR 명령은 SYSTEM opcode 안에서 funct3로 구분하며, CSR 주소는 `[31:20]`입니다. rs1 field를 register 번호로 쓰는 형태와 5비트 immediate로 쓰는 형태가 있습니다.

명령	rd에 반환하는 값	CSR에 요청하는 값
CSRRW	이전 CSR 값	rs1 값
CSRRS	이전 CSR 값	이전 값 OR rs1
CSRRC	이전 CSR 값	이전 값 AND NOT rs1
CSRRWI	이전 CSR 값	zero-extended zimm
CSRRSI	이전 CSR 값	이전 값 OR zimm
CSRRCI	이전 CSR 값	이전 값 AND NOT zimm

CSRRS/CSRRC는 rs1 field가 x0일 때 쓰기를 생략합니다. 일반 register의 내용이 우연히 0인 것과 인코딩이 x0인 것은 구분됩니다. immediate set/clear는 zimm=0이면 쓰기를 생략합니다. CSRRW/CSRRWI는 값을 교체하는 명령이므로 0도 유효한 쓰기입니다. 표준 의미와 예외는 [Zicsr 명세](#)를 참고하고 실제 write mask는 `rv32_csr.v`를 확인하십시오.

```
csrr t0,mcause      # csrrs t0,mcause,zero
csrw mie,t0         # csrrw zero,mie,t0
csrsi mstatus,8     # csrrsi zero,mstatus,8
```

이 pseudo-instruction들을 각각 새 하드웨어 명령으로 세지 않습니다. CSR은 memory-mapped register와 주소 공간도 다릅니다. mcause의 12비트 CSR 번호 `0x342`를 RAM 주소 `0x342`로 load하는 것은 CSR 읽기가 아닙니다.

3.20 구현된 CSR와 빠진 기능

CSR 주소	이름	현재 RTL의 저장·읽기·쓰기 동작
0x300	mstatus	MIE bit 3, MPIE bit 7만 갱신
0x304	mie	MTIE bit 7만 저장
0x305	mtvec	하위 2bit를 0으로 저장, direct mode
0x340	mscratch	32비트 scratch 저장; 현재 trap entry에서는 미사용
0x341	mepc	하위 2bit를 0으로 한 복귀 PC
0x342	mcause	마지막 trap 원인
0x344	mip	timer_irq에서 MTIP bit 7을 조합 생성
0xF11 - 0xF14	ID CSR	읽으면 0, write case 없음

현재 core는 `misa`, `mtval`, `mcycle`, `minstret`, `satp`, PMP CSR 등을 제공하지 않습니다. 인식되지 않는 CSR 주소는 `csr_valid=0`이 되어 illegal instruction으로 처리됩니다. `mip`와 ID CSR처럼 읽기는 가능하지만 write case가 없는 대상에 대한 쓰기는 무시되며, 완전한 표준 read-only CSR 위반 trap 검사도 구현하지 않았습니다.

M-mode만 사용하므로 MPP를 포함한 일반적인 privilege 전환 상태를 관리하지 않습니다. `mret`가 더 낮은 privilege로 내려가는 기능도 현재 사용·구현되지 않습니다. 이 표는 현재 OS가 기대하는 최소 CSR 계약이며, 범용 OS를 올릴 수 있는 전체 privileged 구현 목록이 아닙니다.

3.21 decoder의 기본값과 write-back 선택

`always @*` 첫 부분은 `next_pc=pc+4`, `rd_we=0`, `dwstrb_r=0`, `csr_we=0` 같은 기본값을 지정합니다. 이후 명령 계열별로 필요한 부분만 덮어씁니다. 이 구조는 값을 할당하지 않은 경로에서 이전 값을 유지하는 latch가 생기지 않도록 하는 데 중요합니다.

명령 계열	rd_data의 출처	memory write	next_pc 후보
ALU	<code>alu_y</code>	없음	PC+4
LUI/AUIPC	U immediate 또는 PC+U	없음	PC+4
LOAD	word/byte 선택과 확장 결과	없음	PC+4
STORE	rd write 없음	byte strobe	PC+4
BRANCH	rd write 없음	없음	PC+4 또는 branch target
JAL/JALR	PC+4	없음	jump target
CSR	이전 CSR 값	RAM write 없음	PC+4
MRET	rd write 없음	없음	mepc

주소 계산이나 PC+4가 모두 `rv32_alu` 인스턴스를 통과하는 것은 아닙니다. 현재 RTL은 `pc+4`, `rs1+imm`, branch 비교 등을 core 안의 표현식으로도 작성합니다. 합성기는 이 표현식들을 조합 회로로 구현·최적화합니다. “ALU가 하나 선언되어 있으니 FPGA 안에 덧셈 회로도 반드시 하나뿐”이라고 해석하면 안 됩니다.

3.22 trap 우선순위와 정확한 부작용 억제

core의 `take_trap`은 pending timer interrupt 또는 동기 exception에서 발생합니다. 둘이 같은 사이클에 보이면 현재 RTL은 timer를 우선 선택합니다. PC 갱신은 reset, trap, 일반 next_pc 순서입니다. 이 구현에서 어느 명령을 다시 실행할지 이해하려면 이 우선순위를 보아야 합니다.

```
assign dmem_wstrb = take_trap ? 4'b0000 : dwstrb_r;
// register write: rd_we && !take_trap
// CSR write:      csr_we && !take_trap
```

store가 decode된 사이클에 timer trap이 선택되면 write strobe를 0으로 만들어 store를 아직 실행하지 않은 것으로 유지합니다. mepc에 현재 PC를 저장하고 나중에 돌아와 store를 한 번 수행합니다. 만약 store를 먼저 실행하고 같은 PC로 돌아오면 UART 같은 부작용 장치에 두 번 출력하는 오류가 생길 수 있습니다.

RX 데이터 load에는 읽기 부작용인 FIFO pop도 있습니다. 그래서 SoC의 `rx_pop`은 `!core_trap` 조건을 포함합니다. register write와 memory write만 막고 pop을 허용하면, trap에서 돌아와 같은 load를 실행할 때 다음 문자를 읽게 되어 한 바이트가 사라질 수 있습니다. precise한 중단을 만들 때 read side effect도 고려해야 한다는 사례입니다.

3.23 세 명령을 사이클별로 따라가기

다음은 interrupt가 없는 상황의 개념 추적입니다. t0는 정렬된 RAM 주소, t1은 저장할 값이며 instruction은 별도 위치에 있다고 가정합니다.

```
sw    t1,0(t0)
lw    t2,0(t0)
addi  t2,t2,1
```

사이클	조합 계산	마지막 edge의 변경
N	SW decode, daddr=t0, ddata=t1, strobe=1111	RAM[t0]=t1, PC+4
N+1	LW decode, 갱신된 RAM 읽기, rd_data=t1	t2=t1, PC+4
N+2	ADDI decode, 새 t2 읽기, ALU=t1+1	t2=t1+1, PC+4

단일 사이클 설계에서는 이전 명령이 끝난 후 다음 명령을 계산하므로 위 의존성에 별도 hazard stall을 넣지 않습니다. 반면 pipeline에서는 동시에 여러 명령이 진행되기 때문에 동일한 코드에서 forwarding이나 load-use 제어가 필요합니다. ISA 결과는 같고 내부 실행 구조가 달라지는 것입니다.

같은 방법으로 `ecall` 사이클을 추적하면 일반 write 대신 mepc/mcause/mstatus가 바뀌고 다음 PC가 mtvec가 됩니다. 그 다음 사이클부터 trap.S의 첫 명령을 수행합니다. trap frame 전체가 하드웨어에서 한 번에 저장되는 것이 아니라 여러 SW 명령으로 시간에 걸쳐 저장됩니다.

3.24 timing, 합성 resource, 소프트웨어 비용

12.5 MHz의 clock period는 80 ns입니다. 단일 사이클 CPU의 허용 주기는 instruction memory·decoder·register read·연산·data read·write-back mux의 최장 경로와 register setup, clock 관련 여유를 포함해 정해집니다. 클록을 단순히 높이면 컴파일된 C 코드가 더 빨라지는 것이 아니라 timing 위반 가능성이 생깁니다.

이론상 정상 명령은 한 클럭에 하나씩 완료되지만 프로그램 실행 시간을 instruction 수만으로 단순 계산하기도 어렵습니다. UART ready를 기다리는 loop, syscall의 register 저장·복원, timer handler가 추가 명령을 실행합니다. 앱 자체의 static instruction이 9개라는 사실과 전체 동적 실행 명령 수는 크게 다릅니다.

LUT RAM과 register·배선 사용량은 메모리 크기와 read port에 따라 변합니다. 실제 FPGA 자원 수와 최대 동작 주파수는 합성·배선 보고서로 확인해야 합니다. RTL의 줄 수나 ISA 명령 수가 resource 수를 직접 결정하지는 않습니다. 향후 BRAM을 쓰는 multi-cycle 구조는 명령당 cycle이 늘어도 자원과 clock 측면에서 이점이 있을 수 있습니다.

3.25 BNE 기계어를 비트 단위로 검사하기

앱의 실제 `bne a0, zero, 0x400C`는 PC `0x401C`에 있으며 machine word는 `0xFE0518E3`입니다. 먼저 operand와 상대 거리를 정합니다.

```
rs1 = a0 = x10 = 010102
rs2 = zero = x0 = 000002
funct3 = 0012 (BNE)
opcode = 11000112
offset = 0x400C - 0x401C = -16
13비트 offset 표현 = 0x1FF0
```

offset을 인코딩하는 field들은 연속된 한 덩어리가 아닙니다. 아래처럼 나누어 배치합니다.

instruction 위치	넣는 값	이 예제
bit 31	offset bit 12	1
bits 30:25	offset bits 10:5	111111
bits 24:20	rs2	00000
bits 19:15	rs1	01010
bits 14:12	funct3	001
bits 11:8	offset bits 4:1	1000
bit 7	offset bit 11	1
bits 6:0	opcode	1100011

조합하면 1 111111 00000 01010 001 1000 1 1100011 이 되고 16진수로 `FE0518E3`입니다. little-endian memory의 네 byte는 `E3 18 05 FE`입니다. 다시 core의 `imm_b()` 순서로 모으면 부호 확장된 -16이 복구됩니다. assembler와 decoder가 같은 field 계약을 반대 방향으로 사용하는 것입니다.

여기서 offset의 bit 0은 항상 0으로 복구되지만 bit 1은 별도로 표현됩니다. 따라서 인코딩 가능한 모든 B target이 현재 4바이트 instruction 정렬을 만족하는 것은 아닙니다. assembler·linker가 정상 코드 label로 branch를 생성하고, hardware의 미구현 정렬 예외 한계를 알고 프로그램을 작성해야 합니다.

3.26 load와 trap을 함께 보는 타이밍 표

RX FIFO의 앞 원소가 문자 A이고 다음 원소가 B라고 가정합니다. CPU는 `RX_DATA` 주소에 LW를 수행하려 합니다. 아래 두 상황은 instruction은 같아도 edge에서의 결과가 다릅니다.

관찰	interrupt 없음	timer trap이 같은 edge에서 선택됨
조합 주소	0x100000008	0x100000008
조합 read data	A의 값 0x41	A의 값 0x41 이 보일 수 있음
rd_we && !take_trap	1	0
rx_pop	1	0
edge의 rd	A로 갱신	이전 값 유지
FIFO 다음 상태	B가 앞 원소	A가 계속 앞 원소
다음 PC	PC+4	mtvec

trap 상황에서 data wire에 A가 보였다는 이유만으로 A를 읽었다고 판단하면 안 됩니다. architectural 결과는 register write와 peripheral side effect가 실제로 commit됐는지로 결정됩니다. handler 후 같은 LW로 돌아오면 그때 A를 정상 소비합니다.

이 사고방식은 나중에 ready/valid memory bus나 pipeline에도 이어집니다. 요청이 계산된 상태, 장치가 응답한 상태, instruction이 완료된 상태를 구분해야 합니다. 현재 단일 사이클에서는 이 단계들이 가까이 붙어 있어도 trap을 넣는 순간 차이를 명확하게 다뤄야 합니다.

4. 주소 공간과 세 가지 스택 개념

4.1 셸 이미지의 메모리 지도

주소 범위/주소	크기	용도
0x00000000 - 0x00003FFF	16 KiB	커널·셸 코드, 상수, 전역 데이터, 셸 task stack
0x00004000 - 0x00005FFF	8 KiB	앱 업로드 임시 버퍼와 앱 실행 영역
0x00006000 - 0x00007FFF	8 KiB	부팅·idle 문맥의 커널 스택 예약 영역
0x10000000	레지스터	UART TX 데이터 쓰기
0x10000004	레지스터	UART TX ready 읽기
0x10000008	레지스터	UART RX 데이터 읽기, FIFO 소비
0x1000000C	레지스터	UART RX ready 읽기
0x10001000	레지스터	mtime 하위 32비트
0x10001004	레지스터	mtimecmp 하위 32비트
0x80100000 - 0x80101FFF	8 KiB	MiniFS RAM disk

firmware/linker_shell.ld는 커널의 .text, .data, .bss를 하위 16 KiB 안에 배치하고 _stack_top=0x8000을 정의합니다. 마지막 RAM byte는 0x7FFF이지만, 아래로 자라는 빈 스택의 초기 SP는 그 다음 주소인 0x8000이어도 됩니다. 첫 stack allocation이 SP를 낮추기 때문입니다.

주소 공간 4 GiB가 모두 RAM이라는 뜻은 아닙니다. 32비트 주소로 표현할 수 있는 범위 안에서 decoder가 일부 구간만 장치에 연결합니다. RAM disk의 높은 시작 주소 0x80100000은 많은 실제 RAM을 사이에 배치했다는 뜻이 아니라, 별도 장치를 구분하기 쉬운 주소를 선택한 것입니다.

linker의 MEMORY 선언도 FPGA resource를 만들지 않습니다. linker는 코드·데이터 배치를 검사하고 RTL은 실제 storage를 제공합니다. 둘이 일치해야 실행됩니다. _bss_end <= 0x4000 assertion은 하위 정적 영역이 앱 창과 겹치지 않도록 검사하지만, 실행 중 stack이 넘쳐 흐르는 문제까지 정적으로 막아 주지는 않습니다.

4.2 부팅 스택과 셸 task 스택은 다르다

부팅 시 boot.S가 설정하는 sp=0x8000은 kernel_main()과 초기 idle 문맥이 사용합니다. 반면 셸 task 스택은 다음 배열의 한 행입니다.

```
#define STACK_WORDS 256
static uint32_t stacks[NTASK - 1][STACK_WORDS]
    __attribute__((aligned(16)));
```

한 task의 스택은 $256 \times 4 = 1024$ 바이트입니다. 셸 구성은 NTASK=2이므로 idle을 제외한 한 개의 배열 스택을 둡니다. 이 배열은 .bss에 있으므로 하위 16 KiB 영역에 들어갑니다.

현재 확인한 mini_shell.elf에서는 stacks=0x1840, _bss_end=0x1C40입니다. 셸 스택은 [0x1840, 0x1C40)이고 초기 SP는 0x1C40입니다. 이 숫자는 컴파일 결과의 관측값이며 소스가 바뀌면 달라집니다. 고정된 계약은 배열 크기와 linker 경계입니다.

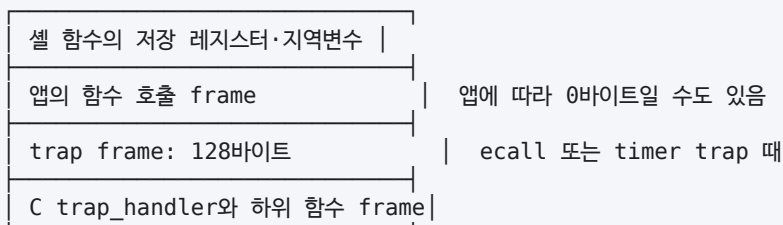
초기 셸 frame의 주소는 예제 symbol 기준 $0x1C40 - 0x80 = 0x1BC0$ 입니다. `task_sp[1]` 에는 이 frame 시작 주소가 저장됩니다. 첫 복원에서 `sp=0x1BC0` 을 선택하고 마지막에 128을 더하면 `sp=0x1C40` 으로 셸 함수를 시작합니다. “task_sp에는 언제나 함수가 보는 현재 SP가 저장된다”고 생각하면 128바이트 차이를 설명할 수 없습니다.

idle stack은 배열 `stacks` 의 slot 0이 아닙니다. `task_sp[0]` 은 첫 interrupt 때 부팅 stack 위에 만들어진 frame을 가리킵니다. 배열은 `NTASK-1` 개만 선언되므로 slot 1은 `stacks[0]` 에 대응합니다. task 번호, 배열 인덱스, 실제 stack 주소를 구분하면 `task_create`의 `slot-1` 도 자연스럽게 이해됩니다.

4.3 앱이 사용하는 스택

`run` 은 새 스택을 만들거나 SP를 `0x6000` 으로 옮기지 않습니다. 셸 task가 일반 함수 호출 방식으로 앱에 진입하므로 앱은 셸의 1 KiB 스택을 이어서 사용합니다. 앱이 syscall을 발생시키면 trap frame과 C handler의 stack frame도 이 스택에 놓입니다.

높은 주소: 셸 stack top



낮은 주소: 스택 성장 방향 ↓

8 KiB 앱 영역은 코드·포함된 데이터용 공간입니다. 앱이 8 KiB 전용 스택을 가진다는 뜻이 아닙니다. 큰 지역 배열, 깊은 재귀, 긴 함수 호출 체인은 1 KiB의 공유 스택을 넘을 수 있으며 현재는 guard page나 stack overflow trap이 없습니다.

스택에는 함수의 반환 주소가 항상 자동 저장되는 것은 아닙니다. JAL/JALR는 먼저 ra register를 갱신하고, compiler가 필요하다고 판단할 때 SW로 stack에 저장합니다. leaf 함수는 stack을 전혀 사용하지 않을 수도 있지만, trap은 언제 발생할지 모르므로 최소 128바이트와 handler 호출 여유를 별도로 고려해야 합니다.

스택 사용량을 분석할 때는 `task_shell` → `shell_run` → `app_main`이라는 C의 호출 구조와 compiler의 inline 결과를 함께 보아야 합니다. 최적화로 일부 함수가 합쳐져 symbol이 없어질 수 있습니다. 실습 확장에서는 GCC의 stack 사용량 보고와 ELF prologue의 `addi sp, sp, -N` 을 대조하고, 최대 호출 깊이에 trap 비용을 더해 예산을 잡을 수 있습니다. 큰 지역 buffer 하나만 보지 말고 동시에 살아 있는 frame의 합을 평가해야 합니다.

4.4 실행 이미지와 저장 파일의 수명 비교

동일한 앱이 세 곳에 존재할 수 있습니다. host의 `.bin`, MiniFS의 APP1 파일, unified RAM의 실행 image입니다. 하나를 바꾸거나 지워도 다른 복사본이 자동으로 모두 바뀌지는 않습니다.

동작	host .bin	MiniFS hello.app	실행 RAM 0x4000
<code>make hello-app</code>	새로 생성/갱신	변화 없음	변화 없음
upload 성공	읽기만 수행	header+payload로 교체	업로드 임시 buffer 내용
<code>run hello.app</code>	사용하지 않음	읽기만 수행	payload만 다시 복사
앱이 <code>.data</code> 수정	변화 없음	변화 없음	해당 byte 변경

동작	host .bin	MiniFS hello.app	실행 RAM 0x4000
rm hello.app	변화 없음	table entry 해제	이전 실행 byte가 남을 수 있음
PL 재구성	변화 없음	초기화	초기 펌웨어 이미지 상태

파일 삭제 후에도 실행 RAM에 machine code가 남을 수 있지만 `run`은 파일부터 읽으므로 삭제한 이름으로 정상 재실행하지 못합니다. 반대로 host `.bin`을 지워도 이미 MiniFS에 저장된 파일은 보드 전원이 유지되는 동안 run할 수 있습니다. 파일 저장과 실행 image의 수명을 분리하는 것이 loader 설계의 기본입니다.

현재는 별도 permission이 없으므로 임의의 함수 pointer가 실행 RAM 잔여 내용을 호출하는 것까지 막지는 않습니다. 그러나 정상 셸 인터페이스의 실행 경로는 항상 파일 검증을 거칩니다. interface의 동작 규칙과 모든 machine instruction에 대한 보호는 다른 수준의 보장입니다.

5. 리셋에서 셸 프롬프트까지

5.1 구성 시 초기화와 CPU reset의 차이

FPGA 구성 시 `rv32_soc`의 초기화 코드가 프로그램 RAM과 RAM disk를 0으로 초기화하고 `MEM_HEX`를 프로그램 RAM에 적재합니다. 합성에서는 이 초기값이 FPGA 구성 데이터에 반영됩니다. FPGA가 켜질 때 호스트의 파일 시스템에서 `.hex`를 다시 읽는 것은 아닙니다.

CPU reset은 PC·CSR· 주변 상태를 초기화하지만 RAM disk 내용을 지우는 RTL 경로는 없습니다. `boot.S`는 `.bss`를 지우며, `fs_init()`은 RAM disk의 superblock이 유효하면 기존 파일을 유지합니다. 이 차이는 실습에서 매우 중요합니다.

동작	프로그램 실행 상태	MiniFS 파일
screen 접속·종료	CPU는 계속 실행	유지
SW20 CPU reset	부팅 코드부터 다시 실행	유효한 파일 시스템이면 유지
PL 비트스트림 재다운로드	구성 초기 상태부터 실행	초기화됨
전원 차단	실행 중단	휘발성 RAM 내용 소실

CPU reset은 모든 `.data`를 원본 이미지로 되돌리는 일반적인 프로그램 재적재와도 다릅니다. 현재 부팅 코드는 `.bss` 초기화만 수행하므로 앞으로 수정 가능한 초기값 전역변수를 추가할 때는 warm reset 정책도 검토해야 합니다.

초기화 주체도 구분해야 합니다. RTL `initial`의 RAM 초기값은 FPGA configuration 경로의 일이고, `boot.S`의 zero loop는 RISC-V CPU가 SW 명령으로 수행하는 일입니다. 후자는 CPU reset 때마다 다시 실행되지만 전자는 CPU reset만으로 반복되지 않습니다. 이 차이가 `.bss`는 지워지고 RAM disk는 유지되는 동작을 만듭니다.

`fs_init()`이 기존 파일을 보존하는 판단은 superblock 네 word가 맞는지에 달려 있습니다. table 전체의 모든 범위나 checksum을 검증하는 mount 과정은 아닙니다. 따라서 reset 후 magic이 맞다고 손상된 모든 metadata가 복구되는 것은 아닙니다. 현재 실습의 reset 보존은 정상적으로 기록된 파일 시스템을 전제로 합니다.

5.2 boot.S의 초기화 순서

리셋 PC는 `0x00000000`입니다. linker가 `_start`를 그 위치에 배치합니다.

```
PC=0 → _start
1. sp ← _stack_top
2. [_bss_start, _bss_end)를 0으로 채움
3. mtvec ← trap_entry
4. kernel_main 호출
```

스택을 먼저 준비해야 C 함수가 지역변수와 반환 주소를 저장할 수 있습니다. `.bss`를 지워야 `ticks`, `cur`, `task pointer`와 배열이 정상 초기 상태가 됩니다. `mtvec`를 먼저 설정해야 이후 interrupt나 `ecall`이 올바른 handler로 이동합니다.

zero loop의 `bgeu t0,t1,2f`는 시작 주소가 끝 주소에 도달하면 끝낸다는 unsigned 주소 비교입니다. `sw zero,0(t0)` 뒤 `addi t0,t0,4`로 한 word씩 지우며, `j 1b`는 앞에 있는 숫자 label 1로 되돌아갑니다. `2f`의 `f`는

forward, 1b의 b는 backward입니다. linker가 끝을 4바이트 경계로 맞추므로 word 반복이 경계를 안전하게 처리할 수 있습니다.

mtvec는 일반 함수 포인터와 비슷한 주소를 담지만 CPU가 trap 때 직접 참조하는 CSR입니다. `call kernel_main` 이전에 설정해 두면 이후 예외 진입 경로가 정의됩니다. 다만 처음 몇 명령이 실패하는 경우까지 완전히 복구하는 boot 구조는 아닙니다. 초기 SP·정렬·메모리 이미지가 옳다는 기본 전제 위에서 다음 환경을 단계적으로 구성합니다.

5.3 가짜 초기 trap frame으로 셸 시작하기

`kernel_main()`은 `fs_init()` 다음에 `task_create(1, task_shell)`을 실행합니다. `task_create()`는 셸 스택 꼭대기에 128바이트 frame을 만들고, mepc 슬롯에 `task_shell` 주소를 넣습니다. ra 슬롯에는 task 함수가 반환했을 때 머물 `task_exit` 주소를 넣습니다.

이 frame은 실제 interrupt가 저장한 것은 아닙니다. 그러나 형식이 같으므로 trap 복귀 코드를 재사용할 수 있습니다. 초기 timer interrupt에서 scheduler가 이 frame을 선택하면 복원 코드의 `mret`가 최초로 `task_shell`에 진입합니다.

이 방법은 scheduler가 “처음 시작”과 “중단 후 재개”를 동일한 restore 함수로 처리하게 합니다. 처음에는 mepc를 entry 주소로 만든 인공 frame을, 다음부터는 trap.S가 저장한 실제 frame을 사용합니다. task마다 별도 시작용 jump 코드를 많이 두지 않아도 됩니다.

`ra=task_exit`는 task entry가 실수로 반환했을 때 잘못된 주소로 가지 않도록 하는 최소 처리입니다. 이 함수는 현재 무한 loop일 뿐 자원 해제나 task 제거를 수행하지 않습니다. 앱의 `return`은 이 경로가 아니라 `shell_run`의 호출 지점으로 돌아갑니다. task의 최초 entry return과 앱 함수 return을 구분해야 합니다.

5.4 timer를 켜 뒤 나타나는 두 메시지

`kernel_main()`은 `mini shell boot`를 출력하고 다음 tick을 예약합니다. `mie.MTIE=1`, `mstatus.MIE=1`로 timer interrupt를 허용한 뒤 idle loop에 머뭙니다. 첫 tick이 셸 task를 시작시키면 셸이 다음을 출력합니다.

```
Mini Shell ready. Type help.  
rv>
```

이 메시지는 출력된 시점의 UART 데이터입니다. 나중에 터미널을 접속해도 자동으로 재전송되지 않습니다. Enter를 보내면 빈 명령행 처리 후 새 프롬프트가 출력되므로 현재 셸 상태를 확인할 수 있습니다.

timer에는 두 단계 enable이 있습니다. `mie`의 `MTIE`는 machine timer라는 원인을 허용하고, `mstatus`의 `MIE`는 전역 허용입니다. 실제 조건은 `MIE && MTIE && timer_irq`입니다. compare를 먼저 미래로 잡은 뒤 enable을 켜야 초기화 도중 원치 않는 interrupt에 들어갈 가능성을 줄일 수 있습니다.

부팅 문자열은 `kernel_main`에서 직접 UART 함수를 호출하고, 셸 프롬프트는 task가 `SYS_PUTC`를 반복 호출하는 경로로 출력됩니다. 화면에는 연속된 두 문자열처럼 보여도 실행 문맥과 진입 경로가 다릅니다. 첫 메시지만 보이고 두 번째가 보이지 않는다면 timer enable, 초기 frame, MRET 경로를 의심하는 근거가 됩니다.

5.5 최초 task 시작의 PC·SP 변화표

다음 표는 관측한 symbol 주소를 사용한 개념 추적입니다. kernel_main의 정확한 현재 SP는 compiler frame과 interrupt 시점에 따라 달라지므로 KSP 로 표시합니다. task stack top과 trap vector는 현재 산출물의 값입니다.

시점	PC/실행 위치	SP와 frame	의미
reset 해제	<code>_start=0x0</code>	register reset 값에서 시작	아직 C 실행 환경 없음
<code>la sp, _stack_top</code> 이후	boot.S	SP=0x8000	boot stack 준비
task_create 완료	kernel_main	<code>task_sp[1]=0x1BC0</code>	셀 최초 frame 미리 생성
첫 timer trap 직전	idle loop	SP=KSP	boot/idle 문맥 실행 중
trap frame 저장	<code>trap_entry=0x40</code>	frame=KSP-128	idle register와 PC 보존
schedule 반환	trap.S	a0=0x1BC0	slot 1 선택
frame 복원 완료	MRET 직전	SP=0x1C40	셀의 초기 SP 준비
MRET 직후	task_shell entry	SP=0x1C40	셀 함수의 prologue 시작

실제 task_shell이 시작하면 compiler가 자신의 frame 크기만큼 SP를 더 낮출 수 있습니다. 따라서 C 본문의 SP가 항상 0x1C40인 것은 아닙니다. 최초 entry에서의 SP, 함수 prologue 이후의 SP, trap 진입 이후의 SP를 서로 다른 관측 시점으로 나누어 기록하십시오.

이 구조에서 첫 시작도 MRET를 사용한다는 점이 중요합니다. 셀 함수가 일반 `call task_shell`로 시작되는 것이 아니라 scheduler가 선택한 artificial frame을 복원하여 시작됩니다. 이후 앱 호출은 셀 함수 내부의 일반 JALR이므로 두 시작 경로를 비교해 볼 수 있습니다.

6. UART에서 명령행까지

6.1 serial bit, byte, command line의 세 단계

UART RX 핀에는 한 번에 1비트만 들어옵니다. `uart_rx`가 start bit, 8개의 data bit, stop bit를 받아 한 바이트로 만듭니다. `rv32_soc`는 이 바이트들을 FIFO에 보관합니다. C의 `task_shell()`은 바이트들을 문자열로 모아 명령어로 해석합니다.

```
UART_RX 핀 → uart_rx → rx_data[7:0], rx_valid
      |
      v
    RX FIFO 64 bytes
      | MMIO load
      v
    SYS_GETC 결과 a0
      |
      v
line[96]에 문자 누적
      | Enter
      v
    shell_command(line)
```

사용자가 `ls`와 `Enter`를 누르면 보드에는 보통 `0x6C`, `0x73`, `0x0D` 순서가 도착합니다. 세 byte는 각각 독립된 UART frame이며, 문장 전체에 대해 하나의 start bit가 붙는 것이 아닙니다. 셸이 command line을 완성하는 시점은 마지막 CR/LF를 읽은 뒤입니다.

UART는 수신과 송신이 별도 선으로 구성되므로 full-duplex 전송이 가능합니다. 키 입력을 받으면서 echo를 보낼 수 있지만 OS의 처리 속도와 FIFO 용량은 여전히 제한입니다. physical full-duplex라는 사실이 무한히 빠른 입력을 안전하게 받아 준다는 뜻은 아닙니다.

6.2 uart_rx와 uart_tx는 어디에 있는가

두 모듈의 인스턴스는 `zcu104_top`에 있습니다. `rv32_soc`에는 CPU가 접근하는 레지스터와 FIFO가 있고, top이 이 바이트 인터페이스를 물리적인 직렬 송수신 회로에 연결합니다.

```
uart_rx #(.CLOCK_HZ(12_500_000), .BAUD(115_200)) serial_rx (
    .clk(cpu_clk), .rst(rst), .rx(UART_RX),
    .data(rx_data), .valid(rx_valid)
);
```

CPU는 UART bit timing을 직접 세지 않습니다. RX 회로는 입력 핀을 동기화한 뒤 start bit 중량을 확인하고 각 bit 중량을 표본화합니다. 정상적인 stop bit가 확인되면 data를 확정하고 valid를 한 클록 동안 올립니다.

설정은 115200-8-N-1입니다. 12.5 MHz에서 정수 분주값은 108이므로 회로의 nominal bit period는 108클록이며 실제 baud는 약 115741입니다. 이는 이 RTL의 분주식으로 계산한 값입니다. CPU 한 사이클은 80 ns이지만 UART 한 바이트에는 start/stop을 포함해 약 86 μ s가 걸립니다.

RX 상태 전이를 클록 관점에서 보면 IDLE은 high인 선에서 low를 기다리고, START는 반 bit 뒤에도 low인지 확인합니다. BITS는 한 bit 간격으로 0번 bit부터 7번 bit까지 채우고, STOP은 다음 bit가 high인지 확인합니다. 짧은 low 잠음이 반 bit 뒤 사라졌다면 false start로 IDLE로 돌아갑니다. stop bit가 틀리면 현재 구현은 valid를 올리지 않습니다.

TX는 {stop=1, data[7:0], start=0} 의 10비트 frame을 shift하는 상태 기계입니다. ready는 busy의 반대이며, SoC에서 생성한 등록된 data/valid가 다음 clock sampling으로 전달됩니다. ready가 낮을 때 firmware가 무조건 써도 CPU가 자동으로 기다리는 버스가 아니므로, software의 polling 규약이 송신 손실을 막는 데 필요합니다.

6.3 FIFO와 polling

RX FIFO는 64바이트를 저장하며 read/write pointer로 비었는지 가득 찼는지를 구분합니다. CPU가 다른 작업을 하는 동안 도착한 몇 글자를 잠시 보관할 수 있습니다. 가득 찼는데 같은 사이클에 pop도 하지 않으면 새 바이트는 저장되지 않습니다. 현재는 RTS/CTS나 overflow 통지 레지스터가 없습니다.

```
#define UART_RX_DATA (*(volatile uint32_t *)0x10000008u)
#define UART_RX_READY (*(volatile uint32_t *)0x1000000cu)
```

SYS_GETC handler는 ready의 bit 0이 1이면 data를 읽고, 비어 있으면 -1을 돌려줍니다. FIFO가 비었는지 먼저 검사해야 데이터 값 0과 “문자 없음”을 혼동하지 않습니다. RX 데이터 읽기는 다음 원소로 read pointer를 이동시키는 부작용이 있습니다.

현재 RX는 interrupt 방식이 아닙니다. 셸이 SYS_GETC를 반복 호출하는 polling 방식이며, 입력이 없으면 SYS_YIELD를 호출합니다. timer interrupt는 별도 경로로 계속 동작합니다.

현재 간단한 버스에는 별도 read-enable 신호가 없고 RX pop은 주소 일치·FIFO 상태·trap/reset gating으로 생성됩니다. 펌웨어는 RX 데이터 주소를 read 전용으로 사용합니다. 향후 범용 버스로 확장할 때는 유효한 load transaction과 pop을 명시적으로 연결하는 편이 적절합니다.

read/write pointer는 7비트이고 배열 인덱스에는 하위 6비트를 사용합니다. 하위 6비트가 같아도 상위 wrap bit가 다르면 한 바퀴 차이가 있으므로 full입니다. 7비트 전체가 같으면 empty입니다. 원소 64개에 주소 bit 6개만 필요하지만 포인터에는 추가 상태 bit 하나가 필요한 이유입니다.

예를 들어 write pointer가 64, read pointer가 0이면 둘 다 배열의 index 0을 가리키지만 FIFO에는 64바이트가 있습니다. CPU가 하나를 읽는 edge에 새 byte도 들어오면 같은 사이클의 pop을 이용해 새 byte를 받을 수 있습니다. 약 11520byte/s에서 64바이트는 약 5.6 ms 분량이므로, 긴 handler나 긴 출력이 입력 소비를 막으면 overflow 위험을 분석해야 합니다.

6.4 한 줄을 만드는 코드

task_shell()의 line buffer는 96바이트입니다. 끝의 NUL을 위해 한 바이트를 남기므로 일반 명령행은 최대 95문자입니다. 인쇄 가능한 ASCII 문자를 받으면 buffer에 추가하고 sys_putc()로 다시 출력합니다. 이 동작이 키 입력이 화면에 보이는 echo입니다.

Enter에 해당하는 CR(0x0D) 또는 LF(0x0A)를 받으면 문자열 끝에 NUL을 넣고 shell_command()를 호출합니다. CR 바로 뒤에 오는 LF는 last_cr로 무시하여 CRLF 입력이 두 번의 Enter로 처리되지 않도록 합니다. Backspace(0x08)와 Delete(0x7F)는 직전 문자 하나를 지웁니다.

출력 쪽에서 \n은 셸 모드의 uart_putc()가 \r\n으로 바꿉니다. LF는 줄을 내리고 CR은 가로 위치를 맨 앞으로 돌립니다. CR이 없으면 터미널 설정에 따라 rv>가 오른쪽으로 밀리는 현상이 나타날 수 있습니다.

line buffer는 입력을 기록하는 배열이고 FIFO는 아직 CPU가 읽지 않은 byte를 보관하는 hardware queue입니다. 두 buffer는 위치와 역할이 다릅니다. line이 길어졌다고 FIFO 크기가 늘어나지 않으며 FIFO가 비었어도 아직 Enter가 오지

않았다면 line에는 미완성 명령이 남아 있을 수 있습니다.

95문자를 넘으면 overflow를 세워 Enter에서 line too long을 출력합니다. 일부 문자만 잘린 위험한 명령을 자동 실행하지 않도록 한 처리입니다. word()는 공백을 NUL로 바꾸며 같은 line 배열 안의 token pointer를 반환합니다. 따옴표, escape, pipe, 변수 확장, command history를 처리하는 일반 shell 문법은 구현하지 않았습니다.

6.5 built-in command와 외부 앱

help, ls, cat, write, rm, upload, run은 셸 펌웨어에 미리 포함된 built-in 명령입니다. shell_command()의 문자열 비교 분기로 선택됩니다. hello.app은 나중에 업로드하는 파일이며, run이라는 built-in 명령이 이 파일을 실행합니다.

일반 명령행 buffer의 95문자 제한이 앱 크기 제한은 아닙니다. upload 명령행을 해석한 후에는 별도의 수신 반복문이 hex stream을 처리하기 때문입니다.

write hello.txt Hello World에서 첫 token은 명령, 두 번째는 이름이며 뒤의 문자열이 파일 내용입니다. 현재 write는 파일 전체 교체이고 append가 아닙니다. cat은 text로 의미를 해석하지 않고 읽은 byte들을 UART로 보내므로 binary 앱 파일을 cat하면 제어문자나 읽기 어려운 값이 나올 수 있습니다. 앱 검사는 cat 출력 대신 크기·header·host binary를 이용합니다.

run hello.app도 parser 단계에서는 문자열 명령입니다. run 분기가 선택된 뒤에야 loader가 파일 형식을 검사합니다. 따라서 이름 확장자 .app 자체가 실행 가능 여부를 결정하지 않습니다. header와 길이 검사를 만족해야 하며, 실행 진입 주소는 현재 고정 ABI로 결정됩니다.

6.6 문자 C의 serial frame을 손으로 그리기

문자 C의 ASCII 값은 0x43, 이진수는 01000011입니다. UART는 least significant bit부터 보내므로 data bit의 시간 순서는 1, 1, 0, 0, 0, 0, 1, 0입니다.

시간 →	idle	start	d0	d1	d2	d3	d4	d5	d6	d7	stop	idle
선의 값	1	0	1	1	0	0	0	0	1	0	1	1

각 bit는 nominal 108개의 12.5 MHz clock 동안 유지

RX의 shift register는 도착한 bit를 shift[bit_no]에 기록하므로 시간 순서가 LSB first여도 최종 data는 0x43이 됩니다. memory의 little-endian word 순서와 UART bit 전송 순서는 다시 다른 층위입니다. 하나는 multi-byte 객체의 주소 순서이고 다른 하나는 한 byte의 serial wire 순서입니다.

start bit를 찾은 뒤 매 bit 중앙을 sample하는 이유는 경계 근처의 전환을 피하기 위해서입니다. host와 FPGA의 baud가 정확히 같지는 않아도 한 frame 안에서 sample 위치가 data 안정 구간에 남으면 통신할 수 있습니다. 현재 회로는 간단한 단일 sample 방식이며 복잡한 oversampling·majority vote 수신기는 아닙니다.

7. ECALL과 운영체제 서비스의 경계

7.1 시스템 콜 번호는 OS가 정한다

RISC-V의 ECALL은 실행 환경에 예외를 발생시키는 명령입니다. 문자 출력, 파일 생성 등 서비스의 종류는 CPU의 opcode가 아니라 OS가 정한 레지스터 규약으로 전달합니다. 이 프로젝트는 a7에 서비스 번호, a0부터 인수를 넣습니다.

번호	서비스	입력	결과
1	SYS_PUTC	a0 : 문자	문자 출력
2	SYS_YIELD	없음	scheduler 선택 문맥으로 복귀
3	SYS_GETTICK	없음	a0 : ticks
4	SYS_FS_CREATE	a0 : 이름 주소	a0 : 0 또는 -1
5	SYS_FS_WRITE	a0 : 이름, a1 : 데이터, a2 : 크기	a0 : 기록 크기 또는 -1
6	SYS_FS_READ	a0 : 이름, a1 : 버퍼, a2 : 용량	a0 : 파일 크기 또는 -1
7	SYS_FS_DELETE	a0 : 이름	a0 : 0 또는 -1
8	SYS_FS_LIST	없음	UART 목록 출력, a0=0
9	SYS_GETC	없음	a0 : 0 - 255 또는 -1
10	SYS_FS_READ_AT	a0 : 이름, a1 : 버퍼, a2 : 용량, a3 : offset	a0 : 읽은 크기, EOF 0, 오류 -1

이는 이 Mini OS만의 ABI입니다. Linux의 syscall 번호와 호환되지 않습니다. 함수 호출에서 쓰는 표준 레지스터 규약과 OS별 syscall 번호 규약도 구분해야 합니다.

포인터 인수는 문자열 자체가 아니라 메모리 주소입니다. SYS_FS_WRITE에 a0=name, a1=data를 넘기면 handler는 같은 주소 공간에서 그 위치를 읽습니다. 현재는 user/kernel 주소 변환이나 안전한 copy-in 검사가 없으므로 caller가 올바른 주소와 길이를 제공해야 합니다. 나중에 U-mode로 나누면 이 전제가 달라집니다.

반환값 -1은 register에서는 0xFFFFFFFF라는 bit pattern입니다. C stub이 이를 int로 해석하므로 ch < 0으로 입력 없음·오류를 판단합니다. 정상 UART byte 0xFF는 zero-extension한 255로 반환하므로 -1과 다릅니다. API의 signed 반환형을 잘못 unsigned로 바꾸면 error 검사가 달라질 수 있습니다.

7.2 한 문자를 출력하는 앱 코드

```
static void putc_sys(char c)
{
    register uint32_t a0 __asm__("a0") = (unsigned char)c;
    register uint32_t a7 __asm__("a7") = 1u;
    __asm__ volatile ("ecall" : "+r"(a0) : "r"(a7) : "memory");
}
```

register ... __asm__("a0")는 inline assembly operand를 지정된 레지스터에 연결합니다. "+r"은 입력과 출력 operand, "r"은 입력 레지스터 operand입니다. volatile은 compiler가 이 assembly의 부작용을 무시하여

제거하지 못하게 하며, "memory" clobber는 주변 메모리 접근의 compiler 재배치를 제한합니다. 하드웨어 cache를 지우는 명령이라는 뜻은 아닙니다.

현재 handler는 frame을 통해 일반 레지스터를 복원하고 서비스 결과가 필요한 경우 저장된 a0 를 수정합니다. 이 구현 규약을 이용하여 앱이 ecall 이후에도 필요한 레지스터 값을 유지할 수 있습니다.

compiler는 일반 C 함수의 호출 규약만으로 inline assembly가 어떤 메모리를 바꾸는지 알 수 없습니다. 따라서 operand와 clobber 선언은 compiler와 작성자 사이의 계약입니다. "memory" 는 syscall이 pointer가 가리키는 메모리에 영향을 줄 수 있음을 알리는 보수적인 방법입니다. 단순 출력 stub도 같은 형태를 사용해 예제를 일관되게 구성했습니다.

이 stub에 a0 만 output으로 선언해도 되는 이유는 현재 OS가 다른 일반 register를 frame에서 복원한다는 구현 계약 때문입니다. 모든 OS의 ECALL에 같은 clobber 규칙을 그대로 적용할 수 있는 것은 아닙니다. 자신이 호출하는 runtime의 ABI와 compiler가 보는 operand 계약이 서로 일치해야 합니다.

7.3 하드웨어 trap 진입

M-mode에서 ecall 을 decode하면 cause 11의 동기 예외가 됩니다. core와 CSR 회로가 다음을 수행합니다.

```
mepc ← ecall 명령어 자신의 PC
mcause ← 11
MPIE ← 이전 MIE
MIE ← 0
PC ← mtvec
```

일반 레지스터를 RAM에 저장하는 일은 아직 하지 않습니다. 그 작업은 trap 진입점의 어셈블리 코드가 수행합니다. trap이 선택된 사이클에는 core가 일반 명령의 register/CSR/memory write를 막습니다.

ECALL 은 정상 완료된 명령처럼 retire되는 것이 아닙니다. 예외를 발생시킨 명령 주소가 mepc 에 저장되고, 소프트웨어가 서비스를 처리한 뒤 다음 명령으로 돌아가기로 결정합니다. 이 의미는 [RISC-V Machine-Level ISA의 Environment Call 설명](#)과 일치합니다. 소스 주석의 "completed"라는 표현은 서비스 처리 후 다음 명령으로 진행한다는 의도로 읽되, ISA의 retire 의미와 혼동하지 않아야 합니다.

mtvec 에는 trap_entry 라는 함수 이름이 아니라 linker가 정한 주소 값이 들어갑니다. 현재 관측 ELF에서 이 주소는 0x40 입니다. hardware는 symbol 이름을 모른 채 PC를 이 값으로 바꿉니다. 이후 addi sp,sp,-128 부터의 assembly가 순차 실행되어 frame을 만듭니다.

trap 진입 직후 일반 register는 아직 caller의 값을 유지합니다. 그래서 가장 먼저 t0를 임시로 덮어쓰면 원래 t0가 사라질 수 있습니다. trap.S는 원래 register들을 먼저 저장하고, x5/t0를 안전하게 저장한 뒤 그 register를 mepc 읽기 등에 사용합니다. 이 순서는 C handler보다 아래 단계의 정확성 조건입니다.

7.4 왜 f->mepc += 4인가

handler는 CAUSE_ECALL_M 이면 저장된 frame의 mepc 를 4 증가시킵니다. 현재 코어는 32비트 고정 길이 명령만 사용하므로 다음 명령 주소가 현재 주소 +4입니다. 이 증가가 없으면 mret 이후 같은 ecall 을 다시 실행하여 서비스를 무한 반복합니다.

timer interrupt의 경우에는 증가시키지 않습니다. 이 core는 interrupt를 받아들이는 사이클의 현재 명령을 commit하지 않으므로, 그 PC로 돌아가 해당 명령을 실행해야 합니다.

예를 들어 앱의 ECALL 주소는 0x4014 이고 이어지는 LBU는 0x4018 입니다. frame의 값을 0x4018 로 바꾸어도 즉시 CPU PC가 바뀌지는 않습니다. trap.S가 나중에 frame에서 이 값을 읽어 CSR mepc에 쓰고, MRET가 그 CSR을 선택할 때 제어가 이동합니다. RAM의 frame PC, CSR mepc, 현재 hardware PC는 서로 다른 저장 위치입니다.

timer와 ECALL이 같은 사이클에 보이는 경우 현재 core의 우선순위는 timer입니다. ECALL 자체는 아직 처리되지 않았으므로 timer handler는 PC를 증가시키지 않고 돌아옵니다. 이후 ECALL이 다시 decode되어 syscall을 수행합니다. 각 handler가 “이미 처리한 명령인가”를 구분해야 중복 서비스나 명령 누락을 피할 수 있습니다.

7.5 파일 읽기 syscall의 인수·결과 추적

loader가 payload 첫 60바이트를 읽는 순간을 보겠습니다. 이름 문자열은 셀의 line buffer 안에 있고, 그 주소를 NAME 이라고 표기합니다. 다음 값들이 ECALL 직전 register에 준비됩니다.

register	값	해석
a7 / x17	10	SYS_FS_READ_AT
a0 / x10	NAME	파일 이름 문자열 주소
a1 / x11	0x4000	복사할 destination buffer
a2 / x12	60	최대 읽을 byte 수
a3 / x13	12	파일 header 다음부터 읽기

trap.S가 이 값을 frame에 저장한 뒤 handler가 REG(f,17) 을 읽어 서비스를 선택합니다. fs_read_at(name,buffer,60,12) 가 RAM disk에서 실행 RAM으로 byte를 복사하고 60을 반환하면, handler는 frame의 a0 슬롯에 60을 씁니다. 복원 후 앱/셀의 a0는 NAME pointer가 아니라 결과 60입니다.

```
ECALL 전 a0 = 파일 이름 pointer
handler 중 f->x[9] = 60으로 변경
MRET 후 a0 = 60, C stub의 return 값도 60
```

argument register가 return register로 재사용되므로, caller가 이름 pointer를 나중에도 필요로 하면 compiler가 다른 register나 stack에 보존합니다. syscall wrapper의 +r(a0) 선언은 바로 이 입력·출력 재사용을 compiler에 알려 줍니다. pointer의 내용이 반환값으로 바뀌는 것이 아니라 pointer를 담았던 register가 바뀌는 것입니다.

8. Trap frame과 셸 모드의 스케줄링

8.1 frame의 정확한 메모리 형식

trap.S와 C의 struct frame은 정확히 일치해야 합니다.

```
struct frame {
    uint32_t mepc;
    uint32_t x[31];
};
#define REG(f, n) ((f)->x[(n) - 1])
```

frame 시작으로부터 offset	저장 내용
0	mepc
4	x1 / ra
8	trap 이전 x2 / sp 기록
12	x3 / gp
40	x10 / a0
68	x17 / a7
124	x31 / t6

총 크기는 $4 + 31 \times 4 = 128$ 바이트입니다. x0는 항상 0이므로 저장하지 않습니다. REG(f, 10)은 x[9]이므로 정확히 a0의 슬롯을 가리킵니다.

실제 memory 주소는 frame_base + 4*n으로 x_n 슬롯을 계산합니다. 예를 들어 frame이 0x1BC0이면 a0는 0x1BE8, a7은 0x1C04입니다. C의 x[9]는 struct 시작부터 4바이트 mepc를 지난 뒤 9개 word를 더한 위치이므로 같은 offset 40을 얻습니다. 배열 index와 architectural register 번호의 -1 관계를 이렇게 계산할 수 있습니다.

frame에 a0를 저장한 뒤 C handler의 인수 a0는 frame pointer로 바뀝니다. 원래 task의 a0와 handler의 현재 a0가 잠시 다른 의미를 갖는 것입니다. handler는 REG(f, 10)을 통해 저장된 task 값을 읽고 반환 결과도 그 슬롯에 써야 합니다. 현재 register a0에만 값을 넣고 frame을 수정하지 않으면 restore에서 이전 값으로 덮일 수 있습니다.

8.2 저장·C 호출·복원

trap_entry는 SP를 128바이트 낮추고 일반 레지스터와 mepc를 저장합니다. 이후 a0=sp로 설정하고 C의 trap_handler(frame*)를 호출합니다. C handler의 반환값 a0는 복원할 frame의 주소입니다.

```
mv    a0, sp
call  trap_handler
mv    sp, a0
```

반환된 frame이 같은 주소이면 기존 문맥으로 돌아갑니다. 다른 task의 frame이면 mv sp, a0 이후 그 task의 레지스터를 복원하므로 context switch가 됩니다. 마지막으로 mepc를 CSR에 쓰고 mret를 실행합니다.

SP는 저장된 x2 슬롯을 `lw sp, ...` 로 복원하지 않습니다. frame 주소를 SP로 선택한 다음 마지막에 128을 더하여 원래 SP를 얻습니다. 따라서 x2 슬롯은 디버깅용 기록이며, frame 배치 자체가 SP 복원 규칙입니다.

일반 함수 호출에서는 caller-saved register를 caller가 필요할 때만 저장하면 됩니다. interrupt는 어느 명령 사이에서든 발생할 수 있으므로 task가 그 시점에 어떤 register를 쓰는지 handler가 알 수 없습니다. 그래서 이 trap entry는 x1 - x31을 넓게 보존합니다. 이것이 C 함수의 보통 prologue보다 저장량이 큰 이유입니다.

C handler에서 다른 frame을 반환할 때 기존 stack에 올라간 handler frame은 C의 epilogue가 먼저 정리합니다. 그 후 assembly의 `mv sp, a0` 가 실행됩니다. 아직 C 함수가 사용하는 도중 임의로 SP를 바꾸는 방식이 아니므로 C 호출 규약을 유지하면서 context를 교체할 수 있습니다.

8.3 일반 OS와 셸 모드의 scheduler 차이

일반 OS는 idle, A, B, C 네 슬롯을 round-robin으로 선택합니다. 셸 모드의 코드는 다음처럼 항상 slot 1을 고릅니다.

```
task_sp[cur] = f;
#ifdef SHELL_MODE
    cur = 1;
#else
    if (++cur >= NTASK) cur = 0;
#endif
return task_sp[cur];
```

첫 tick은 idle에서 셸로 전환합니다. 이후 셸의 timer interrupt나 `yield`는 셸 문맥을 다시 선택합니다. 따라서 셸 모드에서 A/B/C가 계속 출력되거나 idle과 10 ms씩 번갈아 실행된다고 설명하면 실제 코드와 맞지 않습니다.

입력이 없을 때 `yield`를 호출하더라도 셸이 다시 선택되므로 현재 구현은 busy polling 성격을 가집니다. 에너지 절약형 sleep이나 입력 대기 queue는 구현되어 있지 않습니다. CPU가 빠르게 FIFO를 비울 수 있다는 장점과 불필요한 실행량이라는 비용을 함께 관찰할 수 있습니다.

일반 OS의 C task가 문자 하나를 출력한 뒤 `yield`하면 자신의 time slice를 자발적으로 끝냅니다. A/B가 계속 출력하는 것과 비교하면 C의 문자 수가 적을 수 있습니다. 공평한 round-robin 선택 횟수와 같은 실행 명령 수·UART 문자 수는 동일한 지표가 아닙니다. 셸 모드는 입력 응답을 위해 이 데모 정책과 별도의 선택 규칙을 사용합니다.

셸 slot의 frame pointer는 trap마다 현재 SP에 따라 달라질 수 있습니다. 앱이 더 깊은 함수를 호출한 시점의 interrupt는 더 낮은 stack 위치에 frame을 만듭니다. `task_sp[1]`은 고정된 초기 frame 주소만 영원히 가리키는 것이 아니라, 가장 최근에 중단된 문맥을 가리키도록 갱신됩니다.

8.4 frame에 모든 CSR이 들어 있지는 않다

이 frame은 일반 정수 레지스터와 `mepc`만 저장합니다. `mstatus`, `mcause`, 모든 CSR을 task별로 저장하는 범용 context가 아닙니다. 현재의 단일 M-mode, 비중첩 timer handler, 동일한 interrupt 정책에서 쓰는 최소 구조입니다. 사용자 모드나 중첩 interrupt, task마다 다른 CSR 상태를 지원하면 저장·복원 계약을 확장해야 합니다.

현재 `mstatus`의 MPIE는 hardware에 하나만 존재하고 task별 frame에는 복사하지 않습니다. handler가 MIE를 다시 켜지 않고 모든 task를 같은 정책으로 실행하기 때문에 간단한 restore가 가능합니다. 이 전제를 바꾸면 다른 task의 interrupt 상태를 실수로 물려주는지 검토해야 합니다.

중첩 trap을 허용하면 새 trap이 `mepc`와 `mcause`를 다시 덮어씁니다. 바깥 trap의 PC는 frame에 저장되어 있어도 cause, privilege 상태, stack 선택, 재진입 가능한 C 함수의 조건까지 따져야 합니다. `mscratch`를 이용한 전용 kernel

stack 전환은 가능한 확장 방법이지만 현재 trap.S는 이를 사용하지 않습니다.

9. MiniFS: 실행 파일을 보관하는 구조

9.1 RAM disk의 배치

MiniFS는 8 KiB를 512바이트 block 16개로 나눕니다. 저장 형식은 `minifs.h`와 `minifs.c`에 정의되어 있습니다.

block	byte offset	CPU 주소	내용
0	0x000 - 0x1FF	0x80100000 부터	superblock
1 - 2	0x200 - 0x5FF	0x80100200 부터	file table 예약 공간
3 - 15	0x600 - 0x1FFF	0x80100600 부터	file data, 총 6656바이트

superblock의 앞 네 word는 magic 0x3153464D (MFS1), version 1, block count 16, block size 512입니다. `fs_init()`은 이 네 값을 검사해 기존 형식을 인식합니다. 구조가 일치하지 않으면 `fs_format()`으로 초기화합니다.

block 번호 b 의 CPU 시작 주소는 $0x80100000 + b * 512$ 입니다. 예를 들어 block 3은 0x80100600 이고, 이 주소의 SoC RAM disk word index는 $\text{offset } 0x600 / 4 = 384$ 입니다. testbench가 `ramdisk[384]`를 검사하는 이유가 여기에 있습니다. C 파일 API의 block 번호가 RTL 배열 index로 변환되는 과정을 직접 계산할 수 있습니다.

superblock의 magic을 마지막에 기록하는 것은 초기화가 완료되기 전에 유효한 형식처럼 보이는 시간을 줄입니다. 그러나 현재 방식은 journal이나 원자적 transaction을 제공하지 않습니다. reset이나 오류가 write 중간에 끼어들었을 때 항상 이전 또는 새 상태만 보장하는 수준으로 해석하면 안 됩니다.

9.2 file table entry

```
struct file_entry {
    char name[16];
    uint32_t size;
    uint32_t start_block;
};
```

이 환경에서 entry 하나는 24바이트이고 최대 32개이므로 실제 table 크기는 768바이트입니다. 두 block의 예약 공간 1024바이트 중 256바이트는 사용하지 않습니다. 이름은 NUL을 포함한 16바이트이므로 보통 최대 15문자입니다.

파일 데이터는 `start_block`부터 연속된 block에 놓입니다. 파일 이름이나 크기가 코드의 실행 주소를 뜻하지 않습니다. `hello.app`을 block 3에 저장해도 링크 주소는 여전히 0x4000입니다.

entry 번호 i 의 주소는 $\text{DISK_BASE} + 512 + 24 * i$ 입니다. size field는 그 entry의 +16, start_block은 +20에 있습니다. 모든 entry 크기가 4의 배수이므로 두 `uint32_t` field가 자연 정렬됩니다. 여기서는 target의 32비트 정수 크기와 little-endian 배치에 기대고 있으므로, 다른 CPU에서 파일을 직접 생성한다면 같은 직렬화 규칙을 명시해야 합니다.

사용 중 여부는 이름의 첫 byte가 0인지로 구분합니다. `fs_create()`는 나머지 field를 먼저 준비하고 이름 첫 byte를 마지막에 공개합니다. 이 방식도 특정 순서에서 미완성 entry를 덜 노출하는 단순 기법이며, 멀티코어 동기화나 저장 매체 flush 규약을 대신하지는 않습니다.

9.3 할당·읽기·삭제

파일 크기에서 필요한 block 수는 다음처럼 계산합니다.

```
blocks = (size + 511) >> 9
```

할당기는 table을 조사해 사용 중인 extent와 겹치지 않는 연속 구간을 찾습니다. 현재는 별도 free bitmap이 없습니다. 전체 빈 공간이 충분해도 연속 공간이 부족하면 쓰기에 실패할 수 있습니다.

`fs_read()`는 버퍼에 파일 전체가 들어갈 때만 성공합니다. `fs_read_at()`은 offset부터 capacity만큼 부분 읽기를 허용하므로 셸의 `cat`과 앱 로더가 64바이트 단위로 읽을 수 있습니다. EOF는 0, 없는 파일이나 오류는 -1입니다.

삭제는 table entry의 이름 첫 바이트를 0으로 하고 크기와 시작 block을 비웁니다. 데이터 block을 모두 0으로 지우지는 않습니다. 이후 할당기가 그 구간을 다시 사용합니다. 삭제된 바이트가 물리적으로 즉시 사라진다는 뜻은 아닙니다.

연속 할당의 단편화를 작은 예로 보겠습니다. data block 3, 5, 7이 비어 있고 다른 block은 사용 중이면 빈 공간은 총 3개 block이지만 2개 연속 block을 요구하는 파일은 들어갈 수 없습니다. 현재 allocator는 파일의 block 목록을 나누어 기록하는 구조가 아니므로 가장 긴 연속 빈 구간이 중요한 조건입니다.

기존 파일을 덮어쓸 때 allocator는 그 파일이 쓰던 구간을 재사용 후보로 취급합니다. 공간을 찾지 못하면 이전 metadata를 바꾸지 않고 실패하지만, 복사 중 전원·reset 문제까지 보장하는 transactional write는 아닙니다. `fs_read_at`은 매 호출마다 이름을 찾아 offset을 계산하므로 fd와 지속적인 file position을 관리하는 POSIX API와도 다릅니다.

9.4 60바이트 앱이 ls에서 72바이트인 이유

호스트의 `hello_app.bin`은 순수 payload 60바이트입니다. 보드에 저장할 때 셸이 12바이트 APP1 header를 붙입니다. 따라서 MiniFS의 논리 파일 크기는 72바이트이며, 실제 할당은 한 block인 512바이트입니다.

payload	60 bytes
APP1 header	12 bytes
MiniFS file size	72 bytes
allocated disk space	512 bytes

앱 실행 영역은 8192바이트지만 전체 file data 공간은 6656바이트입니다. 여기에 header 12바이트를 제외하면 payload 최대값은 6644바이트입니다. 다른 파일이 있다면 가용 연속 공간에 따라 더 작은 앱도 실패할 수 있습니다.

file size와 할당 크기를 구분하면 앱 크기 제한도 정확히 설명할 수 있습니다. $\text{payload } 6644 + \text{header } 12 = 6656$ 바이트가 data block 13개를 전부 차지합니다. 이론적 최대 앱 하나를 저장하면 다른 비어 있지 않은 파일을 둘 공간은 남지 않습니다. 파일 table entry가 남아 있다는 사실과 data 공간이 남아 있다는 사실은 별개입니다.

현재 header는 파일 안에 저장되지만 실행 RAM에는 payload만 복사됩니다. 따라서 `ls`의 size, uploader JSON의 bytes, linker의 section size가 서로 다른 값을 표시할 수 있습니다. 실험 보고서에서는 “무엇의 크기인지” 단위를 함께 적어야 서로 모순되는 결과로 오해하지 않습니다.

9.5 두 파일을 저장하고 삭제하는 전체 예

비어 있는 FS에서 `hello.app` 72바이트와 `notes.txt` 600바이트를 차례로 기록한다고 가정합니다. 첫 파일은 한 block, 두 번째는 두 block이 필요합니다. 현재 first-fit 연속 할당에서는 다음과 같은 상태가 가능합니다.

entry	name	size	start_block	실제 data block
0	hello.app	72	3	3
1	notes.txt	600	4	4 - 5

hello.app의 첫 byte 주소는 0x80100600이고, 그 payload 첫 byte는 header 뒤인 0x8010060C입니다. 이 payload를 실행할 때는 RAM disk에서 그대로 fetch하지 않고 0x4000으로 복사합니다. notes.txt의 시작은 $0x80100000 + 4 * 512 = 0x80100800$ 입니다.

rm hello.app은 entry 0을 비우지만 block 3의 byte를 모두 지우지 않습니다. 이어서 100바이트의 new.txt를 만들면 entry 0과 block 3을 재사용할 수 있습니다. 새 파일의 size가 100이면 읽기는 100바이트까지만 허용되므로 block 뒤에 남은 이전 byte는 정상 파일 내용에 포함되지 않습니다.

이 예는 file table이 “어떤 byte가 현재 파일에 속하는가”를 정의한다는 점을 보여 줍니다. RAM 안에 bit가 남아 있는 사실과 파일 시스템에서 접근 가능한 파일이라는 사실은 다릅니다. crash consistency나 기밀 삭제가 필요한 시스템에서는 이 단순 table 정책보다 더 많은 설계가 필요합니다.

10. C 프로그램에서 실행 파일까지

10.1 세 종류의 이미지 구분

프로젝트에서 “binary”나 “hex”라고 부르는 파일들이 모두 같은 역할을 하는 것은 아닙니다.

파일	내용	소비하는 주체
zcu104_mini_shell.bit	FPGA 논리·배선·메모리 초기값 구성 데이터	FPGA configuration 회로
mini_shell.elf	커널·셸 기계어와 ELF 메타데이터	호스트의 linker·검사 도구
mini_shell.hex	프로그램 RAM 초기화용 32비트 word 텍스트	RTL \$readmemh / 합성 도구
hello_app.elf	0x4000 에 링크된 앱과 ELF 메타데이터	호스트의 objcopy, objdump
hello_app.bin	앱의 기계어·상수·포함된 데이터 바이트	업로드 스크립트
hello_app.hex	앱 payload를 word 단위로 표현한 텍스트	셸 시뮬레이션 testbench
MiniFS의 hello.app	APP1 header + payload	보드의 shell_run()

.bit 는 RISC-V CPU가 실행하는 명령어 파일이 아닙니다. .bin 은 FPGA 회로를 구성하는 파일이 아닙니다. FPGA 회로를 먼저 구성하면 그 안의 CPU가 펌웨어를 실행하고, 그 펌웨어가 나중에 업로드한 앱을 읽어 실행합니다.

확장자가 파일 내용을 강제로 바꾸지는 않습니다. .elf 이름을 .bin 으로 바꿔 업로드하면 ELF header부터 payload로 취급되어 실행에 맞지 않습니다. 반대로 .bin 에 읽기 좋은 문자가 포함되어 있어도 전체가 text file이라는 뜻은 아닙니다. 생성 도구와 변환 단계를 따라 형식을 확인해야 합니다.

ELF의 file size에는 header와 symbol table 등이 포함되므로 raw image보다 훨씬 클 수 있습니다. FPGA 실행 RAM의 필요량은 ELF 파일의 host storage 크기와 같지 않습니다. 현재 앱에 필요한 실제 loadable byte는 objcopy의 raw payload와 linker section 배치로 판단합니다.

10.2 예제 C 프로그램

firmware/hello_app.c 의 핵심은 다음과 같습니다.

```
__attribute__((section(".text.app_entry")))
void app_main(void)
{
    const char *p = "Hello from loaded app!\n";
    while (*p) putc_sys(*p++);
}
```

putc_sys() 는 7장에서 설명한 ecall stub입니다. 문자열 끝의 NUL은 반복 종료 조건이고 출력하지 않습니다. 소스의 \n 은 OS 출력 경로에서 CRLF로 바뀝니다. 함수 끝에 도달하면 caller로 반환합니다.

app_main 이라는 이름 자체를 CPU가 인식하는 것은 아닙니다. compiler와 linker가 이 함수의 기계어를 약속된 첫 위치에 배치하고, loader가 그 주소를 호출합니다. 함수 이름은 호스트 ELF의 symbol로 남아 역어셈블과 디버깅에 쓰입니다.

const char *p 는 문자열을 가리키는 pointer 변수이고 문자열 자체는 image의 상수 영역에 있습니다. p++ 는 pointer가 가리키는 byte 주소를 1씩 증가시킵니다. 문자열 길이를 미리 저장하지 않아도 NUL을 만날 때까지

반복하므로, 실행 중 필요한 ISA는 주소 덧셈·byte load·조건 branch·ECALL입니다.

app_main에 main이라는 이름을 쓰지 않은 것은 별도 C startup을 연결하지 않기 때문입니다. hosted 환경의 main(argc,argv) 호출·종료 처리를 제공하는 runtime이 없으며, loader가 약속된 함수를 직접 호출합니다. 함수 이름과 entry section annotation이 함께 맞아야 현재 build 계약을 만족합니다.

10.3 make hello-app의 빌드 단계

```
hello_app.c + app.ld
  | GCC: compile + assemble + link
  ▼
build/firmware/hello_app.elf
  | objcopy -O binary
  ▼
build/firmware/hello_app.bin
  | bin2hex.py
  ▼
build/firmware/hello_app.hex
```

Makefile은 GCC 한 명령으로 compile·assemble·link를 수행합니다. .s와 .o가 이 target에서 별도 파일로 남지 않아도 내부 단계는 존재합니다. 기계어를 확인하려면 최종 링크 주소가 확정된 ELF를 역어셈블하면 됩니다.

```
make hello-app
riscv64-unknown-elf-readelf -h -S build/firmware/hello_app.elf
riscv64-unknown-elf-objdump -d -M no-aliases,numeric \
  --start-address=0x4000 --stop-address=0x4024 \
  build/firmware/hello_app.elf
xxd -gl build/firmware/hello_app.bin
```

Nothing to be done for 'hello-app'.은 의존 파일보다 산출물이 최신이어서 다시 만들 일이 없다는 뜻입니다. 업로드나 실행이 실패했다는 메시지가 아닙니다.

compile 단계는 C를 target assembly로 내리고, assembler는 mnemonic과 operand를 instruction encoding으로 바꿉니다. linker는 여러 symbol의 최종 주소를 정하고 relocation을 해결합니다. objdump -d는 이 최종 기계어를 다시 사람이 읽는 mnemonic으로 표시합니다. .s를 .dis로 text 변환하는 도구가 아니라는 점을 구분하십시오.

Make는 target과 prerequisite의 시각으로 재생성 여부를 판단합니다. 컴파일러 옵션만 임의로 바꾼 경우 모든 의존성이 자동으로 추적되는 build 시스템은 아닙니다. 수업에서 설정을 바꾸어 결과를 비교할 때는 강제 재빌드 여부와 실제 사용된 명령행을 함께 기록해야 stale artifact를 새 결과로 오해하지 않습니다.

10.4 compiler 옵션의 의미

옵션	이 프로젝트에서의 역할
-march=rv32i	RV32I 명령을 목표로 앱 코드 생성
-mabi=ilp32	int, long, pointer를 32비트로 사용하는 ABI
-O1	기본 최적화 적용; 실제 명령 수는 소스 줄 수와 다름
-ffreestanding	일반 hosted C 실행 환경을 전제로 하지 않음
-fno-builtin	표준 함수 의미를 이용한 compiler 변환 제한

옵션	이 프로젝트에서의 역할
<code>-fno-pic</code>	현재의 고정 주소 코드 생성 정책
<code>-msmall-data-limit=0</code>	gp 기반 small-data 배치를 피하는 설정
<code>-mno-relax</code> , linker의 <code>--no-relax</code>	gp 초기화 등이 필요한 링크 최적화 경로 방지
<code>-nostdlib -nostartfiles</code>	표준 startup·libc·기본 라이브러리 자동 링크 생략
<code>-T firmware/app.ld</code>	앱 메모리 배치·entry 규칙 지정

tool 이름에 `riscv64`가 있어도 `-march=rv32i -mabi=ilp32`를 사용하면 이 환경에서는 ELF32 RISC-V 결과가 만들어집니다. 현재 앱은 `printf`, `malloc` 같은 libc 서비스가 없습니다. C의 곱셈·나눗셈도 무조건 금지되는 것은 아닙니다. 상수 연산은 shift/add로 바뀔 수 있지만 runtime helper가 필요한 연산을 쓰면, `libgcc`를 자동 링크하지 않기 때문에 미해결 symbol이 생길 수 있습니다.

현재 앱은 CSR 명령을 직접 사용하지 않습니다. 나중에 다른 버전의 GNU toolchain으로 커널을 빌드할 때 CSR instruction에 대한 ISA extension 지정 규칙이 달라질 수 있으므로 도구의 오류와 실제 지원 ISA를 맞춰야 합니다. 기존 Makefile의 도구 조합에서 확인한 설정을 최신 toolchain 전체에 대한 보장으로 읽지 않아야 합니다.

`-march`와 `-mabi`는 각각 하드웨어 명령 선택과 함수 호출·자료형 약속을 결정합니다. 예를 들어 명령어는 RV32I로 제한하면서도 다른 ABI를 섞으면 함수 인수나 객체 배치가 호환되지 않을 수 있습니다. 앱과 OS 사이의 syscall stub, linker 주소, 자료형 폭을 함께 맞춰야 합니다.

freestanding이라는 옵션은 compiler가 helper나 library 호출을 절대 만들지 않는다는 보장이 아닙니다. 큰 struct 복사, 64비트 나눗셈 등에서 지원 routine이 필요할 수 있습니다. 현재 `-nostdlib` 구성에서 unresolved reference가 나오면 해당 symbol이 어디서 필요한지 disassembly와 link 오류로 확인하고, 연산을 단순화하거나 필요한 구현을 명시적으로 제공해야 합니다.

10.5 ELF에서 raw binary로 바뀌면서 사라지는 것

ELF에는 entry address, section 정보, symbol 등이 있습니다. `objcopy -O binary` 결과에는 그 메타데이터가 포함되지 않습니다. 메모리에 실릴 바이트들이 raw image로 남습니다. GNU 도구의 동작은 [objcopy 공식 문서](#)에서도 확인할 수 있습니다.

그래서 `hello_app.bin`의 첫 바이트만 보고 원래 링크 주소가 `0x4000`이었는지 일반적으로 복원할 수 없습니다. 현재 loader는 그 정보를 파일에서 읽는 대신 환경의 고정 규칙으로 알고 있습니다.

현재 ELF의 `.text` 주소는 `0x4000`이지만 raw `.bin` 앞에 16384바이트의 0이 자동으로 붙는 것은 아닙니다. 이 예제의 가장 낮은 loadable section부터 image가 시작하므로 파일 offset 0이 실행 주소 `0x4000`에 대응합니다. section 사이에 주소 간격이 있으면 raw image의 간격 처리도 확인해야 하지만, 현재 예제는 연속된 60바이트입니다.

APP1 header의 크기 field는 raw payload 크기만 기록합니다. entry, architecture, required privilege, relocation 정보는 header에 없습니다. loader가 단순한 이유는 이 정보를 모두 플랫폼의 고정 약속으로 둔 결과입니다. 파일 형식을 확장할수록 그 약속 일부를 metadata로 옮기고 검증하는 책임이 생깁니다.

11. 링커와 앱 ABI의 계약

11.1 ENTRY와 실제 첫 바이트는 별개다

앱 linker script의 주요 부분은 다음과 같습니다.

```
OUTPUT_ARCH(riscv)
ENTRY(app_main)
MEMORY { APP (rwx) : ORIGIN = 0x00004000, LENGTH = 8K }
SECTIONS
{
    . = ORIGIN(APP);
    .text : ALIGN(4) {
        KEEP(*(.text.app_entry))
        *(.text .text.*)
        *(.rodata .rodata.*)
    } > APP
    .data : ALIGN(4) { *(.data .data.*) } > APP
    .bss (NOLOAD) : ALIGN(4) { *(.bss .bss.*) *(COMMON) } > APP
    ASSERT(SIZEOF(.bss) == 0,
           "minimum app format does not support .bss")
}
```

ENTRY(app_main) 은 ELF의 진입점 정보를 지정합니다. 이 기능은 [GNU ld의 Entry Point 문서](#)에 설명되어 있습니다. 현재 raw loader는 ELF header를 읽지 않으므로, 이 선언만으로 올바른 실행이 보장되지는 않습니다.

실제 보장은 app_main 을 .text.app_entry 에 넣고, 그 section을 0x4000 의 첫 위치에 배치하는 데서 나옵니다. KEEP 는 section garbage collection을 사용하는 구성에서도 해당 입력 section을 유지하도록 하는 지시입니다. 현재 Makefile은 garbage collection 옵션을 쓰지 않지만 entry의 의도를 명시합니다.

output section의 순서가 raw image의 layout을 결정합니다. KEEP(*(.text.app_entry)) 뒤의 *(.text .text.*) 가 나머지 함수 코드를 모으고, 이어서 .rodata 를 같은 output .text 에 넣습니다. linker script를 읽을 때 input section 이름과 output section 이름을 구분하면 disassembly에 문자열이 섞이는 이유도 설명할 수 있습니다.

ENTRY 를 다른 symbol로 바꿔 ELF entry만 변경해도 현재 loader는 그 값을 읽지 않습니다. 이 실험으로 ELF metadata와 raw loader의 고정 규약을 구분할 수 있습니다. 진입 위치를 바꾸려면 파일 형식에 entry offset을 추가하고 검증하거나, entry section이 실제로 첫 byte에 놓이도록 유지해야 합니다.

11.2 왜 다른 주소에 복사하면 안 되는가

현재 예제는 문자열 주소 0x4024 를 명령어의 immediate로 구성합니다. payload 전체를 0x5000 으로 옮겨 실행하면 코드 안의 문자열 참조는 여전히 0x4024 를 가리킵니다. 상대 branch 일부가 맞게 동작해도 절대 주소를 포함한 참조는 틀릴 수 있습니다.

임의 주소 실행을 원하면 position-independent code와 그 실행 환경을 설계하거나, relocation 정보를 유지하고 loader가 적용해야 합니다. 현재 APP1 header에는 relocation table이 없습니다.

고정 주소 코드에는 함수·문자열·전역변수 주소가 여러 형태로 들어갈 수 있습니다. 명령어에 완성된 32비트 주소 한 개가 그대로 놓이는 경우만 생각하면 안 됩니다. LUI와 ADDI의 조합처럼 서로 떨어진 immediate field에 나뉘어 표현될 수도 있습니다. raw byte를 무작정 일정 값만큼 수정하는 relocation은 올바르지 않습니다.

현재 방식의 장점은 loader가 relocation type을 해석하지 않아도 된다는 것입니다. 비용은 앱 하나가 사용할 수 있는 위치가 고정된다는 점입니다. 여러 앱을 동시에 배치하려면 각각 다른 고정 주소로 링크하거나, position-independent 규칙·relocation·가상주소 중 하나 이상의 방법을 도입해야 합니다.

11.3 앱 시작 때 이미 준비된 것

앱은 `boot.S` 를 다시 실행하지 않습니다. 셸이 준비한 SP, CSR, timer, trap vector, UART, 파일 시스템 환경을 그대로 사용합니다. app entry의 함수형은 `void app_main(void)` 이며 인수와 반환값을 정식 ABI로 전달하는 코드는 없습니다.

일반 함수 호출에서 `ra` 는 반환 주소이고 `sp` 는 stack pointer입니다. `s0 - s11` 은 callee-saved이며 `a·t` 계열은 caller-saved입니다. 표준 정수 호출 규약은 16바이트 stack alignment를 사용합니다. 이 레지스터·정렬 규칙의 기준은 [RISC-V psABI의 Calling Convention](#)입니다. syscall 번호는 이 표준이 아닌 Mini OS 코드가 정합니다.

현재 앱은 셸로 정상 반환하려면 callee-saved 레지스터와 SP를 지켜야 합니다. 별도 syscall `exit` 는 없습니다. `return` 이 일반 함수 복귀 명령으로 변환되어 셸로 돌아옵니다.

caller-saved register는 앱이 바뀌도 셸 compiler가 호출 앞뒤에서 필요한 값을 처리합니다. callee-saved register는 앱 compiler가 사용하면 저장·복원해야 합니다. 이것은 “OS가 모든 앱 함수를 위해 register를 자동 보존한다”는 의미가 아닙니다. 일반 호출은 ABI로, 비동기 trap은 trap frame으로 보호하는 두 기구가 함께 작동합니다.

현재 `gp` 를 초기화하는 startup은 앱에 없습니다. 그래서 small-data와 relaxation 관련 설정을 보수적으로 선택했습니다. `tp` 기반 TLS, C++ 전역 생성자, 동적 초기화, 환경변수, heap 제공도 이 최소 runtime의 계약에 포함되지 않습니다. 단순 C 함수에서 자연스럽게 쓰던 환경 의존 기능을 독립 앱에 가져올 때 이 초기화 책임을 점검해야 합니다.

11.4 .data와 .bss

초기값이 있는 `.data` 는 raw image에 포함되면 실행 RAM으로 함께 복사됩니다. 따라서 동일 앱을 다시 run 하면 파일에 저장된 초기 데이터가 다시 로딩될 수 있습니다.

반면 `.bss` 는 일반적으로 파일에 초기값 바이트를 저장하지 않고 실행 전 0으로 초기화하는 영역입니다. 현재 APP1 형식은 `.bss` 주소·크기를 저장하지 않으며 loader도 이를 지우지 않습니다. 그래서 linker가 `.bss` 크기 0을 강제합니다.

```
static unsigned counter; /* 대개 .bss가 필요하므로 현재 앱 형식에서 거부 */
```

stack에 두는 지역변수까지 금지하는 것은 아닙니다. 다만 작은 공유 스택을 사용합니다. `const char *p = "..."` 의 문자열은 이 linker에서 `.rodata` 가 `.text` 출력 section에 합쳐져 payload에 포함됩니다.

예를 들어 초기값이 7인 수정 가능한 전역변수가 `.data` 에 배치되면 파일에 그 7이 들어갑니다. 앱이 실행 중 9로 바꾸어도 MiniFS의 파일을 직접 수정하지 않았다면 다음 run의 재로딩으로 다시 7이 됩니다. 실행 RAM의 수정과 저장 파일의 수정은 독립적입니다.

미초기화 전역변수의 `.bss` 는 “컴파일러가 0을 채운 byte를 파일에 저장한다”는 방식이 아닐 수 있습니다. 그래서 raw image 크기만큼 복사하는 loader로는 C가 기대하는 zero initialization을 일반적으로 제공할 수 없습니다. linker assertion은 이 환경의 제한을 link 시점에 드러내는 장치입니다. section을 임의로 바꾸어 assertion을 우회하기보다 runtime 규칙을 명확히 확장해야 합니다.

11.5 앱 간 연결과 외부 symbol

앱을 컴파일할 때 `kernel.c`와 다시 링크하지 않습니다. 커널 내부 함수 이름을 임의로 호출하면 linker는 그 정의를 찾을 수 없습니다. `syscall`은 커널 함수의 주소를 앱에 고정하지 않고 서비스를 이용하는 경로입니다.

현재는 동적 linker, shared library, symbol lookup table이 없습니다. 더 큰 라이브러리를 쓰려면 앱과 함께 정적으로 링크하고 크기·ISA·초기화 조건을 검토하거나 명시적인 OS 서비스로 제공해야 합니다.

`syscall`의 이점은 커널 내부 함수의 위치가 바뀌어도 서비스 번호와 의미가 유지되면 앱 호출 경로를 유지할 수 있다는 것입니다. 현재 `uart_putc`의 실제 주소는 build마다 바뀔 수 있지만 앱은 그 주소를 모르고 `a7=1`을 사용합니다. 이 경계가 binary interface의 중요한 사례입니다.

향후 서비스를 추가할 때는 번호뿐 아니라 인수 자료형·길이 단위·오류 값·register 보존 규칙도 문서화해야 합니다. kernel이 지원하지 않는 번호는 현재 -1을 반환합니다. `syscall` 이름을 C header로 공유하면 숫자의 오타를 줄일 수 있지만, 이미 빌드된 앱과 새 kernel의 호환성은 여전히 ABI 버전 정책으로 관리해야 합니다.

11.6 하위 함수를 호출하는 앱의 stack 예제

현재 hello 앱은 leaf 형태이지만 일반 앱은 여러 함수를 호출할 수 있습니다. 아래는 A를 세 번 출력하고 셀에 반환하는 호출 규약 설명용 assembly입니다. 기존 `hello_app.c`를 바꾸는 명령이 아니라 stack 사용을 읽기 위한 별도 예제입니다.

```
.option norvc
.section .text.app_entry,"ax",@progbits
.globl app_main
app_main:
    addi sp,sp,-16
    sw   ra,12(sp)
    sw   s0,8(sp)
    li   s0,3
.Lprint_again:
    call print_a
    addi s0,s0,-1
    bnez s0,.Lprint_again
    lw   s0,8(sp)
    lw   ra,12(sp)
    addi sp,sp,16
    ret

.section .text,"ax",@progbits
print_a:
    li   a0,65
    li   a7,1
    ecall
    ret
```

`app_main`은 셀이 전달한 `ra`를 stack의 +12에 저장합니다. `call print_a`가 `ra`를 내부 복귀 위치로 덮어써도 마지막에 저장된 셀 복귀 주소를 복원할 수 있습니다. `s0`는 callee-saved이므로 `app_main`이 loop counter로 사용한 뒤 원래 값을 복구합니다. 16바이트를 할당하여 호출 지점의 stack alignment를 유지합니다.

`print_a` 안의 `ECALL`은 새로운 function-call `ra`를 만들지 않습니다. trap handler가 사용하는 register들은 trap frame에서 복원되므로 `print_a`의 `ret`는 `app_main`의 loop로 돌아옵니다. 이후 `app_main`의 `ret`는 복원한 `ra`를 사용해 셀로 돌아옵니다. 같은 `ret`라도 호출 깊이에 따라 가리키는 caller가 다릅니다.

SP가 S인 상태로 앱에 들어왔다면 일반 앱 frame은 [S-16, S) 에 있고, print_a의 ECALL frame은 그 아래 [S-144, S-16) 에 놓입니다. C handler frame은 더 아래로 자랍니다. stack 사용 예산을 “앱이 16바이트만 쓴다”로 끝내지 않고 비동기 처리 비용까지 더해야 하는 이유입니다.

12. UART 업로드 프로토콜과 APP1 형식

12.1 host와 board의 대화

업로드 스크립트는 처음 CR을 보내 현재 셸의 새 프롬프트를 받습니다. 이어서 파일 이름, payload 바이트 수, checksum을 command line으로 보냅니다.

```
host → board : CR
board → host : rv>
host → board : upload hello.app 60 000010dd + CR
board → host : send hex:
host → board : b747000093874702... (총 120개의 hex 문자)
board → host : uploaded
board → host : rv>
```

payload의 byte 수는 10진수입니다. checksum은 8자리 16진수이며 $\text{sum}(\text{payload}) \& 0xFFFFFFFF$ 로 계산합니다. 명령 도움말의 `sum8`은 8자리 hex 표현을 가리키며 8비트 checksum을 뜻하지 않습니다.

처음 보내는 CR은 host와 board의 대화 경계를 맞추기 위한 동작입니다. 그 뒤의 `rv>`를 기다리면 셸이 command line을 받을 준비가 됐음을 확인할 수 있습니다. 하지만 이전에 중단된 upload가 payload를 기다리는 상태라면 CR은 공백으로 무시될 뿐 prompt를 만들지 못할 수 있습니다. 현재 protocol에는 상태를 강제로 되돌리는 독립 escape sequence가 없습니다.

host가 `send hex:`를 확인한 뒤 payload를 보내는 이유는 command parser와 payload receiver의 전환을 기다리기 위해서입니다. 데이터 전송이 끝난 뒤에도 `uploaded`와 prompt를 확인해야 실제 file write까지 끝났다고 판단할 수 있습니다. host의 write 호출이 성공했다는 사실만으로 보드의 저장 완료가 보장되지는 않습니다.

12.2 왜 raw byte 대신 ASCII hex를 보내는가

byte `0xB7`은 문자 `'b'`, `'7'` 두 개로 보냅니다. 보드는 첫 hex digit을 상위 4비트, 두 번째를 하위 4비트로 조합합니다. 공백, CR, LF, tab은 수신 parser가 건너뜁니다.

ASCII hex는 사람이 관찰하기 쉽고 작은 parser로 구현할 수 있지만 전송량이 두 배입니다. 115200 baud, 8-N-1의 이론적 byte rate는 약 11520바이트/초이므로 payload의 이상적인 전송률은 약 5760바이트/초입니다. 60바이트 payload는 120개의 UART 문자가 되어 약 10.4 ms가 필요하며, 명령·응답·OS 처리 시간은 별도입니다.

hex 수신기는 대문자 A - F도 허용하므로 `b7`과 `B7`은 같은 byte가 됩니다. 각 byte를 만들 때 상위 nibble을 읽은 뒤 하위 nibble을 기다리며, 그 사이의 허용 공백도 건너뜁니다. 따라서 줄을 나누어 보내는 것이 가능하지만, 빠진 hex digit 하나는 이후 byte 경계를 모두 어긋나게 할 수 있습니다.

현재 host는 hex stream을 최대 256문자 단위로 write하지만 이 단위가 board의 ACK packet은 아닙니다. 각 조각마다 확인을 받고 보내는 flow control이 없으므로 RX FIFO와 CPU 소비 속도에 의존합니다. 작은 예제의 성공을 더 큰 데이터 전송의 무조건적인 신뢰성으로 일반화하지 않아야 합니다.

12.3 두 가지 hex 표현은 바이트 순서가 다르다

첫 명령어 `0x000047B7`을 예로 들어 봅니다.

표현	내용
.bin 의 바이트 순서	B7 47 00 00
UART ASCII hex stream	b7470000
\$readmemh 32비트 word 한 줄	000047b7

UART는 raw byte의 순서를 유지합니다. bin2hex.py 는 4바이트를 little-endian 정수로 해석해서 한 word를 출력합니다. 따라서 .hex 텍스트를 그대로 upload payload로 붙여 넣으면 같은 데이터가 되지 않습니다. 호스트 스크립트에는 .bin 을 전달해야 합니다.

little-endian은 byte의 memory 순서를 말하고, hex 문자열에서 한 byte의 상위 digit을 먼저 쓰는 관례와는 다른 총위입니다. 0xB7 한 byte를 표현할 때는 항상 b7 로 쓰지만, 32비트 word의 네 byte 순서는 낮은 byte부터 전송합니다. “little-endian이므로 문자열의 모든 글자를 뒤집는다”는 규칙은 없습니다.

bin2hex.py 는 마지막 byte 수가 4의 배수가 아니면 0을 덧붙여 word를 완성합니다. 이 padding은 \$readmemh 표현을 위한 것이고 host uploader는 원래 .bin 길이를 사용합니다. 현재 60바이트 예제는 이미 4의 배수라 추가 padding이 없습니다.

12.4 셸의 업로드 임시 버퍼

shell_upload() 는 앱 실행 창을 임시 버퍼로도 씁니다.

```
업로드 중의 unified RAM
0x4000 .. 0x400B : APP1 header를 만들 자리
0x400C ..      : UART에서 복원한 payload
```

payload를 전부 받은 뒤 byte sum을 검사하고, header를 작성합니다. 그 다음 SYS_FS_CREATE 와 SYS_FS_WRITE 로 header+payload 전체를 MiniFS에 저장합니다. 이 시점에는 파일이 저장된 것이며, 0x4000 에는 아직 실행할 첫 명령어가 아니라 header가 있습니다.

재실행 시에는 shell_run() 이 파일의 offset 12부터 payload만 읽어 0x4000 에 배치합니다. 이 차이를 이해하지 못하면 header를 명령어로 실행하거나 앱을 12바이트 어긋난 주소로 실행하는 loader를 만들기 쉽습니다.

임시 buffer가 실행 창을 공유할 수 있는 이유는 업로드를 받는 동안 이전 앱이 실행 중이지 않기 때문입니다. 현재는 셸과 앱이 동기적인 함수 호출 관계이므로 앱이 반환한 후 다음 명령을 받습니다. 비동기 앱 실행으로 확장한다면 실행 중인 image를 upload가 덮어쓰지 않도록 별도의 buffer나 lifecycle 보호가 필요합니다.

60바이트를 받을 때 임시 payload는 0x400C - 0x4047 에 놓고 header는 0x4000 - 0x400B 입니다. 저장되는 전체 72바이트를 MiniFS에 복사한 뒤 run은 payload를 0x4000 - 0x403B 로 다시 로딩합니다. 두 시점의 RAM 지도에서 시작 위치가 12바이트 다른 것을 직접 계산해 보십시오.

12.5 APP1 header의 바이트 단위 해석

파일 offset	길이	값	의미
0	4	0x31505041	little-endian 바이트로 41 50 50 31, 즉 APP1
4	4	payload 크기	header를 제외한 바이트 수

파일 offset	길이	값	의미
8	4	byte sum	unsigned payload 바이트 합의 하위 32비트
12	가변	payload	고정 주소 0x4000 에 링크된 기계어·데이터

현재 60바이트 예제의 header와 첫 명령어는 다음과 같습니다.

```
41 50 50 31  3C 00 00 00  DD 10 00 00  B7 47 00 00 ...
└ APP1 ─┘   └ 60 ─┘   └ 0x10DD ┘   └ first insn ┘
```

checksum은 전송 실수 일부를 찾는 용도입니다. byte 두 개를 서로 바꾸거나 합이 같은 값으로 바꾸면 검출하지 못할 수 있습니다. 암호학적 서명, 작성자 확인, 명령어 검증을 제공하지 않습니다.

checksum은 unsigned byte를 하나씩 더합니다. 예를 들어 0xFF는 -1이 아니라 255로 더해야 host의 `sum(payload)`와 같습니다. 합의 누산형이 `uint32_t`인 것은 파일 형식 규약을 분명히 하기 위한 것이며, 현재 최대 payload 6644바이트의 최대 합은 16942200이므로 실제 허용 크기 안에서는 32비트를 넘지 않습니다.

magic의 정수 값과 문자 순서도 직접 검산할 수 있습니다. 낮은 byte부터 41 50 50 31이므로 ASCII로 A P P 1이 됩니다. 이 12바이트 구조에는 alignment padding이 따로 끼지 않으며 세 `uint32_t`를 연속해서 저장합니다. 다른 host tool로 APP1을 만들 때는 host native endian 대신 little-endian을 명시해야 합니다.

12.6 업로드 실패와 현재 프로토콜의 범위

hex digit이 아니면 `bad hex`, 합이 다르면 `checksum mismatch`, 파일 쓰기가 실패하면 `upload failed`가 출력됩니다. host 스크립트에는 응답 timeout이 있지만 보드의 `receive_hex_byte()`에는 수신 timeout이나 취소 명령이 없습니다. payload 전송이 중단되면 셸이 나머지 바이트를 계속 기다릴 수 있습니다.

또한 host는 `screen` 등 다른 reader의 존재를 자동으로 확실히 차단하지 않습니다. 둘이 동시에 같은 serial port를 읽으면 각자 일부 응답을 가져갈 수 있습니다. 실습에서는 한 번에 한 프로그램이 UART를 사용하도록 합니다.

전송 오류가 날 때는 host 쪽 serial timeout과 board가 출력한 오류를 구분합니다. `checksum mismatch`는 보드가 정해진 수의 byte를 받아 합까지 계산했다는 증거입니다. timeout은 그 이전의 어느 단계에서도 생길 수 있으며 baud·연결·포트 중복·board state를 좁혀야 합니다.

파일 교체는 수신 checksum이 맞은 다음 진행되므로 합이 틀린 payload를 즉시 기존 file data에 쓰지는 않습니다. 하지만 실패 후 자동 재시도, 불완전한 입력 drain, 임시 파일을 사용한 commit은 구현하지 않았습니다. 신뢰할 수 있는 대량 전송을 설계할 때는 현재의 단순 성공 경로에 이러한 상태 처리를 더해야 합니다.

13. run 명령의 검증·복사·호출

13.1 실제 동작 순서

`run hello.app` 을 처리하는 `shell_run()` 은 다음 순서로 동작합니다.

1. 파일의 처음 12바이트를 읽고 APP1 magic을 검사합니다.
2. payload 크기가 4 이상이며 6644와 실행 창 크기 이내인지 검사합니다.
3. 파일 offset 12부터 64바이트 이하씩 읽어 APP_BASE+offset 에 복사합니다.
4. 복사한 payload의 byte sum을 계산합니다.
5. 기록된 합과 같은지, 선언된 payload 뒤에 불필요한 byte가 없는지 검사합니다.
6. `running` 을 출력하고 `0x4000` 을 함수처럼 호출합니다.
7. 앱이 반환하면 `returned` 를 출력합니다.
8. 명령 처리로 돌아간 셸이 `rv>` 를 출력합니다.

현재 `SYS_EXEC` 라는 별도 시스템 콜은 없습니다. `run` 로더 자체는 셸 C 코드에 있으며 파일 읽기에 `FS syscall` 을 사용하고, 복사가 끝나면 직접 함수 포인터를 호출합니다. 장래의 OS 설계에서 process 생성 기능을 kernel syscall로 옮길 수 있지만, 그것은 다음 단계의 변경입니다.

검사의 순서도 의미가 있습니다. 크기를 먼저 검사해야 잘못된 size로 kernel 영역이나 stack에 복사하지 않습니다. 복사된 바이트의 합을 확인한 뒤 호출해야 전송·저장 중 일부 손상을 실행 전에 찾을 수 있습니다. 다만 형식 검증은 명령어의 동작 분석이 아니므로 문법적으로 APP1인 위험한 code를 막는 보호 장치로 해석하면 안 됩니다.

loader가 중간에 실패하면 `running` 은 출력하지 않고 함수 호출도 하지 않습니다. 이미 일부 payload가 실행 창에 복사되었더라도 다음 정상 run은 file에서 다시 덮어씁니다. loader의 실패 처리와 실행 창의 잔여 내용은 구분해야 하며, 현재는 실패 때 전체 창을 0으로 지우지는 않습니다.

13.2 파일 offset과 목적지 주소

MiniFS 파일		unified 실행 RAM
offset 0: APP1 header	→	loader의 지역 header[3]
offset 12: payload[0]	→	0x4000
offset 13: payload[1]	→	0x4001
...		...
offset 12+N-1	→	0x4000+N-1

header는 실행 RAM 이미지의 앞에 남겨두지 않습니다. $N=60$ 이면 payload 마지막 byte는 `0x403B` 입니다. 파일 시스템의 block 위치와 실행 위치 사이에는 “내용을 복사한다”는 관계만 있습니다.

copy loop에서 `off` 는 payload 기준 offset, FS 읽기의 offset은 `12+off` 입니다. 두 변수를 혼동하면 첫 12바이트를 건너뛰지 않거나, 반대로 payload 중간부터 읽는 문제가 생깁니다. 첫 iteration은 `off=0` , source file offset 12, destination `0x4000` , chunk는 `min(size, 64)` 입니다.

payload가 150바이트라면 chunk는 64, 64, 22바이트이고 destination은 각각 `0x4000` , `0x4040` , `0x4080` 입니다. 마지막 byte 주소는 `0x4000+149=0x4095` 입니다. 64바이트 chunk는 stack에 큰 파일 buffer를 만들지 않고도 파일 전체를 처리하게 해 줍니다.

13.3 왜 마지막 byte를 한 번 더 읽는가

loader는 offset 12+size 에서 1바이트 읽기를 시도하고 EOF인 0을 기대합니다. 파일이 선언된 길이보다 짧으면 복사 중 truncated app, 길이보다 길거나 합이 틀리면 bad app checksum 으로 거절합니다. header와 실제 file size의 일관성을 검사하는 것입니다.

다만 파일이 CPU에 맞는 모든 명령으로 구성되었는지, 잘못된 주소에 store하지 않는지, 정상적으로 반환하는지까지 검사하지는 않습니다. APP1 header와 byte sum만으로 실행의 안전성이 보장되지 않습니다.

허용되는 file size는 정확히 12+header.size 입니다. header가 60을 선언했는데 file size가 71이면 마지막 read가 부족해 truncated, 73이면 EOF 확인에서 추가 byte가 발견됩니다. 선언된 60바이트는 맞아도 한 byte 값이 달라 합이 틀리면 checksum 오류입니다. 길이 검사와 내용 검사는 서로 보완합니다.

현재 최소 크기는 4바이트지만 payload 전체 크기가 반드시 4의 배수인지는 검사하지 않습니다. code 이후 문자열·데이터 길이는 임의 byte 수일 수 있기 때문입니다. entry에 최소 한 instruction이 있다는 크기 조건과 모든 code target이 올바르게 정렬되었다는 조건은 다릅니다.

13.4 함수 포인터 호출의 의미

```
((void (*)(void))APP_BASE)();
```

정수 주소 0x4000 을 “인수가 없고 반환값도 없는 함수의 주소”로 해석한 뒤 호출합니다. 이 코드는 해당 GCC·bare-metal 실행 환경에서 사용하는 구현 방식입니다. 임의의 C 환경에서 숫자 주소를 함수처럼 호출해도 된다는 뜻은 아닙니다.

현재 확인한 셸 ELF에서는 다음 명령어가 나왔습니다.

```
0x0A14: lui    a5, 0x4
0x0A18: jalr   ra, 0(a5)
0x0A1C: ...           # 앱에서 돌아온 뒤 실행
```

JALR 는 ra=0x0A1C 를 기록하고 PC를 0x4000 으로 변경합니다. core는 같은 명령어를 일반 함수 호출에도 사용합니다. 파일 실행을 위한 별도의 “EXEC instruction”은 필요하지 않습니다. 이 주소들은 현재 빌드 관측값이며 코드 수정·compiler 최적화에 따라 달라질 수 있습니다.

CPU에게 실행 파일의 개념을 추가하지 않아도 되는 이유를 이 한 줄에서 볼 수 있습니다. loader는 자료를 RAM에 쓰는 C 프로그램이고, 함수 포인터 호출은 기존 JALR로 구현됩니다. CPU는 loader가 어떤 이름의 파일을 읽었는지 몰라도 address 0x4000의 bit pattern을 정상 instruction으로 decode합니다.

현재 함수형이 void이므로 앱의 a0에 남은 값을 exit status로 출력하지 않습니다. 반환 코드를 도입하려면 entry를 int app_main(void) 같은 계약으로 정하고 셸이 결과를 수집하도록 함께 바꿔야 합니다. 표준 C의 main exit code 규칙이 이 함수 포인터 호출에 자동으로 생기지는 않습니다.

13.5 반환 경로가 살아 있어야 하는 이유

앱이 ra를 훼손하거나 SP를 복구하지 않으면 정상 반환이 실패할 수 있습니다. 다른 함수를 호출하는 앱은 compiler가 필요한 prologue/epilogue와 ra 저장을 생성합니다. 수동 assembly로 앱을 작성할 때는 같은 규칙을 직접 지켜야 합니다.

현재 예제는 출력 stub이 inline되어 leaf 함수 형태이고, `ecall` 은 하드웨어에서 `ra` 를 바꾸지 않습니다. trap 내부 C 호출이 `ra` 를 사용해도 trap frame에서 원래 값이 복원됩니다. 따라서 예제의 마지막 `ret` 는 셸이 넣어 준 복귀 주소를 사용할 수 있습니다.

앱 내부에서 다른 함수를 JAL로 호출하면 `ra` 는 그 내부 복귀 주소로 바뀝니다. compiler는 앱이 끝난 뒤 셸로 돌아갈 주소가 필요하므로 stack 등에 기존 `ra` 를 보존하고 epilogue에서 복원합니다. 이를 수동 assembly로 작성한다면 save/restore를 빠뜨리지 않아야 합니다.

현재 예제의 `ecall` 은 `ra` 를 destination으로 지정하지 않지만 trap.S의 `call trap_handler` 는 `ra` 를 씁니다. 원래 앱의 `ra` 는 그 전에 frame offset 4에 저장되어 있으므로 syscall 뒤에도 셸 복귀 주소가 유지됩니다. 함수 호출 규약과 trap 저장 규약이 만나는 구체적인 사례입니다.

13.6 잘못된 앱 파일에 대한 검증 경로

아래 표는 현재 `shell_run()` 의 조건식을 기준으로 예상한 동작입니다. 모든 사례가 이미 자동 test에 포함되어 있다는 뜻은 아닙니다. 후속 negative test의 기대값으로 사용할 수 있습니다.

파일 상태	실패하는 조건	출력	앱 진입
이름이 없음	첫 12바이트 read 결과가 -1	not an app file	없음
file size가 8	첫 read 결과가 12보다 작음	not an app file	없음
첫 12바이트는 있으나 magic이 다름	header[0] 비교	not an app file	없음
magic은 맞고 payload size=3	최소 크기 검사	invalid app size	없음
size=60인데 payload가 59바이트	chunk read 크기 비교	truncated app	없음
size=60이고 합이 다름	누산 sum 비교	bad app checksum	없음
size=60, 합은 맞지만 trailing byte 존재	EOF read가 0이 아님	bad app checksum	없음
형식은 맞지만 unsupported instruction 포함	loader 검사는 통과 가능	실행 중 PANIC 가능	있음

마지막 행이 특별히 중요합니다. loader는 code의 의미를 증명하는 static analyzer가 아닙니다. 파일 형식이 일관되어도 앱이 잘못된 주소에 쓰거나 무한 loop에 들어갈 수 있습니다. 실행 전에 알 수 있는 구조적 조건과 실행 중에만 드러나는 동작을 나누어 검증해야 합니다.

부정 입력 test에서는 오류 문자열에 더해 kernel image와 file table이 의도치 않게 바뀌지 않았는지, PC가 앱 창으로 넘어가지 않았는지를 관측할 수 있습니다. 복사 도중 실패하면 실행 창 일부는 바뀔 수 있으므로 “모든 RAM이 불변”이라는 잘못된 기대 조건을 세우지 않아야 합니다.

14. 60바이트 앱의 실제 명령어 추적

14.1 명령어와 문자열의 분리

현재 산출물은 총 60바이트이며 앞 36바이트가 9개의 32비트 명령어, 뒤 24바이트가 문자열과 종료 NUL입니다.

주소	기계어 word	명령어	동작
0x4000	000047B7	lui a5,0x4	a5=0x4000
0x4004	02478793	addi a5,a5,36	문자열 주소 0x4024 구성
0x4008	04800513	addi a0,zero,72	첫 문자 H=0x48
0x400C	00178793	addi a5,a5,1	다음 문자 주소로 이동
0x4010	00100893	addi a7,zero,1	SYS_PUTC 선택
0x4014	00000073	ecall	문자 출력 요청
0x4018	0007C503	lbu a0,0(a5)	다음 문자 읽기
0x401C	FE0518E3	bne a0,zero,0x400C	NUL이 아니면 반복
0x4020	00008067	jalr zero,0(ra)	셀로 복귀, ret 의 실제 명령
0x4024 - 0x403B	문자열 byte	Hello from loaded app!\n\0	실행할 명령이 아닌 상수 데이터

compiler는 첫 문자가 상수 H임을 알고 첫 load 대신 immediate를 사용했습니다. 소스의 while 문과 pointer 증가가 기계어에서는 이런 배치가 된 것입니다. C의 한 줄과 한 명령어를 일대일로 대응시키면 이 최적화를 이해하기 어렵습니다.

첫 instruction 0x000047B7의 memory byte는 B7 47 00 00이며 fetch port가 이를 32비트 word로 decoder에 공급합니다. rd=15이므로 a5를 쓰고, U immediate 0x4000을 그대로 전달합니다. 다음 ADDI가 36을 더해 문자열 주소를 완성합니다. raw file, memory word, assembly operand의 세 표현을 같은 값으로 연결해 볼 수 있습니다.

이 표에서 9개는 static instruction 수입니다. 반복문의 0x400C - 0x401C 구간은 문자열의 각 문자마다 여러 번 수행됩니다. syscall handler와 trap.S 명령들은 다른 주소에 있으며 표의 9개에 포함되지 않습니다. 앱 파일이 짧은다는 사실이 전체 OS 작업량이 9클록이라는 뜻은 아닙니다.

14.2 첫 문자 H의 전체 이동

PC=0x4008에서 a0=72가 됩니다. 0x4010에서 a7=1, 0x4014에서 ECALL 예외가 발생합니다. trap frame의 a0 슬롯에는 0x48, a7 슬롯에는 1, mepc에는 0x4014가 저장됩니다.

C handler는 mcause=11을 확인하고 frame의 mepc를 0x4018로 바꿉니다. SYS_PUTC 분기에서 uart_putc('H')를 호출하고 TX ready를 기다린 뒤 0x10000000에 0x48을 씁니다. SoC는 TX data/valid를 만들고 uart_tx가 이를 start/data/stop bit로 직렬화합니다. PC의 터미널이 그 바이트를 문자 H로 표시합니다.

handler가 같은 frame을 반환하면 trap.S가 레지스터와 mepc를 복원하고 mret합니다. PC는 0x4018이 되고 다음 문자 e를 읽습니다. 이 시점에는 아직 앱에서 셀로 돌아온 것이 아닙니다. OS 서비스에서 앱으로 돌아온 것입니다.

첫 문자의 pointer는 ECALL 전에 이미 0x4025가 되어 있습니다. trap frame에 a5가 보존되므로 돌아온 뒤 LBU a0,0(a5)는 H 다음의 e를 읽습니다. 만약 handler가 a5를 복원하지 않으면 UART 출력 함수가 사용한 임시 값으로

pointer가 바뀌어 문자열 순회가 망가질 수 있습니다. 작은 예제가 register 보존 규칙도 시험하는 이유입니다.

MRET 이후의 앱 PC는 0x4018 이고 ra는 여전히 셸 복귀 주소입니다. 즉 복귀 방향을 결정하는 값이 두 종류 동시에 살아 있습니다. 파형에서는 CSR mepc와 register x1을 함께 보면 syscall 복귀와 최종 app return을 구분할 수 있습니다.

14.3 마지막 문자와 복귀

앱이 보내는 payload 문자열은 23문자입니다. 마지막 newline을 포함하며 끝의 NUL은 보내지 않습니다. newline syscall은 셸 출력 경로에서 CR과 LF 두 byte를 전송하므로 앱 메시지의 실제 UART 바이트 수는 24입니다. `running`, `returned`, 명령 `echo`와 `prompt`는 이 수에 포함하지 않습니다.

마지막 newline 이후 `lbu`가 NUL을 읽으면 `bne`가 성립하지 않습니다. 다음 PC는 0x4020 이고 `jalr zero,0(ra)`가 셸의 복귀 주소로 이동합니다. `rd=zero`이므로 새 반환 주소를 보존할 필요가 없습니다.

문자열 출력만 놓고 동적 앱 instruction 수를 계산해 볼 수 있습니다. 초기 설정 3개, 문자 23개마다 loop의 5개, 마지막 `ret` 1개이므로 $3 + 23 \times 5 + 1 = 119$ 개의 앱 instruction 시도가 정상 흐름에 나타납니다. 이 계산에는 `trap.S` handler, timer 중단에 따른 재시도와 UART polling 명령이 포함되지 않습니다. ECALL은 retire되지 않는 예외 명령이므로 이를 retirement counter 값과 동일시하지 않아야 합니다.

정상 반환 직후 셸은 `returned`를 출력하지만 OS가 앱의 모든 동작을 사후 검사한 것은 아닙니다. 앱이 형식과 ABI를 지켜 return한 사실을 관찰한 것입니다. 향후 앱별 결과 값을 검증하려면 지정된 출력·파일·메모리 signature 등을 추가로 검사해야 합니다.

14.4 역어셈블에서 보이는 가짜 명령어

앱 linker는 `.rodata`를 `.text` 출력 section에 합칩니다. 따라서 `objdump -d`가 문자열 영역까지 명령어로 해석하여 `c.flw`, `c.fld` 같은 뜻밖의 mnemonic을 보여 줄 수 있습니다. 이것만 보고 compiler가 압축 명령어나 부동소수점 명령을 앱에 생성했다고 판단하면 안 됩니다.

0x4020에서 반환하므로 정상 실행은 0x4024의 문자열을 instruction으로 fetch하지 않습니다. 데이터 주소, 제어 흐름, `xxd`의 ASCII 표시를 함께 보아야 합니다. 이 예제에서는 `--stop-address=0x4024`로 코드 부분만 역어셈블하면 혼동을 줄일 수 있습니다.

`objdump`는 binary byte와 section 정보를 보고 해석하며 C의 “이 위치는 문자열”이라는 원래 의도를 항상 충분히 아는 것은 아닙니다. `.text` 안에 상수를 모은 현재 script에서는 특히 code와 data의 경계를 직접 확인해야 합니다. 지원하지 않는 mnemonic이 보인다면 그 위치로 실제 제어가 도달하는지를 먼저 추적합니다.

반대로 “아마 데이터일 것”이라고 모든 이상한 instruction을 무시해서도 안 됩니다. 정상 branch의 target이 잘못되어 문자열로 떨어지면 실제 illegal instruction이나 예기치 않은 연산이 발생할 수 있습니다. ELF entry, label, branch target, 마지막 `ret`, memory dump를 묶어 판단해야 합니다.

14.5 한 명령어를 손으로 해석하기

0x00008067의 opcode는 1100111로 `JALR`, `rd`는 `x0`, `rs1`은 `x1`, immediate는 0입니다. 그래서 계산되는 target은 $(x1+0) \& \sim 1$ 입니다. `x1`의 ABI 이름이 `ra`이므로 어셈블러는 이 형태를 `ret`라는 pseudo-instruction으로 표현할 수 있습니다.

`ret` 와 `ecall` 은 서로 다른 복귀·진입 경로입니다. `ret` 는 일반 register `ra`, `mret` 는 CSR `mepc` 를 사용합니다. 이 둘을 구분하는 것이 다음 장의 중심입니다.

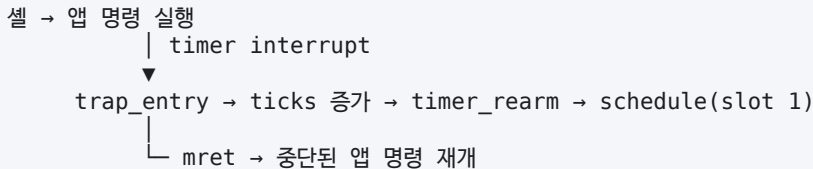
bit field를 계산하는 일반식은 $\text{opcode} = \text{word} \& 0x7F$, $\text{rd} = (\text{word} \gg 7) \& 31$, $\text{funct3} = (\text{word} \gg 12) \& 7$, $\text{rs1} = (\text{word} \gg 15) \& 31$ 입니다. JALR에서는 상위 12비트를 signed immediate로 해석합니다. 이 방법으로 `0x000780E7` 을 해석하면 `rd=x1, rs1=x15, immediate=0`이므로 `jalr ra,0(a5)` 임을 확인할 수 있습니다.

같은 연습을 ECALL의 `0x00000073` 에 적용하면 opcode SYSTEM, funct3=0입니다. 그러나 SYSTEM의 funct3=0인 모든 bit pattern이 ECALL은 아니므로 core는 instruction 전체를 비교합니다. MRET는 `0x30200073` 으로 별도 일치 조건을 갖습니다. major opcode 확인만으로 명령의 유효성을 결정할 수 없는 사례입니다.

15. 실행 중 인터럽트와 세 종류의 복귀

15.1 앱 실행 중 timer가 발생하면

앱 실행 중에도 기존 timer 설정은 유지됩니다. 앱은 별도 task가 아니므로 scheduler의 `cur` 는 계속 셀 slot 1입니다. timer interrupt가 발생하면 앱의 현재 PC와 레지스터가 셀 stack에 저장됩니다. scheduler는 그 frame을 slot 1에 기록하고 다시 같은 slot을 선택합니다.



“timer가 있으니 앱과 셀 명령 입력이 동시에 진행된다”는 의미는 아닙니다. 앱이 반환할 때까지 셀의 명령 parser 함수는 호출 흐름상 대기합니다. 무한 반복 앱에서 셀로 강제로 돌아오는 Ctrl+C 처리도 현재 없습니다.

timer가 앱의 LBU를 중단하면 core는 destination write를 막고 그 LBU의 PC를 mepc에 저장합니다. handler를 마친 뒤 같은 LBU를 실행하므로 pointer와 읽기 순서가 유지됩니다. 일반 RAM read는 부작용이 없지만 UART RX load에는 FIFO pop이 있으므로 3장에서 설명한 trap gating이 필요합니다.

다른 task를 선택하지 않아도 trap save/restore 비용은 발생합니다. timer가 있다는 사실과 실질적인 병렬 처리, 앱의 강제 종료 기능은 각각 다릅니다. 현재 OS는 한 core에서 하나의 instruction stream을 실행하고 interrupt 때 잠시 handler로 이동합니다.

15.2 ECALL, 함수 호출, context switch 비교

구분	진입 원인	복귀 주소의 핵심	복귀 방법
일반 앱 함수 호출	셀의 JALR	ra	앱의 ret
syscall	앱의 ECALL 예외	frame의 mepc=ecall PC+4	MRET
timer 처리	timer level과 enable	frame의 중단된 PC	MRET
다른 task로 전환	scheduler가 다른 frame 반환	선택된 frame의 mepc	그 frame 복원 후 MRET

하나의 앱 실행에는 수십 번의 syscall trap 복귀가 포함될 수 있지만 앱 자체의 최종 함수 복귀는 한 번입니다. trap이 발생했다고 항상 다른 task로 전환하는 것도 아닙니다.

일반 함수의 return은 caller의 호출 지점으로 돌아가지만, task switch는 서로 관련 없는 함수 흐름의 중간으로도 이동할 수 있습니다. frame에 전체 register와 재개 PC가 있으므로 C 호출 관계에 의존하지 않고 다른 실행 문맥을 선택할 수 있습니다. 이 점이 ordinary call stack과 scheduler의 context 관리가 다른 이유입니다.

현재 task 전환에서도 페이지 테이블을 바꾸는 주소 공간 전환은 없습니다. 모두 같은 physical memory를 사용합니다. 일반적인 process context switch는 이러한 register 상태 이외에 주소 공간·권한·kernel bookkeeping도 다룰 수 있습니다. 현재 문서의 context switch 설명은 해당 최소 구현 범위로 읽어야 합니다.

15.3 timer 재설정과 interrupt masking

timer IRQ는 `mtime >= mtimecmp` 동안 1인 level입니다. handler는 다음 compare 값을 미래로 옮겨 IRQ 조건을 해제합니다. FPGA 설정의 `TICK_CYCLES=125000`은 12.5 MHz에서 약 10 ms입니다. handler가 현재 시각을 읽어 다음 시간을 정하므로 이상적인 고정 위상 tick과는 차이가 있을 수 있습니다.

trap 진입은 `MIE=0`으로 만듭니다. 현재 handler는 중간에 이를 켜지 않으므로 timer handler가 자신을 중첩 호출하지 않습니다. syscall 안에서 UART가 바쁠 때 기다리거나 파일을 복사하는 동안 timer 처리는 지연될 수 있습니다. IRQ는 level로 남지만 지나간 tick마다 독립적인 요청이 쌓이는 queue는 아닙니다.

MIE를 끄는 것은 timer counter를 멈추는 것이 아닙니다. mtime은 계속 증가하고 compare 조건은 계속 평가됩니다. handler가 길어지면 MRET 직후 pending timer를 바로 받을 수 있습니다. interrupt enable과 peripheral의 시간 진행을 별도로 그려 보면 “interrupt를 켜는데 시간이 왜 흘렀는가”라는 혼동이 사라집니다.

현재 `timer_rearm()`은 직전 예정 시각에 `TICK_CYCLES`를 더하지 않고 handler가 읽은 현재 시각에 더합니다. 처리 지연이 tick phase에 누적될 수 있고, 여러 주기가 지났더라도 ticks를 한 번만 올립니다. ticks는 정확한 wall-clock을 복원하는 완전한 clock source가 아니라 handler 진입 횟수라는 점을 기억해야 합니다.

15.4 장시간 동작의 timer 제한

timer 내부 counter는 64비트지만 MMIO는 하위 32비트만 제공합니다. 현재 `mtimecmp`의 상위 word를 갱신하는 인터페이스도 없습니다. 하위 word가 wrap하는 시간은 다음과 같습니다.

$$2^{32} / 12,500,000 = \text{약 } 343.6\text{초} = \text{약 } 5\text{분 } 44\text{초}$$

이 경계 부근에서는 compare 재설정이 의도대로 미래를 가리키지 않아 interrupt가 반복될 수 있고, 경계 이후에는 상위 word 불일치로 IRQ를 해제하지 못할 수 있습니다. 따라서 장시간 켜 놓은 셀의 응답 정지를 단순 UART 접속 문제로만 해석하면 안 됩니다. 수업 실습은 이 제한을 알고 짧게 수행하며, 장시간 사용을 위한 설계 확장은 상·하위 timer MMIO와 안전한 64비트 갱신 규약입니다.

상위 word를 노출하는 것만으로 64비트 접근이 한 번에 원자적으로 되는 것은 아닙니다. RV32 CPU는 보통 상·하위를 여러 load/store로 다루므로 read 사이의 rollover와 compare 갱신 중 일시적으로 지난 deadline이 되는 경우를 처리해야 합니다. high-low-high 재확인 같은 읽기 규약과 안전한 compare write 순서를 함께 설계해야 합니다.

현재 한계를 실험할 때는 elapsed time, 마지막 UART 응답, reset 여부를 함께 기록하십시오. 입력이 멈춘 뒤 screen만 재접속하는 것은 timer 상태를 바꾸지 않습니다. SW20 reset은 timer 상태를 다시 시작하지만 정상 RAM disk는 보존합니다. 원인을 명확히 하기 전에 재구성하면 파일과 실행 증거까지 없어질 수 있습니다.

15.5 64비트 비교와 32비트 재설정의 불일치 예

낮은 word가 끝에 가까운 순간 `mtime=0x00000000_FFFFFFF0`이라고 가정하고, 설명을 단순하게 하기 위해 다음 interval을 `0x200`으로 잡습니다. C가 낮은 word끼리 더하면 `0xFFFFFFFF+0x200`의 `uint32_t` 결과는 `0x00000100`입니다.

값	정상적인 64비트 deadline	현재 low-word만 쓸 때
compare high	0x00000001	기존 0x00000000 유지
compare low	0x00000100	0x00000100

값	정상적인 64비트 deadline	현재 low-word만 쓸 때
compare 전체	0x00000001_00000100	0x00000000_00000100
현재 mtime과 비교	아직 미래	이미 과거

현재 timer는 64비트 unsigned 비교를 하므로 잘못 작은 compare 값에 대해 IRQ를 계속 올립니다. 낮은 word가 wrap하기 직전에도 이 상황이 생길 수 있으며, 이후 mtime high가 1이 되면 compare high 0으로는 미래 deadline을 표현할 수 없습니다. 단순히 32비트 덧셈이 wrap한다는 사실보다 전체 비교 폭의 불일치가 핵심입니다.

이 예에서 high word를 나중에 쓰는 기능을 추가하더라도 중간 상태에 IRQ가 순간적으로 걸리지 않도록 write 순서를 정해야 합니다. software와 peripheral 양쪽의 ABI를 함께 고치는 문제이며, 문서의 현재 구현에는 아직 그 변경이 적용되지 않았습니다.

16. ZCU104 실습 절차

16.1 호스트 명령과 rv> 명령 구분

`make`, `python3`, `screen`은 Linux 호스트 터미널에서 실행합니다. `ls`, `run hello.app`은 UART로 보이는 `rv>`에 입력합니다. 아래 code block은 어느 쪽 명령인지 문맥에 표시합니다. 프롬프트 문자열 자체를 입력할 필요는 없습니다.

UART 장치 경로는 현재 보드의 FT4232 채널 D입니다. 다른 케이블 일련번호나 USB 열거 순서에서는 달라집니다. `/dev/ttyUSB3`보다 `/dev/serial/by-id/...` 경로가 장치 식별에 유리합니다.

두 터미널을 쓰면 하나는 `host build·upload`용, 다른 하나는 UART `screen`용으로 역할을 정해 두는 것이 편리합니다. 다만 uploader가 실행되는 순간에는 `screen`이 포트를 놓아야 합니다. 별도 터미널 창을 사용한다는 사실이 별도 serial hardware를 사용하는 것은 아닙니다.

Linux shell의 `ls`는 PC의 directory를 보여 주고 `rv>`의 `ls`는 FPGA RAM disk를 보여 줍니다. 같은 문자열의 명령이지만 실행 주체와 파일 시스템이 다릅니다. `hello_app.bin`이 host에 존재하는 것과 `hello.app`이 MiniFS에 존재하는 것은 서로 자동 연결되지 않으며 UART upload가 그 사이를 잇습니다.

16.2 처음부터 준비하는 순서

먼저 UART를 사용하는 `screen`을 종료합니다. `screen` 화면에서 `Ctrl+A`를 누르고 손을 떼 다음 `K`, 확인 `y`를 누릅니다. 이미 detach했다면 `screen -ls`로 정확한 세션을 확인하고 `screen -S 세션ID -X quit`으로 그 세션만 종료합니다.

호스트에서 실행합니다.

```
cd /home/eulia/RISC-V
make hello-app
make vivado-zcu104-shell
make program-zcu104-shell
python3 scripts/upload_app.py \
  --file build/firmware/hello_app.bin --name hello.app --run
```

`make vivado-zcu104-shell`은 비트스트림 생성이 필요한 때 수행합니다. 이미 현재 소스의 `.bit`가 있으면 그 단계를 생략할 수 있습니다. 다운로드를 보드 상태를 초기화하므로 업로드보다 먼저 수행해야 합니다.

정상 응답의 예는 다음과 같습니다.

```
{
  "uploaded": "hello.app",
  "bytes": 60,
  "checksum": "000010dd",
  "ran": true,
  "app_output": "run hello.app\r\n\r\nrunning\r\n\r\nHello from loaded app!\r\n\r\nreturned\r\n\r\nrv> "
```

이 JSON은 호스트 스크립트의 보고서입니다. FPGA가 JSON을 출력하는 것은 아닙니다. `app_output` 필드에 실제 UART에서 수신한 앱 실행 응답이 들어 있습니다. 예제 크기·checksum은 소스와 compiler 결과에 따라 바뀔 수 있습니다.

처음 실습할 때는 각 단계의 결과를 하나씩 확인합니다. `make hello-app` 뒤에 ELF와 BIN이 생겼는지, Vivado 뒤에 `.bit`가 생겼는지, program 단계에서 올바른 장치를 선택했는지, upload report에 `uploaded`와 `ran`이 있는지를 봅니다. 앞 단계가 실패했는데 다음 명령을 계속 실행하면 오래된 산출물을 새 결과로 오해하기 쉽습니다.

UART가 여러 개 있으면 host uploader의 `--port`로 정확한 장치를 지정합니다. 현재 자동 선택은 해당 형태의 by-id 포트가 하나일 때를 전제로 합니다. board cable이 달라지면 안내문의 일련번호를 자신의 장치로 바꾸어야 하며, baud는 115200으로 맞춥니다.

16.3 UART에 재접속하여 실행

업로드 스크립트가 끝난 뒤 호스트에서 다음을 실행합니다.

```
screen /dev/serial/by-id/usb-Xilinx_JTAG+3Serial_46635-if03-port0 115200
```

빈 화면이면 Enter를 한 번 누릅니다. 이미 전송된 부팅 메시지와 프롬프트는 새 연결에서 다시 보이지 않을 수 있습니다. Enter가 새 프롬프트를 만들어 줍니다.

```
rv> ls
hello.app 72 bytes
rv> run hello.app
running
Hello from loaded app!
returned
rv>
```

이 단계에서 `make program-zcu104-shell`이나 `make test-zcu104-app`을 다시 실행하면 안 됩니다. 그 명령은 보드를 재구성하여 방금 저장한 RAM disk를 초기화합니다. 같은 파일의 재실행에는 `run hello.app`만 필요합니다.

성공한 uploader는 이미 프롬프트까지 읽고 종료합니다. 새 screen은 그 이후의 serial byte만 보므로 처음에 아무 글자가 없는 상황이 자연스럽습니다. Enter를 누르면 CR이 전달되고 빈 line 처리 후 prompt가 새로 출력됩니다. 이때 다시 boot banner가 안 나온다는 사실은 오히려 보드를 재시작하지 않았다는 상태와 맞습니다.

`run hello.app`을 여러 번 반복하면 매번 파일에서 payload를 다시 복사하고 호출합니다. 한 번 로딩된 함수를 그 자리에서 계속 호출하기만 하는 cache형 loader가 아닙니다. 이 동작은 앱의 `.data` 초기 상태를 다시 공급하는 효과도 가지지만 별도 heap·외부 장치 상태까지 초기화하지는 않습니다.

16.4 이미 셸이 실행 중일 때 앱만 바꾸기

회로·셸 펌웨어·메모리 배치를 바꾸지 않고 앱 C 코드만 바꿨다면 앱을 다시 컴파일하고 업로드하면 됩니다. 매번 FPGA를 합성할 필요가 없습니다.

```
# 먼저 screen을 종료한 뒤 호스트에서 실행
make hello-app
python3 scripts/upload_app.py \
  --file build/firmware/hello_app.bin --name hello.app --run
```

`--run`을 빼면 저장까지만 수행합니다. 같은 이름이 있으면 현재 업로드 경로는 파일 내용을 교체합니다. 이후 screen에서 Enter → `ls` → `run hello.app` 순서로 확인합니다.

새 메시지를 가진 앱을 같은 `hello.app` 이름으로 업로드하면 FS write가 그 파일을 교체합니다. 다른 이름으로 저장하면 table entry는 별도로 생기지만 실행 창은 여전히 하나입니다. `run first.app`, `run second.app` 을 차례로 실행할 수 있고, 두 파일을 동시에 실행하는 의미는 아닙니다.

앱 수정만으로 되는 범위에는 사용 ISA, entry 주소, syscall ABI, 크기·stack 제한을 유지한다는 조건이 붙습니다. 예를 들어 `.bss` 지원이나 새로운 syscall을 추가하면 앱만 교체해서는 안 되고 loader/kernel 변경을 반영한 셸 이미지를 먼저 만들어야 합니다.

16.5 screen의 종료·분리·중지

조작	의미	UART를 다른 프로그램이 써도 되는가
<code>Ctrl+A → K → y</code>	현재 serial window 종료; 유일한 window면 session 종료	해당 session이 종료됐으면 가능
<code>Ctrl+A → D</code>	화면만 detach	session이 포트를 계속 사용하므로 업로드 전 종료 필요
<code>screen -r 세션ID</code>	detached session 재접속	기존 session 사용
<code>screen -x 세션ID</code>	같은 session 화면 공유	별도 uploader와의 동시 사용 허용이 아님
<code>Ctrl+Z</code> 로 stopped된 foreground job	shell 작업 제어상 일시 정지	backend screen이 남을 수 있음

마지막 경우에는 원래 Linux shell에서 `fg %1` 등 해당 job을 foreground로 되돌릴 수 있습니다. 정확한 job 번호는 그 shell의 `jobs`로 확인합니다. 화면 공유와 같은 serial device를 별도 프로세스가 중복해서 읽는 것은 구분해야 합니다.

`screen -x`는 하나의 session이 이미 읽고 있는 화면을 여러 terminal에 보여 주므로 reader가 여러 개 생기는 것과 다릅니다. 반대로 별도의 screen을 같은 device로 열거나 uploader를 함께 실행하면 서로 응답을 나눠 읽을 수 있습니다. 이 경우 한쪽 화면에 일부 글자가 빠지거나 uploader가 marker를 찾지 못할 수 있습니다.

종료할 때는 `screen -ls`에서 확인한 특정 session을 대상으로 합니다. detach는 UI 연결만 끊으므로 “터미널 화면에서 빠져나왔으니 포트가 비었다”는 판단이 틀릴 수 있습니다. serial port를 사용하는 주체를 하나로 유지하는 운영 절차가 현재 host tool의 중요한 전제입니다.

16.6 증상에서 계층을 찾아가기

증상	먼저 해석할 내용	확인·대응
새 screen이 빈 화면	과거 UART 출력은 재생되지 않음	Enter로 새 <code>rv></code> 요청
<code>ls</code> 가 빈 목록	현재 FS에 등록된 파일 없음	업로드 후 재구성 여부 확인
<code>not an app file</code>	파일 없음, header 읽기 실패, magic 불일치를 같은 문구로 보고	<code>ls</code> 로 존재 확인, <code>.bin</code> 재업로드
<code>checksum mismatch</code>	업로드 stream과 요청한 합이 다름	UART 동시 접근·누락 확인
<code>truncated app</code>	header가 선언한 크기만큼 읽지 못함	파일·header 길이 검토
<code>invalid app size</code>	실행 허용 범위 밖	payload 크기와 linker 결과 확인
PANIC cause=00000002	illegal instruction 예외	panic PC와 ELF 역어셈블 대조

증상	먼저 해석할 내용	확인·대응
몇 분 뒤 응답 정지	timer low-word 경계 등 가능	약 343초 제한 확인, SW20으로 재시작
uploader가 응답 timeout	포트·보드·셸 입력 상태 확인 필요	screen 점유, baud, 중단된 upload 상태 점검

SW20 reset은 유효한 RAM disk를 보존하도록 설계되어 있습니다. 전원 재인가나 PL 재다운로드는 파일을 지우므로 같은 대응으로 취급하지 않습니다.

문제 해결은 물리 연결·clock/reset → UART byte → 셸 parser → 파일 형식 → CPU 실행 순서로 좁히면 좋습니다. `rv>`가 보이면 최소한 CPU·TX와 셸 시작 경로는 상당 부분 작동한 것입니다. `ls`에 파일이 보이면 업로드 후 FS metadata가 남아 있다는 증거이고, `running`까지 나오면 loader의 호출 전 검사를 통과했다는 증거입니다.

반면 `returned`가 안 보인다면 앱 무한 loop, illegal instruction, stack/ra 손상, timer 제한 등을 고려해야 합니다. 마지막 메시지와 PC·panic 정보를 수집하면 원인 범위를 줄일 수 있습니다. 서로 다른 계층의 증상을 모두 재다운로드 하나로 처리하면 재현 가능한 원인 분석이 어려워집니다.

17. 검증 방법과 관측 가능한 증거

17.1 host artifact 검사

소프트웨어 단계에서는 ELF32, RISC-V machine, entry 0x4000, section 크기, .bss 부재를 검사합니다. objdump 로 앱 코드가 지원 명령으로 되어 있는지 보고 xxd 로 payload와 문자열을 확인합니다. magic과 checksum을 통과한다고 이 모든 성질이 자동 증명되는 것은 아닙니다.

현재 예제에서 확인한 값은 다음과 같습니다.

항목	관측값
ELF class / endian	ELF32 / little-endian
entry	0x4000
.text 크기	0x3C, 60바이트; 문자열 포함
명령 부분	0x4000 - 0x4023, 36바이트
payload byte sum	0x000010DD
MiniFS 저장 크기	header 포함 72바이트

같은 소스라도 compiler 버전과 최적화에 따라 instruction 배치가 달라질 수 있습니다. 따라서 문서의 0x4014 같은 값은 해당 예제 artifact에서 검산하고, 자신의 결과가 달라졌다면 symbol과 disassembly를 기준으로 새로운 관측 지점을 잡아야 합니다. 계약인 entry 0x4000과 관측값인 내부 branch 주소를 구분하십시오.

재현 기록에는 source revision 또는 변경 내역, GCC 옵션, ELF entry, BIN 길이·합, 사용 bitstream의 SHA-256을 함께 남기는 것이 유용합니다. checksum은 payload 전송 확인용이고 SHA-256은 실험한 큰 artifact의 식별용입니다. 두 값은 역할과 강도가 다릅니다.

17.2 RTL 시뮬레이션이 확인하는 것

```
make sim-shell
```

현재 tb_shell.v 는 serial bit waveform을 uart_rx 에 입력하고 실제 rv32_soc 를 실행합니다. 셸 명령과 앱 업로드 후 다음을 검사합니다.

관찰 대상	검사 의미
written, cat 결과, ls, removed	shell parser와 MiniFS 서비스 연계
ramdisk 내용과 table entry	파일 저장·삭제의 내부 상태
uploaded	hex 수신·검증·파일 저장 성공 경로
0x4000 <= pc < 0x6000	CPU가 앱 실행 영역에서 실제 instruction fetch
mem[4096] == app_words[0]	첫 기계어가 올바른 실행 위치에 로딩됨
앱 메시지와 returned	앱 수행 후 셸 호출 흐름으로 복귀

testbench의 TX는 `uart_tx_ready=1`로 두고 SoC의 byte-level valid/data를 관찰합니다. 따라서 이 테스트만으로 실제 UART TX baud와 물리 pin 연결까지 검증했다고 할 수 없습니다. RX는 직렬 waveform을 사용하고 TX는 빠른 byte interface로 확인하는 혼합형 테스트입니다.

시뮬레이션 펌웨어는 tick을 2000클럭으로 줄여 사용합니다. FPGA 이미지의 125000클럭과 다르므로 테스트 실행 시간과 interrupt 횟수를 보드와 그대로 비교하지 않습니다.

testbench가 앱 영역 PC를 관찰하는 것은 단순한 UART 문자열 검사보다 강한 증거입니다. firmware 안에 같은 문구가 우연히 들어 있어도 그 검사만으로 앱 실행을 입증할 수는 없기 때문입니다. 앱 첫 word 비교는 payload의 로딩 주소와 byte 순서를 추가로 확인합니다. 여러 독립적인 관측을 조합하면 한 종류의 거짓 양성을 줄일 수 있습니다.

`$readmemh`에 image보다 큰 배열을 지정하면 남은 word가 파일에 없다는 warning이 나올 수 있습니다. SoC는 먼저 RAM을 0으로 채우고 실제 image만 적재하므로 이를 무조건 오류로 해석할 필요는 없습니다. 그러나 누락된 image나 잘못된 path 같은 warning은 실제 실패 원인이므로 메시지와 testbench 조건을 함께 확인해야 합니다.

17.3 실제 보드 검증

```
make test-zcu104-app
```

이 target은 `hello-app`을 준비하고, `test-zcu104-shell`을 통해 실제 PL을 재구성한 뒤 셸 파일 명령을 검사하고, 업로드 스크립트로 앱 저장·실행을 검사합니다. 셸 bitstream은 먼저 만들어져 있어야 하며 UART를 사용하는 terminal을 달아야 합니다.

2026-09-29 실제 보드 기록은 [ZCU104 안내](#)에 있습니다. `write/cat/ls/rm/ls` 검사는 176바이트의 응답을 수신하며 통과했습니다. 이어 60바이트 앱을 업로드하여 `running`, `Hello from loaded app!`, `returned`, `rv>`를 확인했습니다. 현재 확인한 당시 bitstream SHA-256은 다음과 같습니다.

```
14e8051181301004070c03b18de9537efd265c66a329f1dd85f4d84109b996dc
```

원본 UART capture는 `build/zcu104_shell/upload_capture.bin`입니다. 업로드를 다시 하면 이 파일은 새 capture로 바뀔 수 있으므로 실험 기록을 보관하려면 별도 이름으로 복사해야 합니다. 이 장의 보드 결과는 해당 시점의 기록이며, 문서 생성 자체가 새 보드 테스트를 뜻하지 않습니다.

실제 보드에서는 RTL 시뮬레이션이 생략한 pin 배치, UART baud, 클럭 분주, 케이블·host serial 경로가 함께 검증됩니다. 반면 FPGA 내부의 모든 register나 RAM을 자동으로 관측하는 것은 아닙니다. UART 성공과 내부 PC 파형은 서로 다른 계층의 증거이므로 한쪽을 다른 쪽의 완전한 대체로 여기지 않습니다.

테스트 target이 PL을 재구성한다는 사실은 실험 설계에도 영향을 줍니다. 기존 파일의 reset 보존을 시험하려는데 매번 재구성 target을 실행하면 다른 조건을 시험한 것이 됩니다. reset 종류, 기존 파일 유무, 업로드 순서를 기록해야 파일 시스템 유지성에 대한 결론이 유효합니다.

17.4 현재 자동 검사의 범위

host uploader는 실행 응답에 `running`, `returned`, 프롬프트가 있는지 검사합니다. 예제 문구 자체의 일치까지 모든 앱에 대해 강제하지는 않습니다. 보드의 예제 메시지는 실제 capture로 확인했습니다. RTL testbench는 그 예제 문구도 검사합니다.

현재 검사는 정상 경로 smoke test이며 잘못된 header, checksum 충돌, 최대 크기, FIFO overflow, stack overflow, 장시간 timer wrap, 악성 코드 격리까지 포괄하지 않습니다. 교재에서 “PASS”는 어떤 관측 조건을 통과했는지와 함께 해석해야 합니다.

추가 test를 설계한다면 normal path, boundary, invalid input을 나누는 것이 좋습니다. boundary에는 최소 4바이트, 마지막 허용 payload 크기, line buffer 경계, file table 32개가 들어갑니다. invalid input에는 없는 파일, 틀린 magic, truncated payload, 추가 trailing byte, 틀린 합이 들어갑니다. 각각 어떤 error와 메모리 변화가 기대되는지 먼저 정해야 합니다.

예를 들어 checksum 실패 test는 단지 오류 문자열뿐 아니라 pc가 앱 창에 진입하지 않는지도 확인할 수 있습니다. syscall 보존 test는 여러 register에 signature를 넣고 trap 후 유지 여부를 검사할 수 있습니다. 현재 자동 test가 이미 이런 모든 검사를 한다고 설명하지 않고, 후속 검증 항목으로 구분해야 합니다.

17.5 파형을 추가해 관찰할 신호

더 깊이 관찰하려면 다음 신호들을 testbench dump 대상으로 선택할 수 있습니다. 현재 tb_shell.v 자체에는 \$dumpfile/\$dumpvars가 없으므로 make sim-shell 만으로 새 shell VCD가 생긴다고 기대하지 않습니다.

신호/상태	확인할 관계
serial_in, rx_byte, rx_valid	serial frame이 한 바이트로 바뀌는 시점
dut.rx_wr, dut.rx_rd	CPU가 입력을 소비하는 속도
pc	셀 → 앱 → trap → 앱 → 셀의 제어 흐름
dut.daddr, dut.dwstrb, dut.dwdata	파일 저장과 실행 RAM 복사
core take_trap, CSR mepc / mcause	ECALL·timer 구분과 복귀 PC
tx_valid, tx_byte	syscall과 출력 byte의 대응

전체 RAM 배열을 모두 dump하면 VCD가 커질 수 있습니다. 처음에는 PC, trap, MMIO bus, mem[4096] 주변처럼 의미가 분명한 신호를 좁혀 관찰하는 것이 좋습니다.

파형은 rising edge를 기준으로 보십시오. combinational rd_data, next_pc가 잠시 변하는 동안 아직 register와 RAM은 바뀌지 않았을 수 있습니다. take_trap=1 인 edge에서 strobe가 0인지, 다음 PC가 mtvec인지, 이후 trap.S의 여러 SW가 frame을 만드는지를 순서대로 찾으면 제어와 data의 관계가 보입니다.

처음에는 ecall 한 번을 확대해 보고, 다음에는 전체 앱 실행 구간으로 축소하여 반복 구조를 확인합니다. serial waveform은 약 86μs 단위 byte, CPU는 80ns 단위 cycle이므로 같은 화면에서 두 시간 규모를 모두 읽으려 하면 복잡해집니다. 목적에 맞게 zoom과 관측 신호를 나누는 것이 중요합니다.

17.6 invariant를 이용한 검증 설계

invariant는 특정 입력 예제 하나보다 넓은 상황에서 유지되어야 하는 조건입니다. 다음은 현재 설계를 읽으며 만들 수 있는 assertion·관측 조건의 예입니다. 일부는 현재 testbench의 검사와 대응하고, 일부는 추가 검증 과제입니다.

조건	검사할 이유	현재 구현과의 관계
x0 read는 항상 0	기본 ISA 상태 보장	regfile의 조합 mux와 write 무시
trap 선택 edge에 dmem_wstrb=0	중단된 store의 중복 부작용 방지	core의 write gating

조건	검사할 이유	현재 구현과의 관계
core_trap이면 RX pop 없음	입력 byte의 손실 방지	SoC의 rx_pop 조건
정상 앱 fetch는 실행 창 안에서 시작	loader entry 계약	현재 test의 saw_app_pc와 연관
frame base는 16바이트 정렬 유지	C handler 호출 ABI	task stack과 128바이트 frame 크기
checksum/길이 실패 시 함수 호출 없음	검증 전 실행 방지	negative test로 보강 가능
앱 반환 뒤 SP와 callee-saved 값 유지	셀의 정상 실행 지속	별도 signature test로 보강 가능

assertion의 위치도 중요합니다. nonblocking assignment의 결과는 edge 처리 후 갱신되므로 testbench가 old state를 읽는지 new state를 읽는지 알고 작성해야 합니다. 단순히 signal 값이 한번 보였다는 사실과 명령이 해당 edge에 완료됐다는 조건을 구분합니다.

모든 조건을 한 번에 자동화할 필요는 없습니다. 우선 관측하려는 실패를 하나 정하고, 정상 상태에서 성립하는 조건을 쓴 뒤, 의도적으로 조건을 깨는 작은 변형에 test가 반응하는지 확인합니다. 검증 코드 역시 잘못 작성할 수 있으므로 test 자체의 민감도를 점검해야 합니다.

18. 현재 구현의 한계와 확장 설계

18.1 M-mode 앱의 권한

앱·셀·handler는 모두 M-mode이며 주소 공간도 같습니다. 앱은 현재의 하드웨어에서 kernel RAM, MMIO, RAM disk를 직접 접근하거나 CSR을 바꿀 수 있습니다. syscall 사용은 소프트웨어의 약속이며 하드웨어가 강제하는 접근 경계가 아닙니다.

APP1 검증은 파일 구조와 단순 합을 확인합니다. 파일이 유효한 형태라는 것과 코드가 안전하다는 것은 별개입니다. 수업에서는 작성 내용을 이해한 앱을 사용합니다. 임의 코드를 격리하려면 U-mode, 접근 보호, 잘못된 접근에 대한 trap, 별도 실행 문맥을 함께 설계해야 합니다.

RISC-V의 M-mode는 machine 수준의 제어 권한을 뜻합니다. 현재 앱이 ECALL을 쓴다고 그 외의 직접 MMIO나 CSR 접근이 hardware에서 막히는 것은 아닙니다. API를 사용하는 모범적인 코드와 권한이 제한된 코드는 다른 개념입니다. 교육 예제에서는 의도한 프로그램을 실행한다는 전제가 있습니다.

격리 설계에서는 잘못된 pointer를 사용하는 FS syscall도 고려해야 합니다. U-mode에서 직접 kernel memory를 못 쓰게 하더라도 handler가 검증 없이 user pointer를 따라가면 보호 경계를 우회할 수 있습니다. privilege, memory permission, syscall의 주소·길이 검사, fault 복구가 함께 설계되어야 합니다.

18.2 앱 종료와 오류 처리

정상 앱은 C 함수 반환으로 셀에 돌아옵니다. 무한 루프 앱은 셀 parser를 진행시키지 않으며 timer가 발생해도 같은 task로 복귀합니다. illegal instruction 등 처리하지 않는 예외는 panic()에서 멈춥니다. 앱 하나만 종료하고 셀을 복구하는 fault recovery는 없습니다.

이를 확장하려면 셀 task와 앱 task를 분리하고, 앱에 대한 소유 stack과 상태를 기록하며, 오류 발생 시 어떤 자원을 반환하고 어느 문맥을 복구할지 정의해야 합니다. 단순히 panic()의 무한 루프를 return으로 바꾸는 것만으로는 올바른 복구가 되지 않습니다.

앱이 반환하지 않을 때 host uploader의 timeout은 host의 기다림을 끝낼 뿐 FPGA 앱을 종료하지 않습니다. 셀 parser가 앱 호출에 묶여 있으므로 host에서 새로운 명령을 보내도 즉시 처리된다는 보장이 없습니다. 현재 recovery는 reset에 의존할 수 있고, 이는 process kill과 다른 동작입니다.

task 종료를 설계한다면 RUNNABLE/RUNNING/EXITED 같은 상태, kernel이 소유한 stack, exit status, 기다리는 부모 문맥을 정의해야 합니다. 또한 종료한 task의 frame을 scheduler가 다시 선택하지 않도록 해야 합니다. 반환 PC만 셀로 바꾸는 임시 처리로는 손상된 SPL나 resource 상태까지 복구할 수 없습니다.

18.3 메모리 시스템의 제약

코어의 memory interface에는 wait-state handshake가 없습니다. 조합식 instruction/data read 때문에 이 RAM을 동기식 BRAM이나 DDR로 그대로 대체할 수 없습니다. BRAM 기반 multi-cycle 또는 pipeline 설계로 확장할 때는 읽기 지연, pipeline stall, load 완료, trap의 정확한 commit 경계를 함께 구현해야 합니다.

unmapped instruction fetch는 현재 SoC에서 NOP를 돌려주고, 여러 잘못된 접근이 access fault로 보고되지 않습니다. 보호·오류 모델이 완전하지 않으므로 PC가 잘못되면 항상 명확한 fault message가 나온다고 가정할 수 없습니다.

동기식 BRAM은 주소를 제시한 사이클과 데이터가 나오는 사이클이 다를 수 있습니다. 현재 LW는 같은 사이클의 data를 바로 rd_data에 사용하므로, 메모리만 바꾸면 이전 값이나 잘못된 값을 쓸 수 있습니다. multi-cycle 설계에서는 요청·대기·완료 상태를, pipeline에서는 memory 응답과 write-back 제어를 명시적으로 넣어야 합니다.

stall은 현재 instruction의 완료를 기다리는 microarchitecture 동작이고 context switch는 OS가 다른 register/PC 문맥을 선택하는 동작입니다. 둘 다 실행이 잠시 멈춰 보일 수 있지만 저장해야 할 상태와 발생 주체가 다릅니다. stall 중 interrupt를 언제 받을지까지 결정하면 precise trap 계약으로 다시 연결됩니다.

18.4 운영체제 기능별 다음 단계

확장 목표	필요한 설계 요소	현재 학습 내용과의 연결
.bss 지원	header의 영역 정보, 범위 검사, zeroing	boot.S의 .bss 초기화 재사용
앱 인수	argc/argv 규약, 문자열·포인터 배치, 수명	ABI와 stack frame
앱별 stack	호출 trampoline 또는 별도 task context	trap frame·SP 복원
여러 앱 동시 실행	task 생성·종료·상태·스케줄링	일반 OS round-robin
U-mode 격리	privilege 전환, PMP 등 보호, pointer 검증	CSR·trap 경계
ELF loader	ELF/segment 파싱, 범위·길이 검사, entry 확인, zero-fill	고정 APP1 로딩의 일반화
임의 주소 로딩	PIC 규약 또는 relocation 적용	linker와 절대 주소
SD 카드 유지 저장	block device driver, 저장 완료·오류 처리	MiniFS와 장치 계층 분리
장시간 timer	64비트 MMIO와 일관된 read/write 순서	interrupt 재설정
cache 추가	instruction/data 동기화와 FENCE.I	store된 코드의 fetch 가시성

이 표는 현재 구현되어 있다는 설명이 아니라 후속 설계 과제입니다. 특히 ELF loader를 추가한다고 자동으로 프로세스 격리나 Linux 호환성이 생기는 것은 아닙니다. 실행 파일 해석, runtime ABI, 권한, 장치 환경은 각각 맞아야 합니다.

확장 순서는 요구하는 관측 결과로 정하는 것이 좋습니다. “초기값 0의 전역변수를 쓰는 앱”이 목표면 .bss 부터, “앱이 멈춰도 셸이 살아 있기”가 목표면 별도 task와 복구부터 다루는 방식입니다. ELF loader는 배치 정보가 많아지는 문제를 해결하지만 CPU 보호나 scheduler 문제를 대신 해결하지 않습니다.

각 단계마다 기존 API를 유지할지 version을 바꿀지도 정해야 합니다. APP2처럼 magic/version을 달리하면 loader가 지원하지 않는 형식을 명확히 거절할 수 있습니다. 한 field의 의미를 조용히 바꾸면 오래된 앱이 형식 검사를 통과하고도 잘못 실행될 수 있습니다.

18.5 파일 시스템과 프로토콜의 후속 과제

MiniFS는 연속 할당과 작은 고정 table을 사용합니다. crash consistency, 권한, directory, 파일 descriptor, 동시 접근 잠금은 제공하지 않습니다. 현재 syscall handler 동안 timer가 마스크되어 파일 작업이 하나씩 진행되지만, 나중에 선점 가능한 kernel이나 멀티코어를 도입하면 명시적인 동기화가 필요합니다.

업로드 프로토콜에는 취소, board-side timeout, packet sequence, 재전송, 강한 무결성 검사, 완전한 flow control이 없습니다. 큰 앱을 안정적으로 전송하려면 block 단위 ACK나 credit, checksum/CRC, 실패 시 임시 파일 처리 정책 등을 정해야 합니다.

SD 카드로 바꾸면 CPU의 load/store 한 번으로 disk byte가 즉시 제공되는 현재 전제가 사라집니다. block read/write 완료를 기다리는 driver와 buffer, 실패·재시도 정책이 필요합니다. 그 위에 같은 file table 개념을 유지할 수는 있지만 persistence가 생기면 중간 write와 전원 차단 of 일관성도 새로운 요구가 됩니다.

프로토콜의 ACK를 추가할 때는 “serial driver가 받았다”, “보드 FIFO가 받았다”, “파일에 commit했다” 중 어느 완료를 의미하는지 정의해야 합니다. 재전송으로 같은 block이 두 번 도착해도 결과가 같게 만드는 sequence 규약도 중요합니다. 현재 단순 byte stream은 이런 성질을 학습하기 위한 출발점입니다.

19. 수업 구성과 연습문제

19.1 4회 실습 수업 구성 예

회차	읽을 내용	실습	제출물
1	2 - 6장	top/parameter 연결, memory map, UART Enter 관찰	module 계층도와 byte 수신 경로
2	7 - 9장	syscall·trap frame·MiniFS layout 추적	ECALL 한 번의 PC·SP 변화표
3	10 - 15장	예제 ELF 분석, header 계산, 앱 실행	disassembly 주석과 파일/메모리 배치
4	16 - 18장	정상·오류 실험, 확장 설계	검증 기록과 설계 제안

보고서에는 실행한 명령, 관측한 주소·바이트, 관측값에서 도출한 결론을 연결하도록 합니다. UART 출력만 보고 메모리 보호까지 검증했다는 식의 범위 확대를 피합니다.

1회차에는 instruction과 data가 같은 RAM을 읽는 경로를 직접 그리게 하고, 2회차에는 ECALL 한 번에서 SP와 mepc가 언제 바뀌는지를 표로 제출하게 합니다. 3회차에는 앱의 첫 word·문자열·header의 서로 다른 위치를 실제 hex dump와 연결하고, 4회차에는 정상 결과 하나와 의도적으로 만든 오류 결과 하나를 비교합니다. 이렇게 하면 code를 읽는 능력과 실행 증거를 해석하는 능력을 함께 평가할 수 있습니다.

평가 예시는 구조·계약 설명 30%, 주소·명령어 계산 25%, 재현 가능한 실험 기록 30%, 한계·확장 분석 15%입니다. 장치 출력만 복사한 보고서보다 “어떤 조건을 바꾸었고 왜 이 결과를 기대했는가”를 설명하는 보고서에 높은 비중을 둡니다. 아래 문제는 정답 암기보다 실제 source와 연결하여 답하도록 구성했습니다.

19.2 기본 이해 문제

1. C의 `-DSHELL_MODE` 만 켜고 top의 memory size는 일반 OS 구성으로 두면 어떤 문제가 생길 수 있는가?
2. `SHELL_BUILD=0` 을 설정해도 셸 bitstream이 선택되는 이유는 무엇인가?
3. `hello_app.bin` 이 60바이트인데 `ls` 에는 72바이트로 보이는 이유를 설명하라. 실제 disk 할당 크기는 얼마인가?
4. `0x4001` 의 바이트를 저장할 때 unified RAM의 word 인덱스와 byte lane은 무엇인가?
5. 일반 함수 호출의 `ra` 와 trap의 `mepc` 를 구분하라.
6. `ecall` handler에서 저장된 `mepc` 에 4를 더하지 않으면 무엇이 반복되는가?
7. Enter를 누르면 비어 보이던 screen에 `rv>` 가 나타나는 이유를 RX와 TX 경로로 설명하라.
8. RAM disk의 파일 시작 주소를 그대로 함수 포인터로 호출하면 안 되는 이유는 무엇인가?

1 - 8번은 각 문장에 source 근거를 하나 이상 연결해 답하십시오. 예를 들어 1번에서는 단순히 “메모리가 부족하다”라고 쓰기보다 linker의 stack top과 RTL RAM 끝 주소를 비교해야 합니다. 5번은 두 register 이름을 나열하는 데서 끝내지 말고 누가 언제 쓰고 어떤 명령이 복귀에 사용하는지 적습니다.

오답에서 흔히 나타나는 혼동은 host 파일과 MiniFS 파일, 파일 offset과 CPU 주소, 앱 return과 MRET, screen detach와 종료입니다. 답안을 작성할 때 각 값의 소유 주체와 단위를 붙이면 많은 오류를 스스로 찾을 수 있습니다.

19.3 계산·코드 해석 문제

9. `STACK_WORDS=256` 이고 `trap frame`이 128바이트라면, `trap frame` 하나만으로 `stack` 전체의 몇 퍼센트를 사용하는가? 왜 남은 부분을 모두 앱 지역변수로 사용할 수 없는가?
10. `payload`가 `01 02 FE FF` 네 바이트인 실험용 파일의 APP1 header를 little-endian 바이트로 작성하라. 이 `payload`를 실제 실행할 필요는 없다.
11. `payload`가 1200바이트라면 APP1 파일 크기와 MiniFS block 수는 얼마인가?
12. 파일 table의 32 entry가 사용하는 공간과 두 block 예약 공간의 차이를 계산하라.
13. `hello_app.bin`의 첫 네 바이트 `B7 47 00 00`이 `.hex`에서는 왜 `000047b7`이 되는가?
14. timer 주기가 125000클록, CPU가 12.5 MHz일 때 tick 간격을 구하라. 하위 32비트 wrap 시간도 계산하라.
15. `.text`의 문자열을 `objdump`가 `c.flw`로 표시한다. 이 사실만으로 core가 F/C extension을 지원한다고 결론 내릴 수 있는가?

계산 문제는 최종 숫자뿐 아니라 식과 단위를 제시합니다. 11번에서는 `payload` 크기, header를 포함한 file size, 올림한 block 수, 할당 byte 수를 순서대로 써야 합니다. 13번에서는 byte 네 개를 word 하나로 조합하는 과정을 보여 주십시오. 14번에서는 Hz가 초당 cycle 수라는 점을 이용합니다.

문제 10의 임의 `payload`는 header 형식을 익히기 위한 데이터이며 실행 가능한 프로그램이라고 가정하지 않습니다. 형식상 최소 크기를 만족하는 것과 유효한 instruction으로 정상 종료하는 것은 다릅니다. 이 차이를 명시하는 것도 답안의 일부입니다.

19.4 실험 문제

16. 앱 문자열을 바꾸고 FPGA 재합성 없이 앱만 다시 업로드하라. 어떤 ABI·주소 계약이 그대로이기 때문에 가능한지 설명하라.
17. `--run` 없이 업로드한 뒤 screen에서 Enter, `ls`, `run`을 수행하라. host report와 board 출력을 구분하여 기록하라.
18. `write note.txt text`로 일반 텍스트 파일을 만들고 `run note.txt`를 입력하라. 어떤 검증에서 거절되는지 코드로 설명하라.
19. 앱을 업로드한 뒤 SW20 CPU reset을 수행하고 `ls`를 관찰하라. 이어 PL을 재다운로드하고 다시 비교하라. 두 번째 단계는 RAM 파일을 지우므로 실습 파일만 둔 상태에서 수행하라.
20. testbench에 PC와 trap 관측을 추가하여 `0x4014 → trap_entry → 0x4018` 경로를 찾고 timer trap과 구분하라.

실험 기록 양식에는 변경한 변수, 초기 상태, 실행 명령, 기대 결과, 실제 결과, 불일치 해석을 넣습니다. 19번의 reset 비교에서는 각 단계 전에 같은 파일이 존재하는지 확인하고, CPU reset과 PL configuration을 정확히 기록해야 합니다. 테스트 target이 내부적으로 재구성하는 경우를 놓치면 실험 조건이 달라집니다.

20번에서는 VCD의 단일 PC 값만 캡처하지 말고 ECALL 주소, `trap_entry`, 복귀 주소를 연속 시간으로 표시합니다. 같은 구간에서 `mcause`가 11인지 `0x80000007`인지 확인하면 `syscall`과 timer를 구분할 수 있습니다. 파형의 시간 축과 clock edge를 보고서를 읽는 사람이 확인할 수 있게 남기십시오.

19.5 설계 문제

21. APP1의 다음 버전에서 `.bss`를 지원하려면 어떤 header field와 검증 조건이 필요한가?

22. 앱 전용 stack을 도입하려고 SP만 바꾸면 어떤 문제가 생기는가? 셸의 복귀 SP와 ra를 어떻게 보존할지 설계하라.
23. UART RX FIFO overflow를 host가 감지하고 복구할 수 있도록 protocol을 설계하라.
24. 무한 반복 앱을 timer 기반으로 종료시키고 셸로 돌아오려면 scheduler와 task lifecycle을 어떻게 확장해야 하는가?
25. instruction cache를 추가한 후 간헐적으로 이전 앱이 실행된다면 loader와 CPU의 어떤 계약을 조사해야 하는가?

설계 답안은 새 자료구조만 제시하지 말고 기존 성공 경로와 실패 경로가 어떻게 바뀌는지 설명해야 합니다. 예를 들어 APP2에 bss_size를 추가하면 field 길이, 정수 overflow 검사, 실행 창 경계, zeroing 시점, 오래된 APP1 처리 정책까지 필요합니다. 추가하는 정보가 loader의 어떤 결정을 가능하게 하는지 연결하십시오.

여러 확장을 한꺼번에 구현하는 것보다 하나의 계약을 바꾸고 그 계약을 검증하는 작은 실험을 제안하는 것이 좋습니다. cache를 넣으면서 동시에 loader·scheduler·UART protocol을 전부 바꾸면 문제가 생겼을 때 원인을 분리하기 어렵습니다. 실험 가능성과 검증 조건도 설계 품질의 일부입니다.

19.6 ISA와 데이터패스 심화 문제

26. `0x000780E7` 에서 opcode, rd, funct3, rs1, immediate를 추출하고 PC가 `0x0A18` 일 때 다음 PC와 ra를 구하라. x15는 `0x4000` 이다.
27. RAM word `0x44332211` 의 offset 2에 `0xAA` 를 SB로 기록한다. write data, strobe, 결과 word를 구하라.
28. `rs1=0xFFFFFFFF` , `rs2=1` 일 때 SLT와 SLTU의 결과를 비교하고, branch의 BLT/BLTU와 연결하라.
29. immediate `0xFF0` 을 32비트로 확장하라. `addi sp,sp,imm` 에서 초기 SP가 `0x1C40` 이면 결과는 무엇인가?
30. PC `0x401C` 에서 `0x400C` 로 BNE한다. B immediate의 값, bit 12·11·10:5·4:1을 구하고 instruction field 배치를 설명하라.
31. `csrr t0,mcause` 와 `csw mie,t0` 의 실제 CSR 명령을 쓰고, rd가 x0인 경우 무엇이 버려지는지 설명하라.
32. UART TX store와 timer trap이 같은 사이클에 선택되었다. 왜 PC만 mtvec로 바꾸면 충분하지 않은가?
33. 46개 명령의 합계를 계열별로 다시 계산하라. `ret` , `li` , `csrr` 와 `syscall` 번호 10이 이 수에 어떻게 관련되는지 설명하라.
34. `SRA(0x80000000,1)` 과 `SRL(0x80000000,1)` 의 결과를 구하라. register shift amount 32는 어떻게 처리되는가?
35. 현재 9개 static instruction으로 이루어진 앱이 23문자를 출력할 때 앱 경로의 instruction 시도를 계산하고, 이 값으로 실행 시간을 바로 계산할 수 없는 이유를 설명하라.

26 - 31번은 bit 계산을, 32번은 edge에서의 부작용을, 33 - 35번은 ISA·ABI·동적 실행 비용의 구분을 평가합니다. 숫자를 compiler 출력과 대조할 수 있지만 먼저 손으로 계산한 식을 제시하십시오. 코드의 명령어 배치가 바뀐 경우 문제에서 주어진 주소를 기준으로 답하고 자신의 빌드 결과와 차이를 따로 적습니다.

20. 해설·용어·코드 찾아보기

20.1 기본 문제 해설

1번은 하드웨어와 소프트웨어 주소 계약 불일치입니다. 셸 linker의 `_stack_top=0x8000`, 앱 주소 `0x4000` 등은 8 KiB 구성의 실제 RAM 범위 밖입니다. 이미지 크기만 우연히 들어가더라도 스택과 실행 영역이 맞지 않습니다.

2번은 Tcl이 값이 아니라 변수 존재를 검사하기 때문입니다. C의 `#ifdef`도 정의 여부 검사이고 Verilog 조건식은 값 검사라는 차이가 있습니다.

3번은 `payload 60 + header 12 =` 논리 파일 72바이트이며 할당은 한 block 512바이트입니다. 4번의 word index는 4096, byte lane은 1이고 strobe는 0010입니다.

5번에서 `ra`는 JAL/JALR가 기록하는 일반 함수의 복귀 주소, `mepc`는 trap 시 CSR에 보존되는 PC입니다. 6번에서는 같은 ECALL을 다시 실행합니다. 7번에서는 CR을 RX가 수신하고 셸이 빈 command line을 처리한 뒤 TX로 새 prompt를 보내기 때문입니다.

8번에서 RAM disk는 현재 instruction port의 저장소가 아니고 file header도 붙어 있으며, 앱 내부 주소는 `0x4000` 기준으로 링크되어 있습니다. 세 이유를 모두 검토해야 합니다.

1번의 구체적인 실패 후보에는 첫 C 함수의 stack 접근, 앱 창의 store/fetch, 커널 image 범위 초과가 있습니다. 어느 것이 먼저 나타나는지는 image와 instruction 흐름에 따라 달라질 수 있으므로 원인을 하나의 UART 메시지로 단정하지 않습니다. 계약 불일치 자체가 핵심입니다.

5 - 6번은 세 저장 위치를 그려 답하면 명확합니다. 현재 PC는 실행 중 명령, CSR `mepc`는 trap 복귀용 hardware 상태, `frame.mepc`는 software가 보관·수정하는 값입니다. `ra`는 이와 독립된 일반 register입니다. `frame.mepc += 4`가 즉시 현재 PC를 바꾸는 것이 아니라 restore와 MRET를 거쳐 효력을 갖는다는 설명까지 포함하면 완전한 답입니다.

20.2 계산 문제 해설

9번은 $128/1024=12.5\%$ 입니다. 남은 공간에도 이미 존재하는 셸 함수 frame, 앱 호출 frame, handler와 하위 함수 frame이 함께 들어갑니다.

10번의 합은 $1+2+254+255=512=0x200$ 입니다. header는 다음과 같습니다.

```
41 50 50 31 04 00 00 00 00 02 00 00
```

11번은 파일 크기 1212바이트, 필요한 block 수는 올림한 3개이며 1536바이트가 할당됩니다. 12번은 실제 table 768바이트, 예약 1024바이트, 차이 256바이트입니다.

13번은 raw byte들을 little-endian 32비트 word로 해석한 결과입니다. 14번은 tick 10 ms, wrap 약 343.6초입니다. 15번은 데이터 바이트의 오해석일 수 있으므로 실제 PC 경로·section 배치와 함께 판정해야 합니다.

11번을 식으로 전개하면 `Nfile=1200+12=1212`, `blocks=(1212+511)>>9=3`, `Nalloc=3*512=1536`입니다. 마지막 block의 미사용 부분은 $1536-1212=324$ 바이트입니다. 이 여유가 생기는 현상을 allocation granularity에 따른 내부 단편화로 설명할 수 있습니다.

14번의 계산은 $125000 / 12500000 = 0.01 \text{ s}$, $4294967296 / 12500000 = 343.59738368 \text{ s}$ 입니다. compare 갱신 직전의 낮은 word wrap 때문에 문제가 정확히 한 순간에만 시작된다고 단정하기보다는 경계 부근의 지속적인 IRQ 가능성을 분석해야 합니다. timer의 전체 비교 폭과 MMIO 노출 폭이 다르다는 구조가 원인입니다.

20.3 설계 문제의 검토 기준

.bss 확장에서는 최소한 초기화 영역의 시작과 길이, 파일에 저장된 영역과 메모리에서만 필요한 영역의 구분, 정수 overflow와 앱 창 경계 검사가 필요합니다. 앱 stack 확장에서는 새 stack의 정렬뿐 아니라 기존 SP, 복귀 주소, callee-saved register, trap 발생 시 소유 문맥을 고려해야 합니다.

UART 확장에서는 sequence와 길이, ACK/재전송, timeout, 수신 buffer 용량을 연결해야 합니다. 앱 강제 종료는 현재 실행 상태를 버린 뒤 복원할 셀 문맥과 자원 소유 관계가 필요합니다. cache 문제에서는 copy 완료와 instruction fetch 동기화, cache invalidate 또는 일관성 처리, FENCE.I 지원을 조사해야 합니다.

21번에서 bss_start와 bss_size를 모두 파일에 저장할 수도 있고, payload 뒤에 연속 배치한다는 규칙으로 일부 field를 생략할 수도 있습니다. 어느 설계를 택하든 더하기의 overflow, 앱 창 초과, 파일에 없는 memory 요구량을 검증해야 합니다. loadable 영역과 zero-fill 영역이 stack을 침범하지 않도록 검사하는 것이 핵심입니다.

22번에서는 셀의 원래 SP를 어디에 보존하고 누가 복원하는지 명확해야 합니다. 새 stack에서 발생한 trap을 기존 task_sp가 어떻게 관리할지도 정합니다. 24번에서는 실행 취소 후 선택할 정상 셀 frame이 필요하며, 앱이 공유 kernel memory를 손상시킬 수 있는 현재 권한에서는 완전한 복구를 보장하기 어렵다는 한계도 언급해야 합니다.

16 - 20번의 실험 해설은 결과를 외우는 대신 조건을 비교하는 데 있습니다. 앱 문자열 수정은 ABI를 유지하면 앱 재업로드로 충분하고, 일반 text 파일 run은 header 검사에서 거절됩니다. 정상 FS에서 CPU reset은 파일을 보존하고 PL 재구성은 초기화합니다. ECALL 파형은 cause 11과 PC+4 복귀를, timer 파형은 cause의 최상위 bit와 같은 PC 재개를 근거로 구분합니다.

20.4 용어 정리

용어	이 문서에서의 의미
top module	합성 대상 회로의 최상위 구조
parameter	Verilog 회로 elaboration 시 정하는 설정
firmware	CPU가 부팅 후 실행하는 커널·셀 코드
bitstream	FPGA의 회로·메모리 초기 구성을 담은 데이터
ABI	바이너리 수준의 호출·레지스터·배치 약속
loader	파일을 검사하고 실행 메모리에 배치하는 코드
payload	APP1 header를 제외한 앱 이미지 바이트
MMIO	load/store 주소를 주변장치 register에 대응시키는 방식
syscall	앱·task가 OS 서비스를 요청하는 약속된 호출 경로
trap	exception·interrupt에 의해 handler로 제어가 이동하는 사건
context	재개를 위해 보존하는 PC·register·stack 관련 상태
task	scheduler가 frame과 stack을 관리하는 실행 단위

용어	이 문서에서의 의미
relocation	최종 배치 주소에 맞춰 주소 참조를 수정하는 처리
.bss	실행 전 0 초기화가 필요한 정적 저장 영역
warm reset	FPGA를 재구성하지 않고 CPU 상태를 다시 시작하는 reset

용어를 함께 묶어 보면 경계가 더 잘 보입니다. ISA는 CPU 명령의 의미, microarchitecture는 그 명령을 구현하는 내부 회로 구조, ABI는 compiler와 runtime이 공유하는 호출 약속입니다. loader는 그 ABI에 맞는 image를 메모리에 놓고 entry로 제어를 넘기는 software입니다. 한 층의 변경이 다른 층의 계약을 깨뜨리는지 항상 확인합니다.

또 다른 묶음은 주소, offset, index입니다. 주소는 CPU address space 안의 위치이고, file offset은 파일 첫 byte에서의 거리이며, array index는 자료형 원소 수를 기준으로 한 위치입니다. `APP_BASE+off`, `12+off`, `mem[4096]`은 관련되지만 단위와 기준점이 다릅니다. 디버깅 메모에 기준점을 함께 적는 습관이 도움이 됩니다.

20.5 코드 찾아보기

파일	중심 함수·모듈	읽을 때 확인할 질문
zcu104_top.v	<code>zcu104_top</code>	parameter가 RAM과 firmware에 어떻게 연결되는가?
build_zcu104.tcl	<code>synth_design</code>	top과 generic은 누가 지정하는가?
rv32_soc.v	<code>mem</code> , <code>ramdisk</code> , <code>RX FIFO</code>	파일 byte는 어떤 주소 디코더를 통과하는가?
rv32_core.v	<code>LOAD/STORE/JALR/SYSTEM</code>	byte 복사와 함수 호출이 어떤 명령으로 수행되는가?
rv32_csr.v	trap entry와 MRET	PC와 interrupt enable은 어떻게 보존되는가?
uart_rx.v	<code>IDLE/START/BITS/STOP</code>	한 byte의 완료는 언제 판단하는가?
uart_tx.v	<code>ready/valid</code> 와 <code>shift</code>	문자 출력이 핀 waveform으로 어떻게 바뀌는가?
boot.S	<code>_start</code>	C 실행 전에 무엇을 준비하는가?
trap.S	<code>trap_entry</code>	frame 저장·선택·복원 순서는 무엇인가?
kernel.c	<code>task_shell</code> , <code>shell_upload</code> , <code>shell_run</code> , <code>trap_handler</code>	사용자 입력에서 앱 실행까지 어떻게 이어지는가?
minifs.c	<code>fs_init</code> , <code>fs_write</code> , <code>fs_read_at</code>	file offset은 disk 주소로 어떻게 바뀌는가?
linker_shell.ld	kernel memory layout	stack과 앱 창은 왜 겹치지 않아야 하는가?
app.ld	APP origin과 entry section	raw image의 첫 명령을 어떻게 보장하는가?
hello_app.c	<code>app_main</code> , <code>putc_sys</code>	앱이 커널 함수 주소 없이 출력하는 방법은 무엇인가?
upload_app.py	host protocol	파일·checksum·UART 응답을 어떻게 다루는가?
tb_shell.v	serial input, PC/메모리 검사	실행을 문자열 이외의 어떤 증거로 확인하는가?

읽기 순서는 처음에는 top → SoC → core, 다음에는 boot → trap → kernel, 마지막에 app linker → host uploader → loader로 잡을 수 있습니다. 각 경계에서 입력과 출력 signal 또는 인수·반환값을 표시하면 전체 code를 한 번에 외우지 않아도 경로를 복원할 수 있습니다.

line 번호는 code 변경 때 움직일 수 있으므로 파일 이름과 module/function 이름을 함께 사용합니다. `rg -n 'shell_run|trap_handler|task_shell' firmware/kernel.c`처럼 symbol로 찾으면 현재 코드 위치를 빠르게 확인할 수 있습니다. compiler가 inline한 함수는 ELF symbol에 없을 수 있어 source와 disassembly를 같이 보아야 합니다.

20.6 참고 자료와 문서의 범위

프로젝트의 구현 상세는 위 소스와 [MiniFS 안내](#), [앱 로더 안내](#), [ZCU104 안내](#)를 기준으로 합니다. 기존 [전체 설계 매뉴얼](#)은 기본 RV32I datapath와 초기 OS를 더 넓게 설명합니다.

표준 ISA의 명령 의미는 [RISC-V RV32I 명세](#), trap/CSR의 기준은 [Machine-Level ISA](#), 명령어 동기화는 [Zifencei](#)를 참고합니다. register 호출 규약은 [RISC-V psABI](#), ELF entry와 binary 변환은 [GNU ld ENTRY](#)와 [GNU objcopy](#)를 참고했습니다. 열람일은 2026-09-30입니다.

이 core는 교육용 최소 구현으로, 표준 privileged architecture의 모든 예외·권한·CSR 동작을 구현한 것은 아닙니다. 문서에서 “현재 구현”이라고 명시한 동작은 해당 RTL·펌웨어의 동작이고, “확장”은 아직 추가해야 할 설계입니다. 실험 결과의 주소·크기는 해당 산출물의 관측값과 고정 ABI 계약을 구분해서 사용해야 합니다.

CSR의 read-modify-write와 immediate 형태는 [RISC-V Zicsr 명세](#)를 함께 참고하십시오. 표준에서 정의된 기능과 현재의 간략화된 구현을 비교하는 것이 목적이며, 지원한다고 적힌 mnemonic 개수만으로 conformance를 주장하지 않습니다. 하드웨어에서 어떤 예외·권한·주소 검사를 생각했는지까지 읽어야 software의 적용 범위를 판단할 수 있습니다.

이 확장판은 firmware나 RTL의 기능을 추가하지 않고 교재 설명을 확장한 것입니다. 새로운 확장 설계·실험 제안은 현재 구현·검증 결과와 구별되어 있습니다. PDF를 재생성할 때는 `make shell-guide-pdf`를 사용하고, 수정 후 페이지 수와 표·code block의 잘림을 확인하여 인쇄 교재의 형태를 유지합니다.

20.7 ISA 심화 문제 해설

26번에서 opcode는 0x67, rd는 1, funct3는 0, rs1은 15, immediate는 0입니다. 다음 PC는 0x4000, ra는 0x0A1C입니다. rd에 쓰는 주소는 jump target이 아니라 caller의 다음 instruction 주소입니다.

27번의 write data는 0xAAAAAAAA, strobe는 0100, 결과 word는 0x44AA2211입니다. 28번에서는 signed -1이 1보다 작아 SLT=1, unsigned 최대값은 1보다 커 SLTU=0입니다. 같은 입력이면 BLT는 taken, BLTU는 not taken입니다.

29번의 immediate는 -16, 32비트 표현은 0xFFFFFFF0이고 SP는 0x1C30이 됩니다. 30번의 offset은 -16이며 13비트 표현은 0x1FF0입니다. bit 12=1, bit 11=1, bits 10:5=111111, bits 4:1=1000, bit 0=0입니다. 이 조각을 B형의 [31], [7], [30:25], [11:8]에 각각 배치합니다.

31번은 CSRRS t0,mcause,x0와 CSRRW x0,mie,t0입니다. 후자에서 이전 CSR 값의 일반 register 반환을 버리지만 mie에 새 값을 쓰는 동작은 남습니다. 32번은 store가 이미 장치에 반영되면 같은 PC 재개 때 중복 출력할 수 있기 때문입니다. trap 선택 edge에 memory·register·CSR write와 RX pop 같은 부작용을 함께 억제해야 합니다.

33번은 10+9+5+3+6+2+2+1+1+6+1=46입니다. ret/li/csrr은 기존 명령어로 표현되는 assembler 문법이고 서비스 번호 10은 ECALL로 전달하는 OS 데이터입니다. 34번의 SRA 결과는 0xC0000000, SRL 결과는 0x40000000입니다. register shift amount 32는 하위 5bit가 0이므로 shift하지 않습니다.

35번은 정상 앱 흐름에서 $3+23*5+1=119$ 개의 instruction 시도입니다. 이 수에는 handler의 수많은 저장·복원·polling 명령이 없고, timer에 의한 재시도도 없으며 ECALL은 retire되지 않습니다. 따라서 119를 그대로 clock 수나 retirement 수로 사용하면 틀립니다. dynamic trace를 기준으로 각 종류의 비용을 구분해야 합니다.